

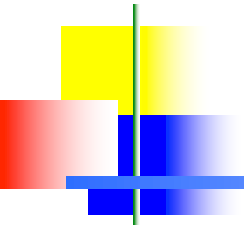
软件工程与方法

第5讲 面向对象方法

主要内容

1. OO的基本概念
2. OO的设计原则
3. 面向对象分析（OOA）
4. 面向对象设计（OOD）
5. OO方法介绍
6. 系统建模
7. 统一建模语言UML
8. 系统建模语言SysML





面向对象方法

1 面向对象的基本概念

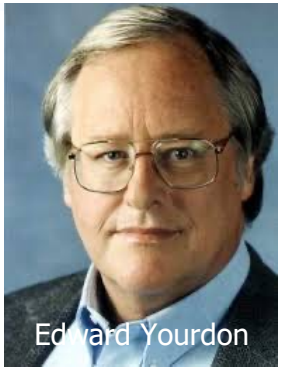
面向对象的基本概念—主要内容

- 对象与类
- 方法和消息



面向对象方法：


Peter Coad


Edward Yourdon

Peter Coad和Edward Yourdon提出用下列等式认识面向对象方法：

面向对象（Object-Oriented ）

= 对象（object）

+ 分类（classification）

+ 继承（inheritance）

+ 通过消息的通信（communication with messages）

Key Characteristics of an OO Methodology

1. Common methods of organization
2. Abstraction
3. Encapsulation
4. Inheritance
5. Polymorphism
6. Message communication
7. Association
8. Reuse

OO方法兴起的原因

- 从认知学的角度来看，面向对象的方法符合人们对客观世界的认识规律。
  - 传统程序设计方法的解空间结构与问题空间结构不一致。
  - 面向对向方法的分析、设计、实现的结果能直接映射到客观世界中系统的实体上，即解空间的结构与问题空间的结构是一致。
- 面向对象方法开发的软件系统易于维护，其体系结构易于理解、扩充和修改。
  - 对象的封装性很好地体现了抽象和信息隐蔽的特征。
- 面向对象方法中的继承机制有力地支持软件复用。

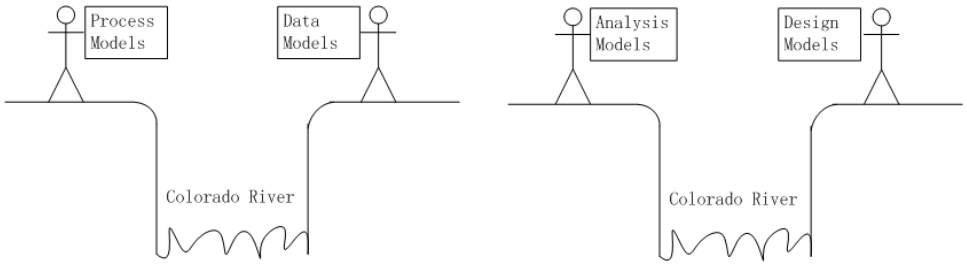
传统方法与面向对象方法

需求分析产生描述系统功能和问题域基本特征的综合文档。

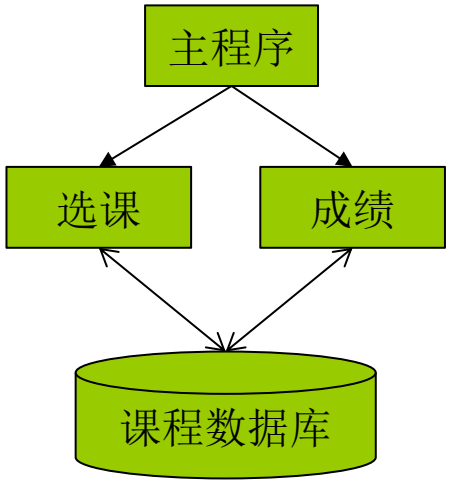
- 传统文档：面向功能，把系统看成一组功能，围绕输入－加工－输出的需求视角。
- OOA文档：把问题当作一组相互作用的实体，并确定实体间关系



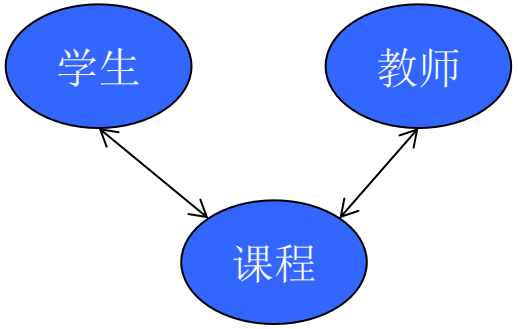
科罗拉多河（Colorado River）



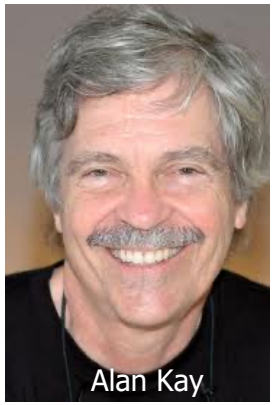
Two Classic Systems Development Problem



传统方法



面向对象的方法



Alan Kay总结了smalltalk中对象的5大基本特征：

① 所有的东西都是对象。

② 程序是一大堆对象的集合，他们通过消息传递，各个对象之间知道要做些什么。

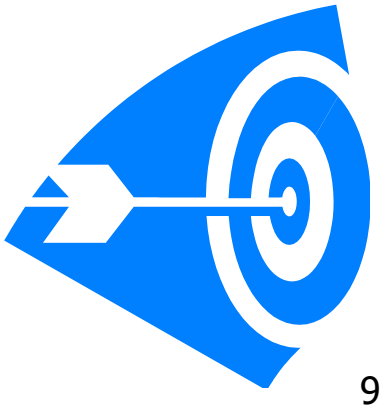
③ 每个对象都分配有自己的存储空间，可容纳其他对象。

④ 每个对象都有一个类型。

⑤ 同一类的所有对象能接收相同的消息。

对象（Object）

- 客观世界是由许多具体的事物或事件、抽象的概念、规则等组成的。因此，我们将任何感兴趣或要加以研究的事、物、概念都统称为对象。
- OO方法正是以对象作为最基本的元素，它也是分析问题、解决问题的核心。
- 对象是指一组属性以及这组属性上的专用操作的封装。属性（attribute）通常是一些数据，有时它也可以是另一个对象。



1) 对象是人们要进行研究的任何事物

- 从最简单的整数到极复杂的自动化工厂都可看作对象。
- 对象不仅能表示具体的实体，也能表示抽象的规则、计划或事件。
- 主要有如下对象类型：
  - 有形的实体：指一切看得见、摸得着的实物。
  - 作用：指人或组织所起的作用。
  - 事件：在特定的时间所发生的事。
  - 性能说明：如产品性能说明。

2)对象实现了数据与操作的结合

- 标识（Identity）——区别于其它对象标志，每一个对象有唯一的ID。在对象建立时，除有对象名外，还要有唯一且永久的标识符OID（Object Identify）。
- 状态（State）——当对象施加方法时对象的反映，通常用数据描述。
- 行为（Behavior）——说明这个对象能做什么，就是对象能进行什么操作，由方法或函数描述。

3)对象必须参与一个或一个以上的对象类

- 类（Class）是对象（Object）的抽象，对象是类的实例（instance），对象与类实例是同义词。
- 在大多数OO方法中，类也可以看是对象，从而实现了类与对象的统一。
- 创建类的类称为元类（Metaclass），此时，类可以作为元类的实例。

例：用“飞机”类的实例来说明类及对象的关系。

- 每架飞机都是一个具体的对象，如：歼10、歼15、歼20、轰6、运20。













（续）

- 抽取飞机共同的特性：
  - 凡是飞机都能在空中飞行，具有改变飞行方向、控制飞行高度和飞行速度的操作特性；
  - 凡是飞机都有机名、机型、飞行高度、飞行速度等数据，用以描述飞机的结构性能；
  - 飞机还有严格的飞行安全约束规则，如飞行条件、起飞与降落条件等。





对象类（Object Class）

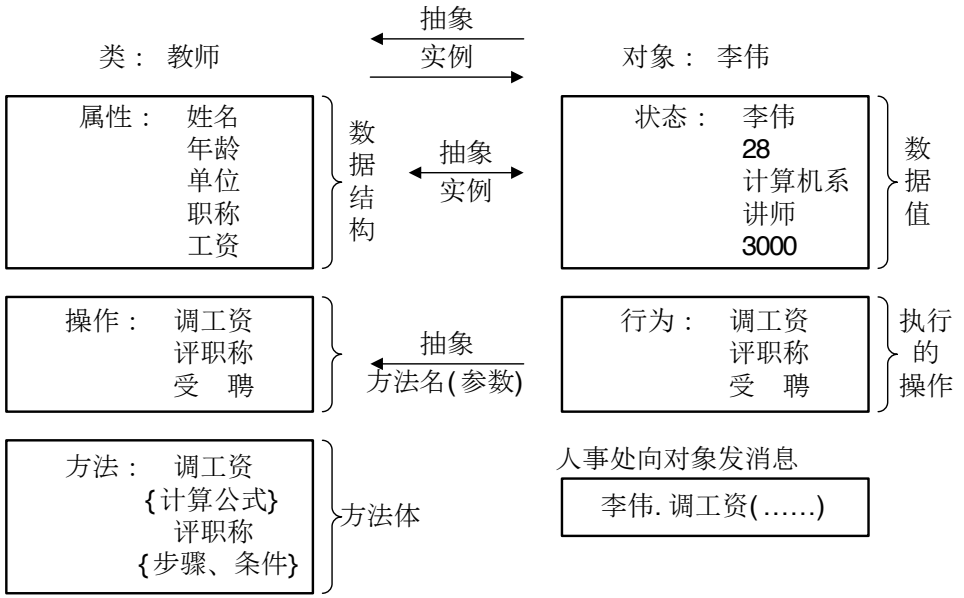
1)对象类的定义

- 将具有相同结构、操作，并遵守相同约束规则的对象聚合成一组，这组对象集合就称为对象类，简称为类（class）。
- 具体对类进行定义时，最低限度应包括如下内容：
  - 类名（class name）
  - 外部接口（external interface）
  - 内部表示（internal interface）
  - 接口的内部实现（implementation）

2)类说明（Class Specification）

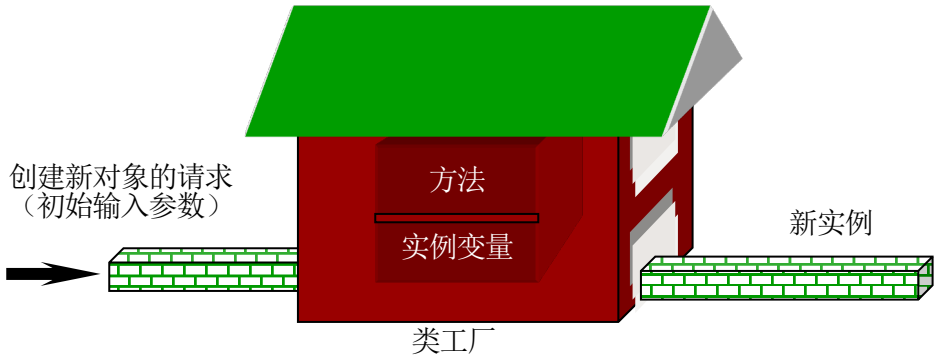
- 类说明也称为类的外部特性或称外部接口。
- 类说明是让用户了解对象是什么和能够做什么事，因而也是与用户沟通的“外部接口”或界面。
- 通常包括：
  - 公共成员（public member）变量
  - 公共成员函数

例：类与对象



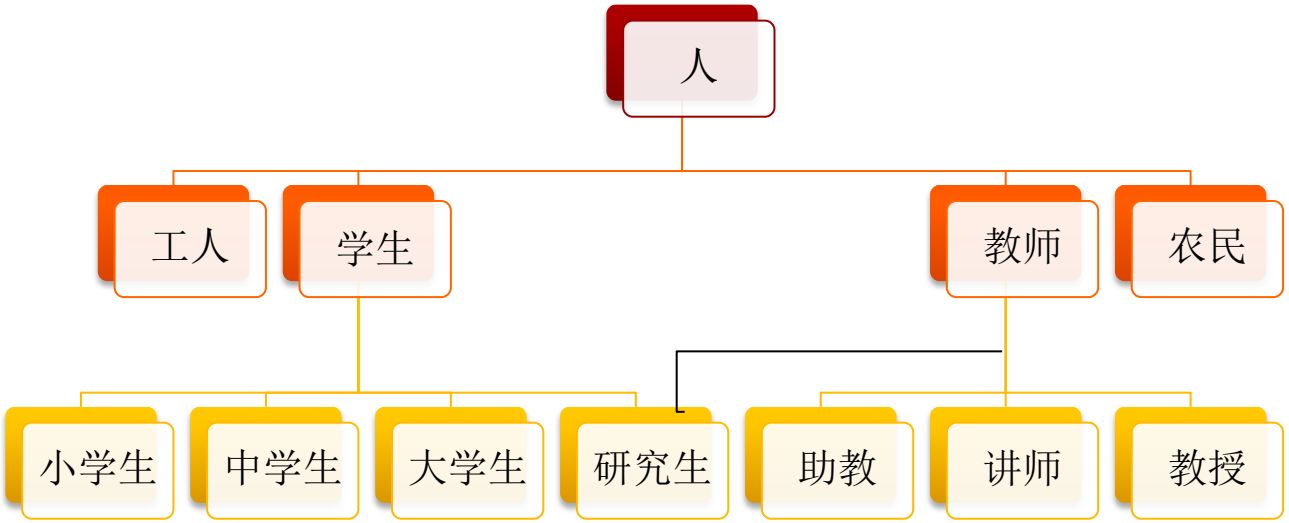
3)类实现（Class Implementation）

- 类实现是类的内部表示及类说明的具体实现方法。
- 类实现对用户隐藏。
- 对应于每一个操作符，都应在内部实现中找出具体的实现这一操作的程序模块。



4) 类层次与类格（Class hierarchy and Class lattice）

- 类继承导致类之间可能存在层次结构或格结构。
- 类层次结构特点是每个子类只有一个超类。
- 类格结构的特点是一个类至少存在一个子类有一个以上的超类。



5)对象类概念的精髓

- 对象类最鲜明的特色是将数据的结构与数据的操作者封装在对象类中。
- 实现了类的外部特性与类实现的隔离。
- 实现了对象使用者与开发者的分离。
- OO方法具有良好的模块化特性，进而为复杂的系统的分析、设计、实现提供有效的方法。

类实例及实例变量

1)类实例（Class Instance）

- 类所描述的对象就称为类的实例。如公司类，就可以有Lenovo 、Hauwei等实例。
- 对象可同时参与到多个类并作为这些类的实例。如张三既可以作为某个工厂的职工，同时又可作为武术协会的会员。
- 凡参与某个类的实例，必然共享该类的全部语义特性；如果还要加入实例自身的特性时，则应另外加补充说明。

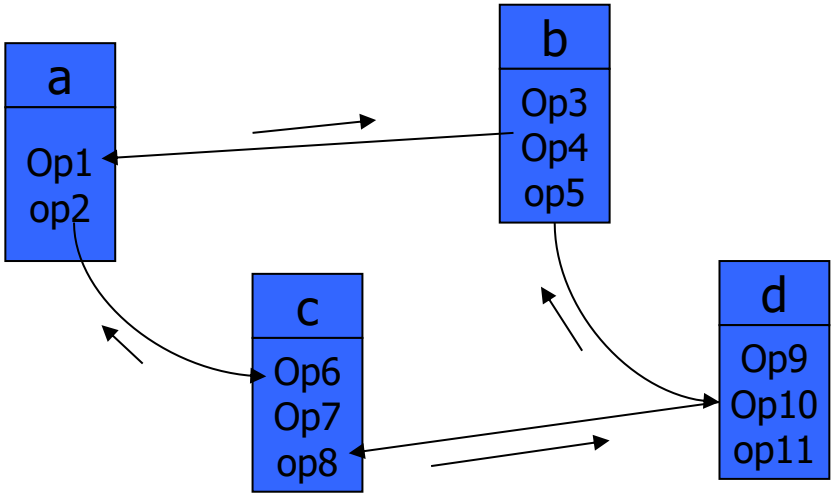
2)类实例变量（Instance Variable）

- 类实例变量为描述类的结构特性提供了统一的框架或模式，而每个实例都有自身的实例变量值。

方法和消息

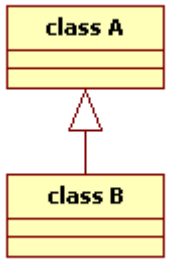
- 对象之间进行通信的结构叫做**消息**（Message），它包含接收对象去执行某种操作的信息。
- 类中操作的实现过程叫做**方法**（Method），一个方法有方法名、返回值、参数、方法体。
- 消息是对象之间相互联系和通信的唯一途径。消息由方法来处理。

- 消息内容包括：
  1. 对象名：接受消息的对象名
  2. 消息名：发送给对象的消息名（即对象名、方法名）
  3. 消息参数：消息内容

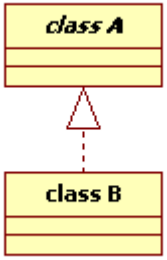


OO中常见的几种类之间的关系

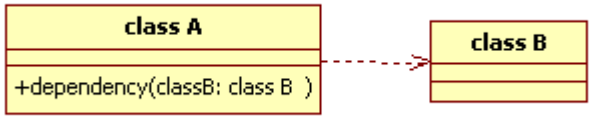
- **继承**
  - 继承指的是一个类（子类）继承另外的一个类（父类）的功能，并可以在子类中增加自己的新属性和功能。
  - 在UML类图表示中，用一条带空心三角箭头的实线表示继承关系，由子类指向父类。



- **实现关系**
  - 实现指的是一个class类实现interface接口（可以是多个）的功能，实现是类与接口之间最常见的关系。
  - 在UML类图中，用一条带空心三角箭头的虚线表示实现关系，从类指向实现的接口。

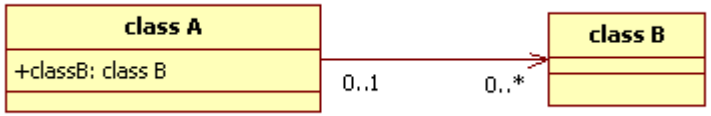


- **依赖关系**
  - 依赖就是一个类A使用到了另一个类B来实现某些功能，而这种使用关系是具有偶然性的、临时性的、非常弱的，但是类B的变化会影响到类A。
  - 在代码中，常表现为类B作为参数被类A在某个方法中使用。
  - 在UML类图中，依赖关系用由类A指向类B的带箭头虚线表示。



关联关系（续）

- 关联体现的是两个类之间语义级别的一种强依赖关系，这种关系比依赖关系更强，一般是长期性的。关联可以是单向、双向的。
- 在代码中常体现在，被关联类B以类的属性形式出现在关联类A中，也可能是关联类A引用了一个类型为被关联类B的全局变量。
- 在UML类图中，用由关联类A指向被关联类B的带箭头实线表示关联关系，在关联的两端可以标注关联双方的角色和多重性标记。



聚合关系

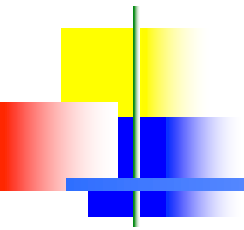
- 聚合是关联关系的一种特例，它体现的是整体与部分的关系，即has-a的关系。此时整体与部分之间是可分离的，它们可以具有各自的生命周期，部分可以属于多个整体对象，也可以为多个整体对象共享。比如学校和学生。
- 在代码中的体现，和关联关系是一致的，只能从语义级别来区分。
- 在UML类图设计中，用空心菱形加实线箭头表示聚合关系。



组合关系

- 组合也是关联关系的一种特例，体现的是一种contains-a的关系，这种关系比聚合更强，也称为强聚合。它同样体现整体与部分间的关系，但此时整体与部分是不可分的，整体的生命周期结束也就意味着部分的生命周期结束，比如人和人的大脑。
- 在代码中也与关联关系是一致的，只能从语义级别来区分。
- 在UML类图设计中，用实心菱形加实线箭头表示组合关系。

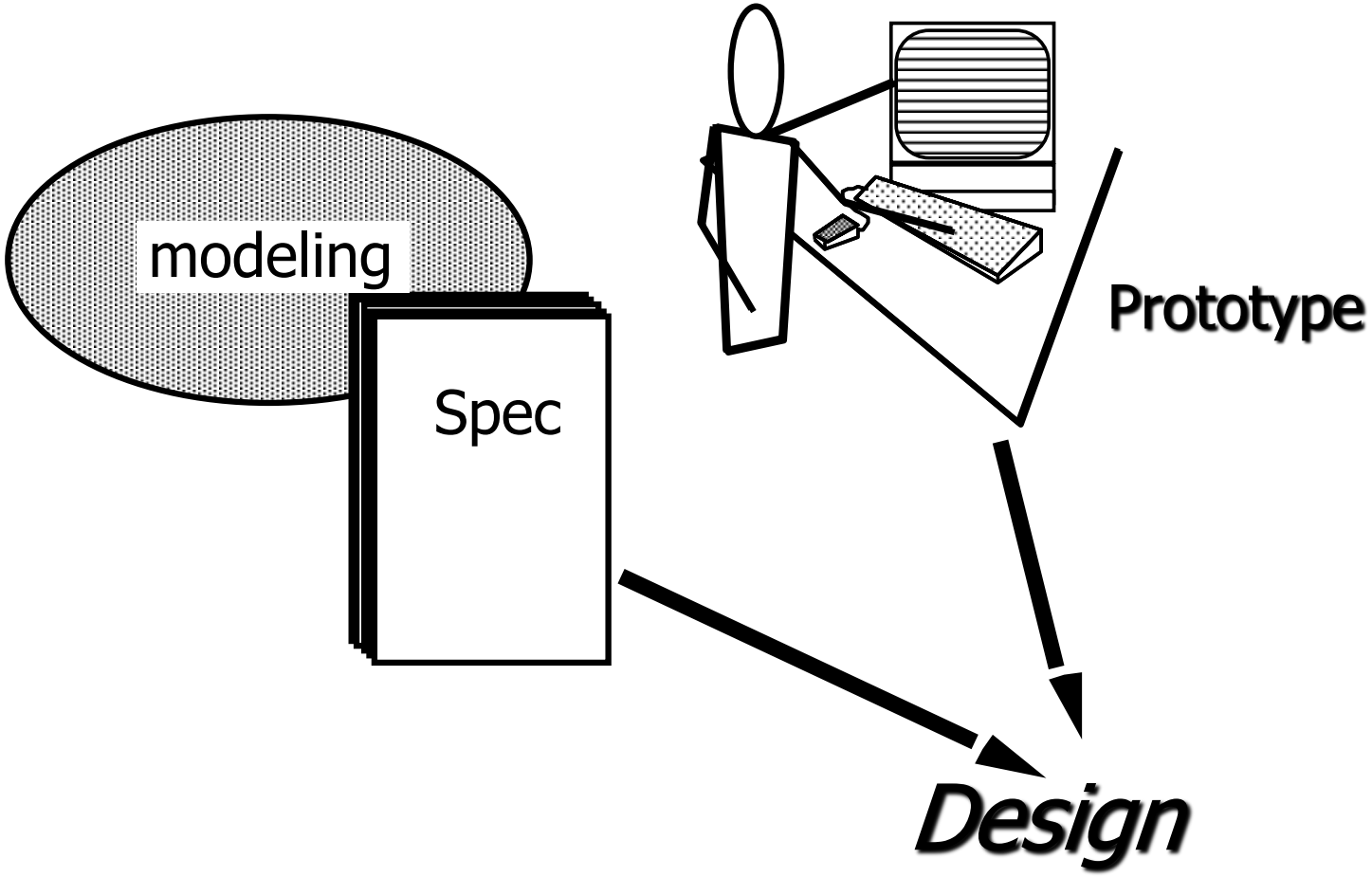




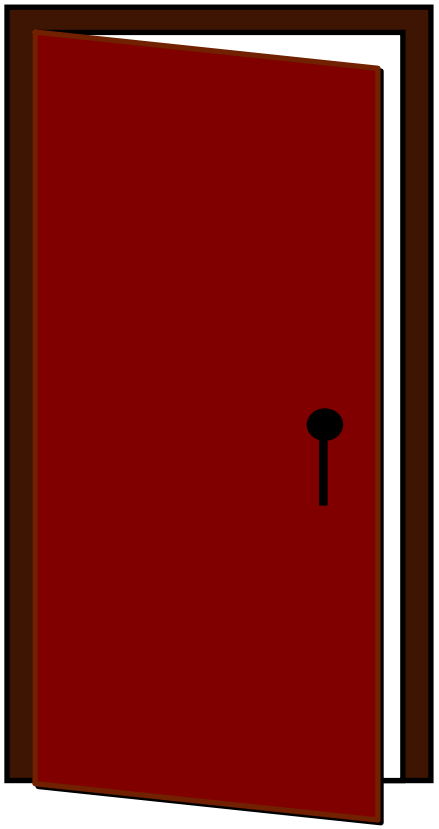
面向对象方法

2 面向对象设计原则

Where Do We Begin?



Data Abstraction



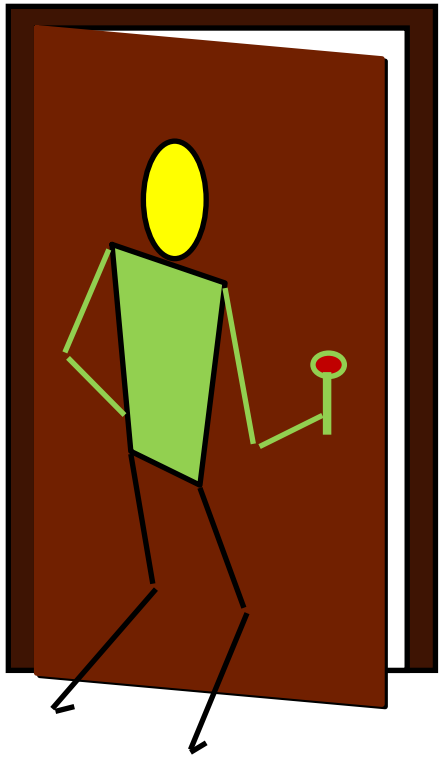


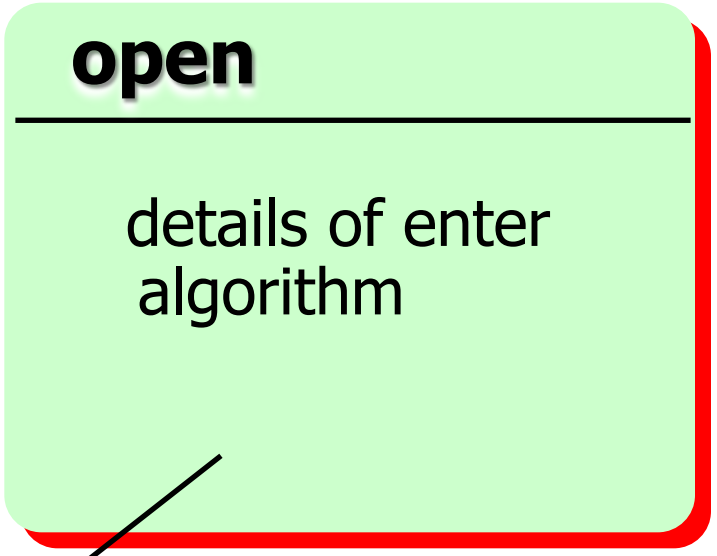
door

manufacturer
model number
type
swing direction
inserts
lights
  type
  number
weight
opening mechanism

implemented as a data structure

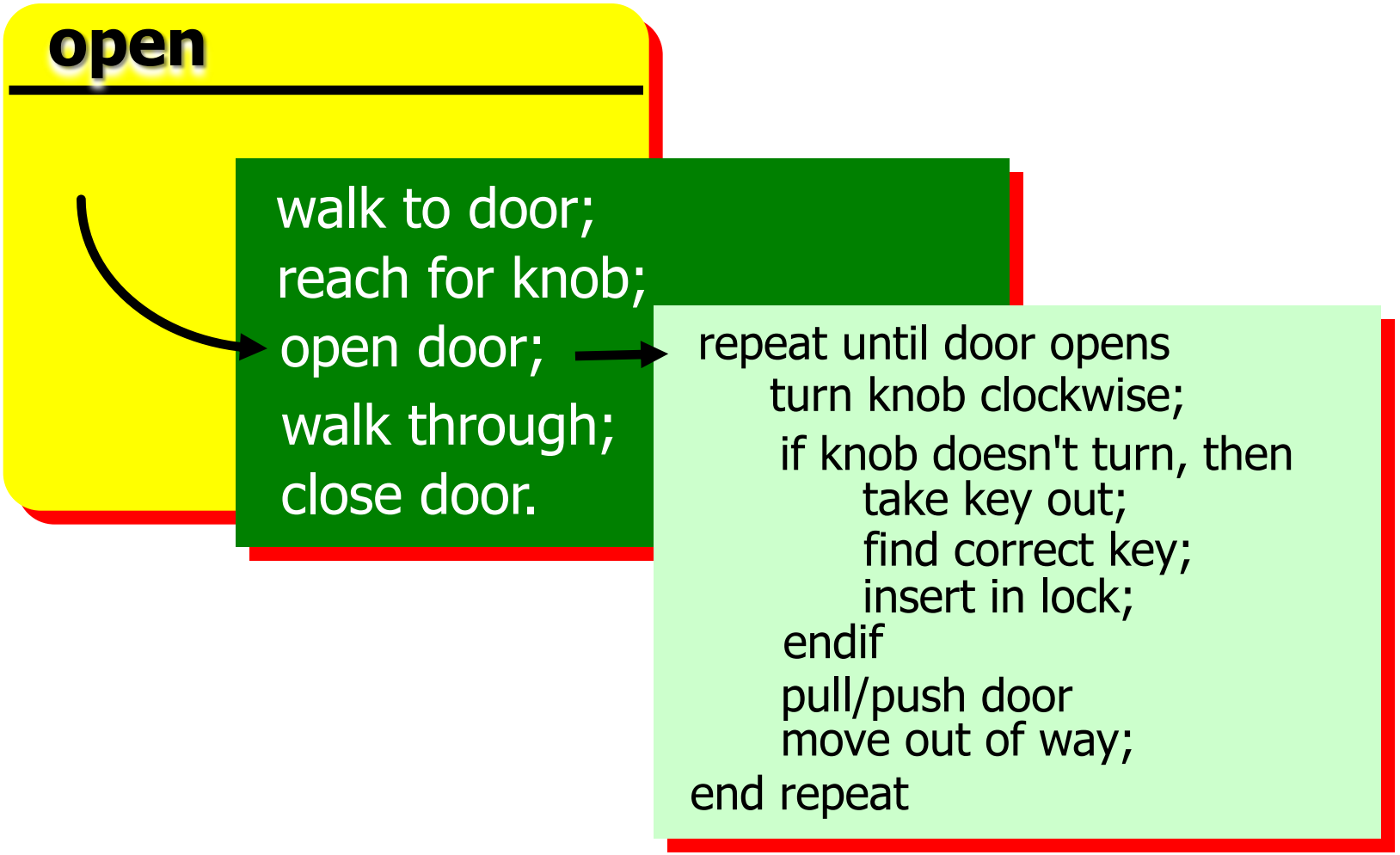
Procedural Abstraction





implemented with a "knowledge" of the object that is associated with enter

Stepwise Refinement



Fundamental Concepts

- Abstraction – a way to simplify complex practical problems, and a process to identify basic behaviors and eliminate irrelevant and tedious details.
  - Data
  - Procedure
  - Control
- Refinement - elaboration of detail for all abstractions
- Modularity - compartmentalization of data and function
- Architecture - overall structure of the software
  - Structural properties
  - Extra-structural properties
  - Styles and patterns
- Procedure - the algorithms that achieve function
- Hiding - controlled interfaces

面向对象的设计原理

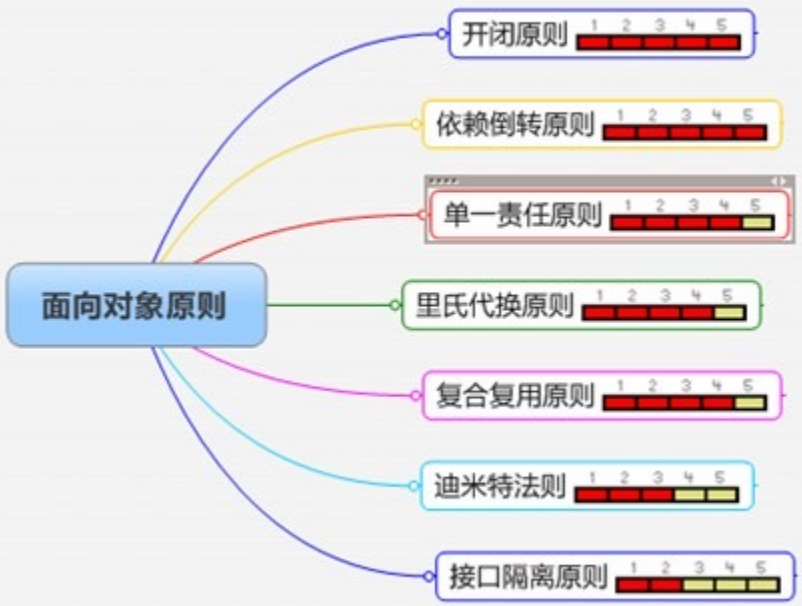
- **模块化**：对象就是模块
- **抽象**：面向对象支持过程和数据抽象，类是一种抽象数据类型，即规格说明抽象。
- **信息隐蔽**：对象封闭支持信息隐蔽
- **弱耦合**：不同对象之间相互关联的紧密程度。对象之间的两类耦合：
  ① 交互耦合（交换消息），使交互耦合尽可能松散，即减少消息中包含的参数个数，降低参数的复杂程度，减少消息数。
  ② 继承耦合。应该提高继承耦合程度。通过继承关系结合起来的基类和派生类构成了更大的模块。
- **强内聚**：共有3种内聚：
  ① 方法内聚，一个方法应该完成一个且仅完成一个功能。
  ② 类内聚，一个类应该只有一个用途，它的属性和方法应该是高内聚的。类的属性和方法应该全都是完成该类对象的任务所必需的。
  ③ 一般-特殊（继承）内聚，这种结构应该是对相应领域知识的正确抽取。紧密的继承耦合与高度的一般-特殊内聚是一致的。
- **可重用**：有两方面的含义：
  ① 尽可能使用已有的类
  ② 在设计新类时考虑未来的可重用性

什么是好的设计？

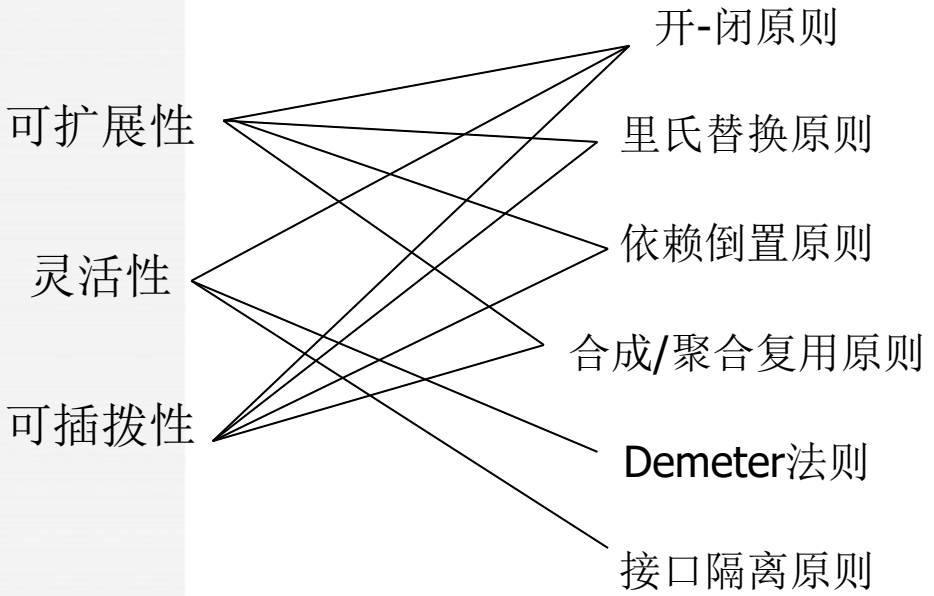
Peter Code提出的一个好的系统设计应该有如下特性：

- **可扩展性**（extensibility）：容易添加新功能。
- **灵活性**（flexibility）：代码修改平稳地发生。
- **可插拔性**（pluggability）：容易将一个类抽出去，同时将另一具有同样接口的类加入进来。

面向对象设计原则



目标与原则的关系



设计原则名称	设计原则简介	重要性
开闭原则 (Open-Closed Principle, OCP)	软件实体对扩展是开放的，但对修改是关闭的，即在不修改一个软件实体的基础上去扩展其功能	★★★★★
依赖倒转原则 (Dependency Inversion Principle, DIP)	要针对抽象层编程，而不要针对具体类编程	★★★★★
里氏代换原则 (Liskov Substitution Principle, LSP)	在软件系统中，一个可以接受基类对象的地方必然可以接受一个子类对象	★★★★☆
单一职责原则 (Single Responsibility Principle, SRP)	类的职责要单一，不能将太多的职责放在一个类中	★★★★☆
接口隔离原则 (Interface Segregation Principle, ISP)	使用多个专门的接口来取代一个统一的接口	★★☆☆☆
合成复用原则 (Composite Reuse Principle, CRP)	在系统中应该尽量多使用组合和聚合关联关系，尽量少使用甚至不使用继承关系	★★★★☆
迪米特法则 (Law of Demeter, LoD)	一个软件实体对其他实体的引用越少越好，或者说如果两个类不必彼此直接通信，那么这两个类就不应当发生直接的相互作用，而是通过引入一个第三者发生间接交互	★★★☆☆

面向对象的设计原则

- 开-闭原则（OCP）：对可变性封装
- 里氏替换原则（LSP）：如何进行继承
- 依赖倒转原则（DIP）：针对接口编程
- 接口隔离原则（ISP）：恰当的划分角色和接口
- 合成复用原则（CRP）：尽量使用合成/聚合，尽量不使用继承
- 迪米特原则（LoD）：不要跟陌生人说话
- 单一职责原则（SRP）：一个类有且只有一个改变的理由。

开闭原则

- 开闭原则定义：一个软件实体应当对扩展开放，对修改关闭。
- Ivar Jacobson：任何系统在其生命周期中都会发生变化。如果我们希望开发出的系统不会在第一版后就被抛弃，那么我们就必须牢牢记住这一点。
- Bertrand Meyer(1988)：软件组成实体（类、模块、函数等等）应该是可扩展的，但是不可修改。



开闭原则分析

① 在设计一个模块的时候，应当使这个模块可以在不被修改的前提下被扩展，即实现在不修改源代码的情况下改变这个模块的行为。

② 抽象化是开闭原则的关键。通过从抽象体派生，也可以扩展模块的行为功能。

③ 还可以通过一个更加具体的对可变性封装原则 （EVP, Principle of Encapsulation of Variation）来描述，这一原则要求找到系统的可变因素并将其封装起来。

④ 开闭原则总结：**面对需求，对程序的改动是通过增加新代码进行的，而不是改变原来的代码。**

开闭原则实例

- 某图形界面系统提供了各种不同形状的按钮，客户端代码可针对这些按钮进行编程，用户可能会改变需求要求使用不同的按钮。

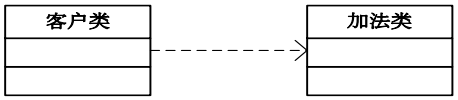


图1

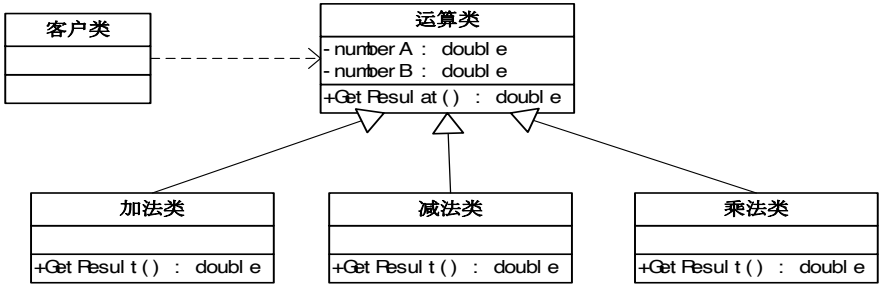
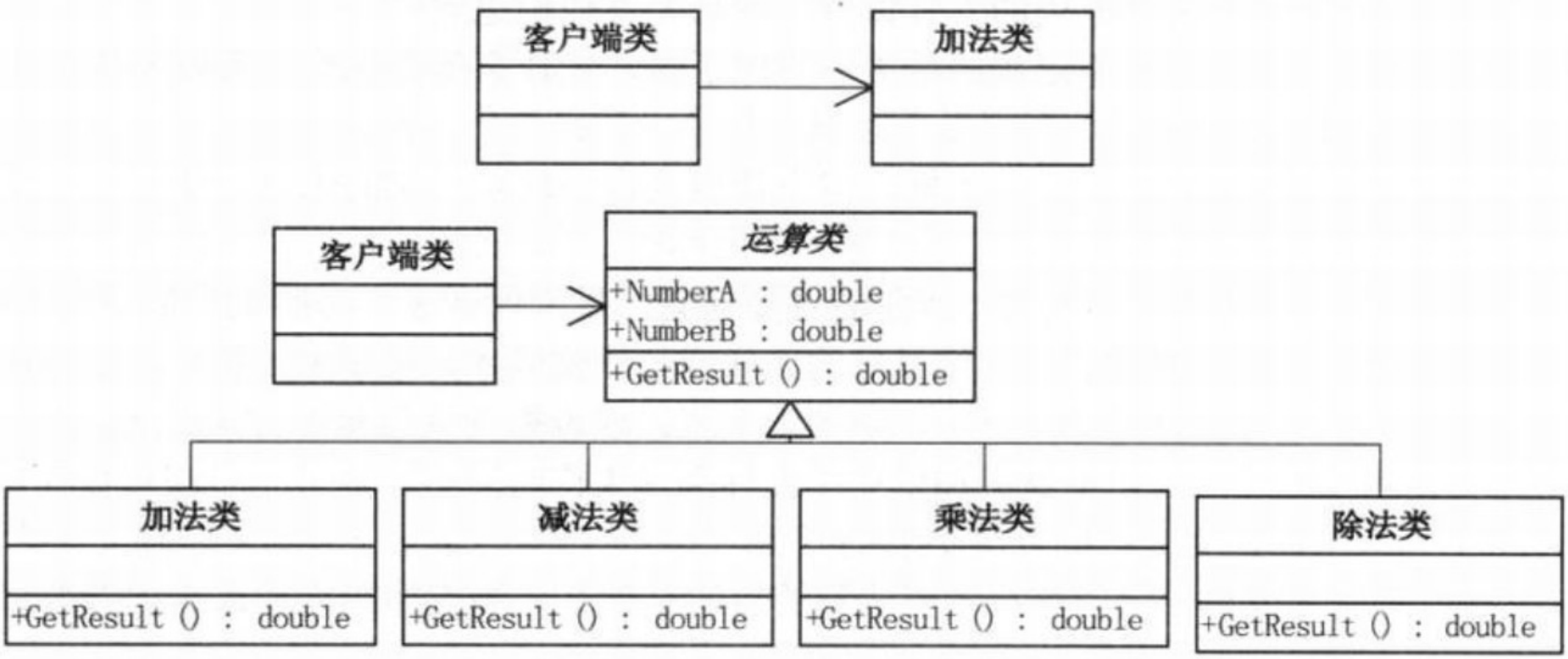


图2

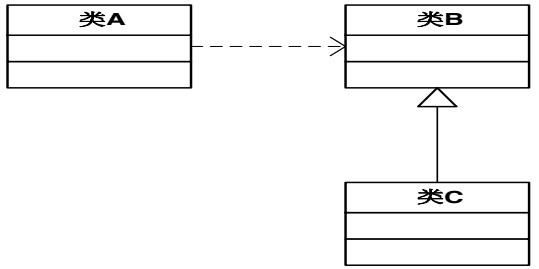
- 图1：客户端的一个方法直接调用加法类。如果想添加一个减法类，就得改变加法类中代码（用switch语句实现），这就违背了“开闭原则”。
- 图2：重构图1，添加一个运算类（加法类的父类），这样再添加减法类的时候就不用修改客户端类。

（续）


客户端类
加法类
客户端类
运算类
+NumberA : double
+NumberB : double
+GetResult () : double
加法类
+GetResult () : double
减法类
+GetResult () : double
乘法类
+GetResult () : double
除法类
+GetResult () : double

替换原则

- 子类应当可以替换父类并出现在父类能够出现的任何地方。这个原则是Liskov于1987年提出的设计原则。它同样可以从Bertrand Meyer 的DbC (Design by Contract) 的概念推出。



- 运用替换原则时，尽量把类B设计为抽象类或者接口，让C类继承类B（接口B）并实现操作A和操作B，运行时，类C实例替换B，这样我们即可进行新类的扩展（继承类B或接口B），同时无须对类A进行修改。

替换原则的定义

Liskov（ 1987）的定义：

I. 如果对每一个类型为S的对象o1，都有类型为T的对象o2，使得以T定义的所有程序P在所有的对象o1都代换成o2时，程序P的行为没有变化，那么类型S是类型T的子类型。
II. 所有引用基类（父类）的地方必须能透明地使用其子类的对象。

替换原则分析

- 如果能够使用基类对象，那么一定能够使用其子类对象。把基类都替换成它的子类，程序将不会产生任何错误和异常，反过来则不成立；如果一个软件实体使用的是一个子类的话，那么它不一定能够使用基类。
- 里氏替换原则是实现开闭原则的重要方式之一，由于使用基类对象的地方都可以使用子类对象，因此在程序中尽量使用基类类型来对对象进行定义，而在运行时再确定其子类类型，用子类对象来替换父类对象。
- 总结：**子类型必须能够替换掉它们的父类型。**

替换原则实例

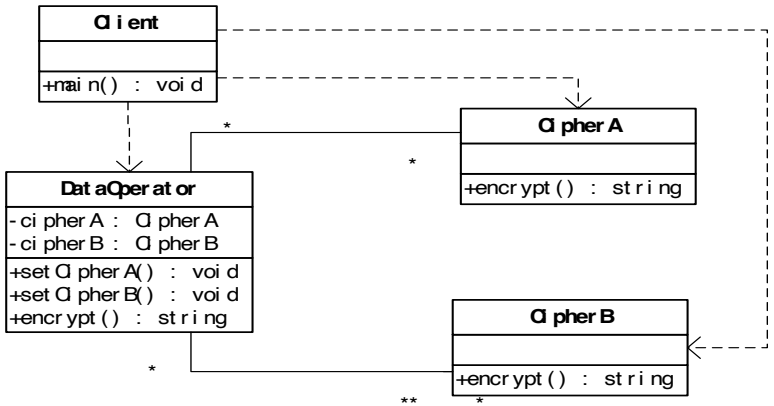


图1

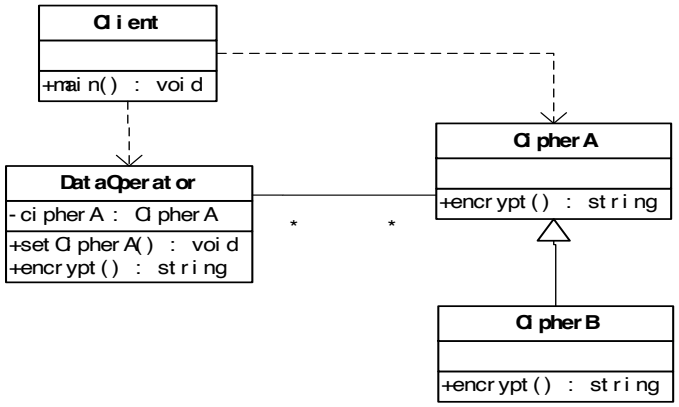


图2

- 某系统需要实现对重要数据的加密处理，在数据操作类DataOperator中需要调用加密类中定义的加密算法，系统提供了两个不同的加密类CipherA和CipherB，它们实现不同加密方法，在DataOperator中可以选择其中的一个实现加密操作。
- 图1为什到图2？因为如果需要更换一个加密算法类或者增加一个新的加密算法类，如将CipherA改为CipherB，则需要修改客户类Client和DataOperator的源代码，违背了开闭原则。现使用里氏代换原则对其进行重构，使得系统可以灵活扩展，符合开闭原则。

接口隔离原则

- 接口污染（interface contamination）：
  - 一个没有经验的设计师往往节省接口的数目，将一些功能相近或功能相关的接口合并，并将这看成是代码优化的一部分。
- 定义：从一个客户类的角度来讲：
  I. 一个类对另外一个类的依赖性应当是建立在最小的接口上的，客户端不应该依赖那些它不需要的接口。
  II. 使用多个专门的接口比使用单一的总接口要好。一旦一个接口太大，则需要将它分割成一些更细小的接口，使用该接口的客户端仅需知道与之相关的方法即可。

接口隔离原则分析

- ISP指使用多个专门的接口，而不使用单一的总接口。每一个接口应该承担一种相对独立的角色，不多不少，不干不该干的事，该干的事都要干。
- 使用ISP拆分接口时，首先必须满足单一职责原则，将一组相关的操作定义在一个接口中，且在满足高内聚的前提下，接口中的方法越少越好。
- 可以在进行系统设计时采用定制服务的方式，即为不同的客户端提供宽窄不同的接口，只提供用户需要的行为，而隐藏用户不需要的行为
- 总结：**类应该完全依赖相应的专门的接口。**

接口隔离原则实例

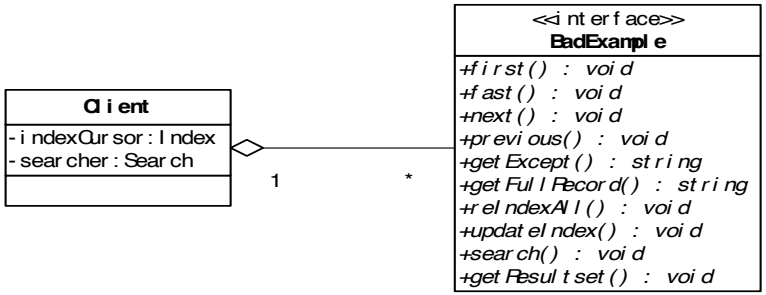


图1

- 图1为坏的例子

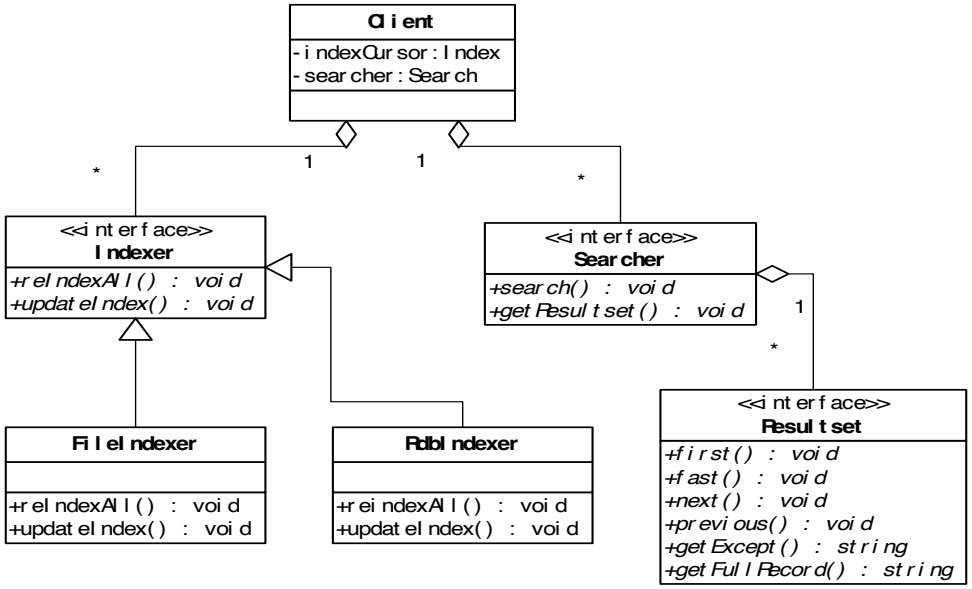
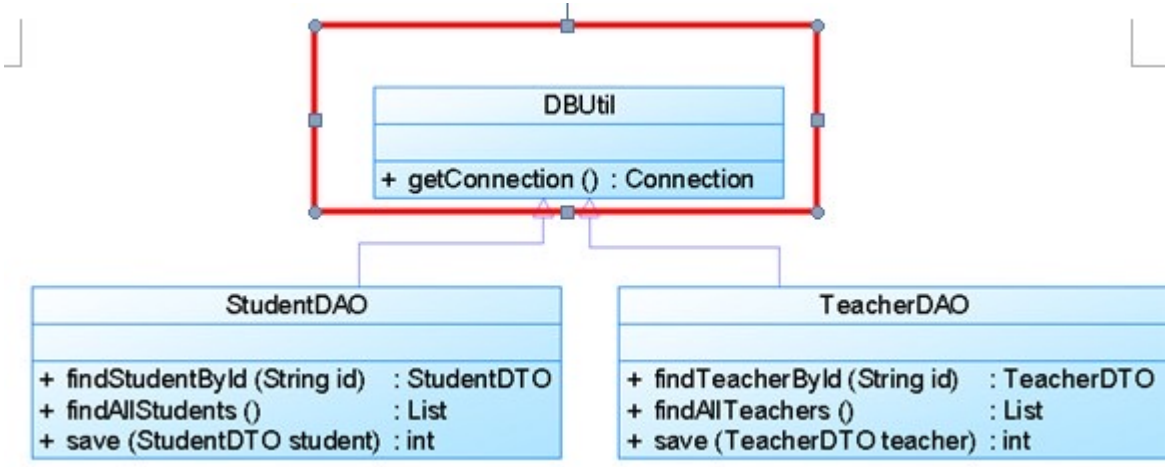


图2

- 图2为对图1使用接口隔离原则

合成复用原则

- 定义：尽量使用对象组合，而不是继承来达到复用的目的。



合成复用原则分析

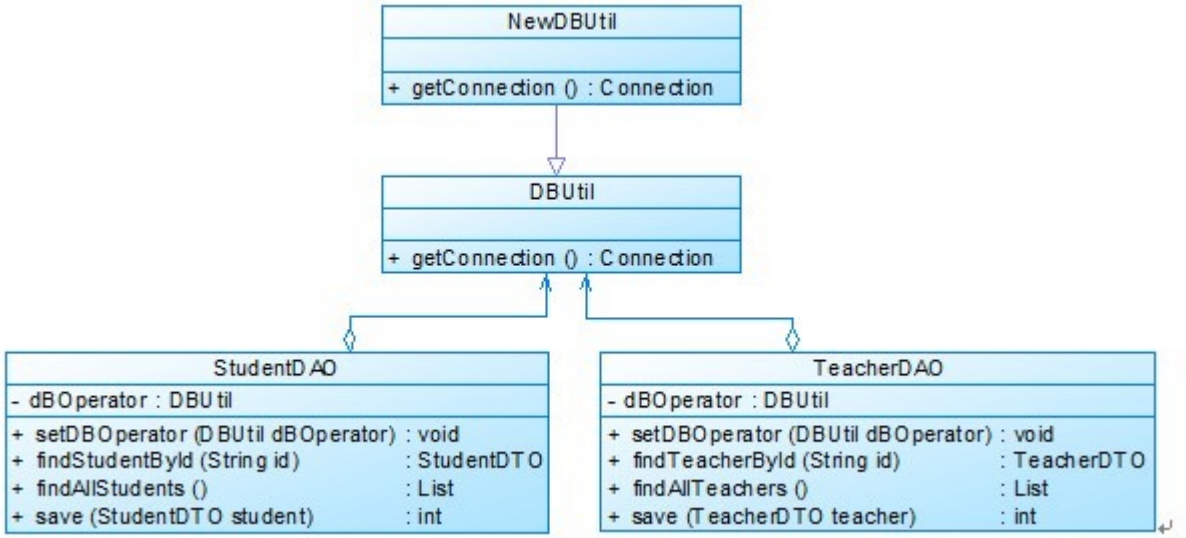
- CRP是指在一个新对象里通过关联关系（包括组合关系和聚合关系）来使用已有的对象，使之成为新对象的一部分；新对象通过委派调用已有对象的方法达到复用其已有功能的目的。简言之：要尽量使用组合/聚合关系，少用继承。
- 在OOD中，可以通过组合/聚合关系或通过继承在不同的环境中复用已有的设计和实现：
  a. 继承复用：实现简单，易于扩展。破坏系统的封装性；从基类继承而来的实现是静态的，不可能在运行时发生改变，没有足够的灵活性；只能在有限的环境中使用。（“白箱”复用 ）
  b. 组合/聚合复用：耦合度相对较低，选择性地调用成员对象的操作；可以在运行时动态进行。（“黑箱”复用 ）

（续）

- 组合/聚合可以使系统更加灵活，类与类之间的耦合度降低，一个类的变化对其他类造成的影响相对较少，因此一般首选使用组合/聚合来实现复用；其次才考虑继承，在使用继承时，需要严格遵循里氏代换原则，有效使用继承会有助于对问题的理解，降低复杂度，而滥用继承反而会增加系统构建和维护的难度以及系统的复杂度，因此需要慎重使用继承复用。

- 总结：**类中应用，尽量使用对象组合而不是用继承来达到复用的目的。**

合成复用原则实例



- 因为如果需要更换数据库连接方式，如原来采用JDBC连接数据库，现在采用数据库连接池连接，则需要修改DBUtil类源代码。如果StudentDAO采用JDBC连接，但是TeacherDAO采用连接池连接，则需要增加一个新的DBUtil类，并修改StudentDAO或TeacherDAO的源代码，使之继承新的数据库连接类，这将违背开闭原则，系统扩展性较差。

迪米特法则

- 定义：每个软件单元对其他的单元都只有最少的知识，而且局限于那些与本单位密切相关的软件单元。
- Lod法则：
  - 最少知识原则
  - 只与你直接的朋友们通信
  - 不要跟“陌生人”说话
- 朋友的条件：
  - 当前对象本身（this）
  - 以参量形式传入到当前对象方法中的对象
  - 当前对象的实例变量直接引用的对象
  - 当前对象的实例变量如果是一个聚集，那么聚集中的元素都是朋友
  - 当前对象所创建的对象

迪米特法则分析

I. LoD指一个软件实体应当尽可能少的与其他实体发生相互作用。这样，当一个模块修改时，就会尽量少的影响其他的模块，扩展会相对容易，这是对软件实体之间通信的限制，它要求限制软件实体之间通信的宽度和深度。

II. 狭义的LoD：可以降低类之间的耦合，但是会在系统中增加大量的小方法并散落在系统的各个角落，它可以使一个系统的局部设计简化，因为每一个局部都不会和远距离的对象有直接的关联，但是也会造成系统的不同模块之间的通信效率降低，使得系统的不同模块之间不容易协调。

III. 广义的LoD：指对对象之间的信息流量、流向以及信息的影响的控制，主要是对信息隐藏的控制。信息的隐藏可以使各个子系统之间脱耦，从而允许它们独立地被开发、优化、使用和修改，同时可以促进软件的复用，由于每一个模块都不依赖于其他模块而存在，因此每一个模块都可以独立地在其他的地方使用。一个系统的规模越大，信息的隐藏就越重要，而信息隐藏的重要性也就越明显。

（续）

IV. 迪米特法则的主要用途在于控制信息的过载。

① 在类的划分上，应当尽量创建松耦合的类，类之间的耦合度越低，就越有利于复用，一个处在松耦合中的类一旦被修改，不会对关联的类造成太大波及

② 在类的结构设计上，每一个类都应当尽量降低其成员变量和成员函数的访问权限

③ 在类的设计上，只要有可能，一个类型应当设计成不变类

④ 在对其他类的引用上，一个对象对其他对象的引用应当降到最低。

- 总结：**一个软件实体应当尽可能少的与其他实体发生相互作用。**

迪米特法则实例

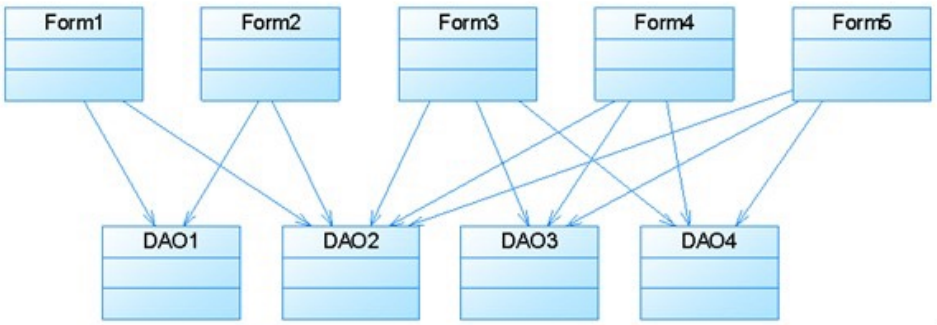


图1

- 某系统界面类（如Form1、Form2等类）与数据访问类（如DAO1、DAO2等类）之间的调用关系较为复杂。

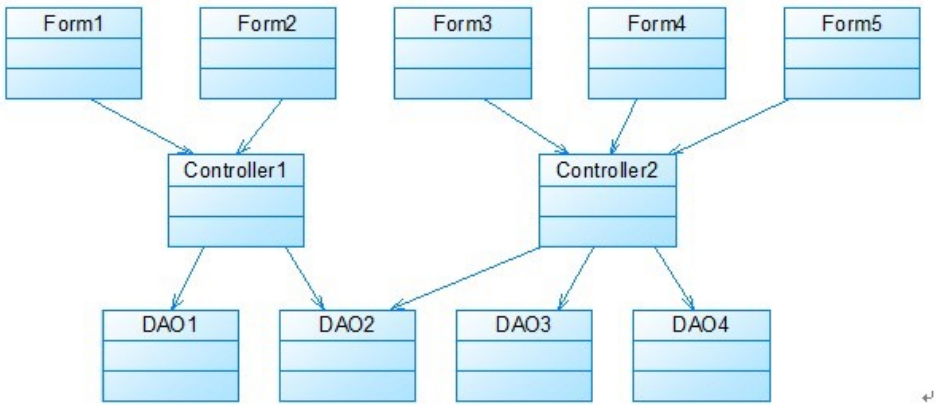


图2

- 图1为什么到图2哪？因为这样就可以降低类的耦合性，是类中功能更加单一，相当于外观模式。

单一职责原则

- 定义
  I. 一个对象应该只包含单一的职责，并且该职责被完整地封装在一个类中。
  II. 就一个类而言，应该仅有一个引起它变化的原因。

- 总结：**就一个类而言，应该仅有一个引起它变化的原因。**

单一职责原则分析

- 一个类（或者大到模块，小到方法）承担的职责越多，它被复用的可能性越小，而且如果一个类承担的职责过多，就相当于将这些职责耦合在一起，当其中一个职责变化时，可能会影响其他职责的运作。
- 类的职责主要包括两个方面：数据职责和行为职责，数据职责通过其属性来体现，而行为职责通过其方法来体现。
- 单一职责原则是实现高内聚、低耦合的指导方针，在很多代码重构手法中都能找到它的存在，它是最简单但又最难运用的原则，需要设计人员发现类的不同职责并将其分离，而发现类的多重职责需要设计人员具有较强的分析设计能力和相关重构经验。

单一职责原则实例

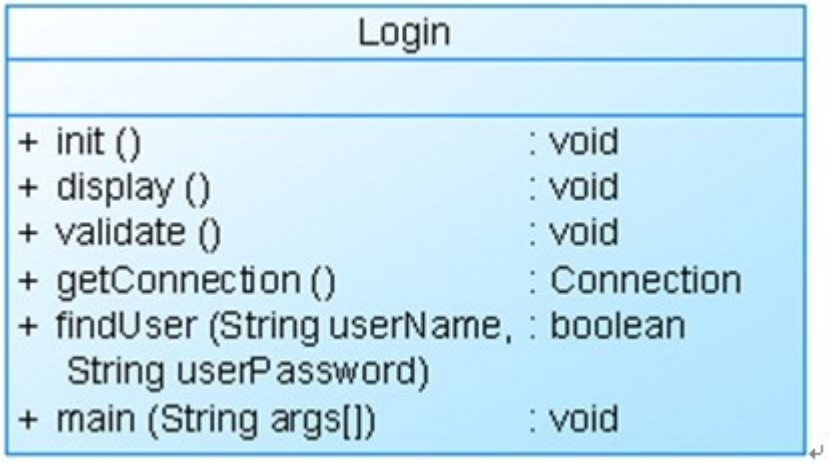


图1

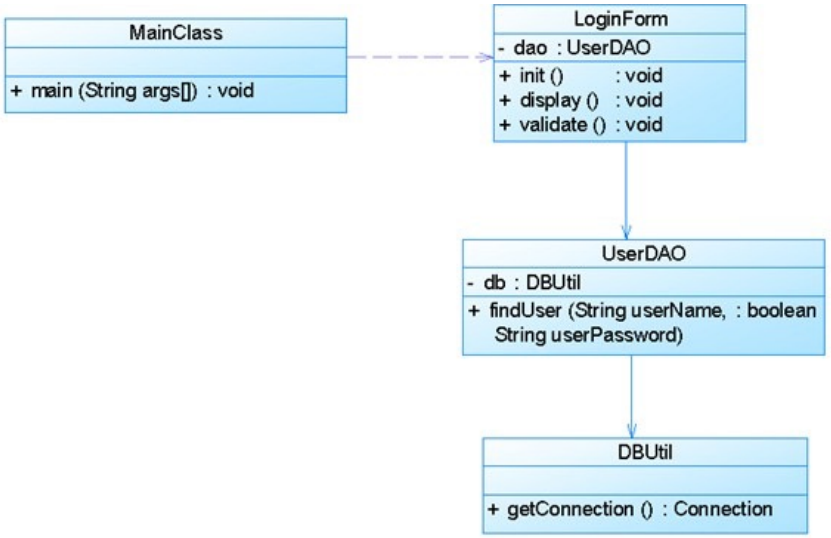


图2

- 某基于Java的C/S系统的“登录功能”通过登录类（Login）实现，见图1。
- 图1功能太过于集成，严重违反类的单一原则。图2为按单一原则重新设计的类。

依赖倒置原则

- 定义：
  I. 高层模块不应该依赖低层模块，它们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。
  II. 要针对接口编程，不要针对实现编程。

- 总结：**高层模块不应该依赖底层模块，两个都应该依赖于抽象；抽象不应该依赖于细节，细节应该依赖于抽象。**

依赖倒置原则分析

- DIP简单说：代码要依赖于抽象的类，而不要依赖于具体的类；要针对接口或抽象类编程，而不是针对具体类编程。
- 实现开闭原则的关键是抽象化，并且从抽象化导出具体化实现，如果说OCP是面向对象设计的目标的话，那么DIP就是面向对象设计的主要手段。
- DIP的常用实现方式之一是在代码中使用抽象类，而将具体类放在配置文件中。
- DIP要求客户端依赖于抽象耦合，以抽象方式耦合是依赖倒转原则的关键。

类之间的耦合关系

- 三种耦合关系：
  ① 零耦合关系
  ② 具体耦合关系
  ③ 抽象耦合关系。
- 依赖注入：
  - 构造注入：通过构造函数注入实例变量
  - 设值注入：通过Setter方法注入实例变量
  - 接口注入：通过接口方法注入实例变量

依赖倒转原则实例

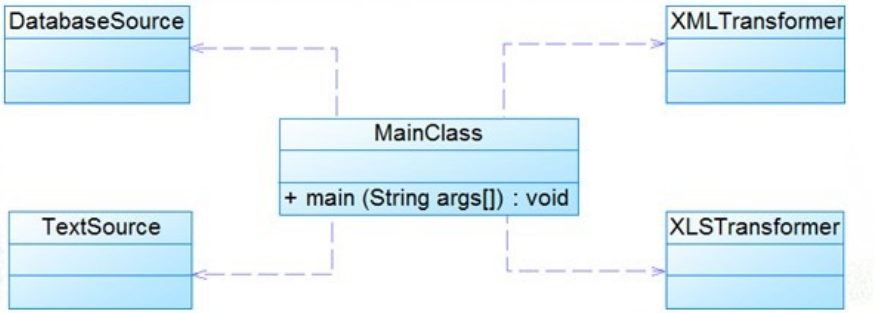


图1

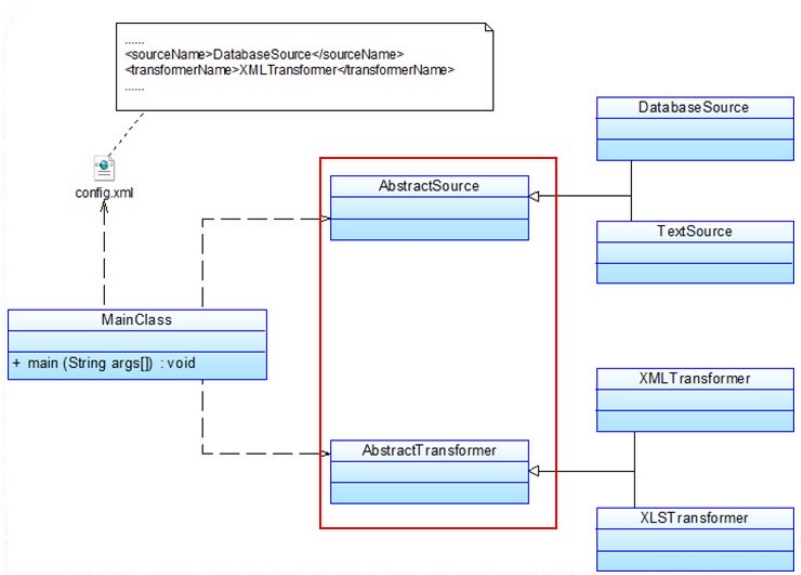
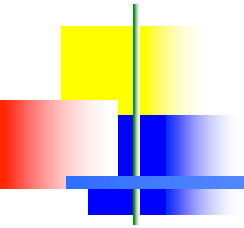


图2

- 某系统提供一个数据转换模块，可以将来自不同数据源的数据转换成多种格式，如可以转换来自数据库的数据(DatabaseSource)、也可以转换来自文本文件的数据(TextSource)，转换后的格式可以是XML文件(XMLTransformer)、也可以是XLS文件
- 图1需要增加新的数据源或者新的文件格式时，客户类MainClass都需要修改源代码，以便使用新的类，但违背了开闭原则。现使用依赖倒转原则对其进行重构。



面向对象方法

3 面向对象分析

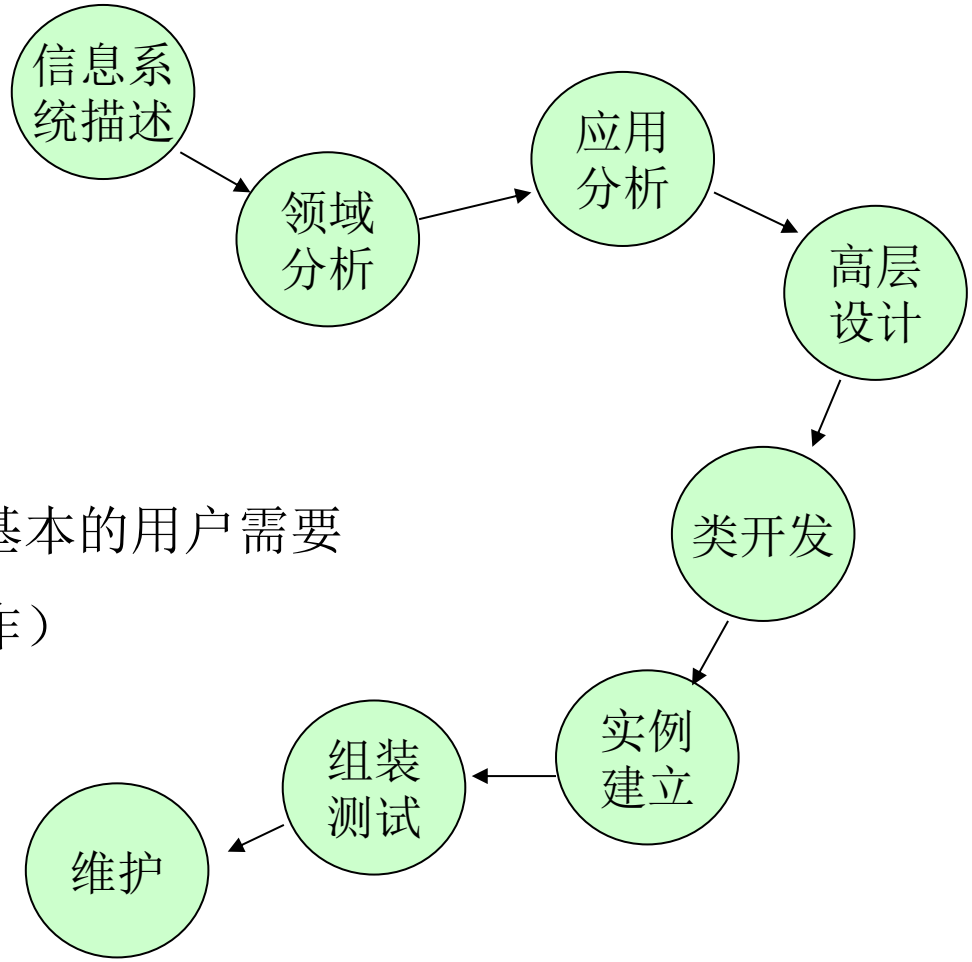
面向对象分析—主要内容

1. 面向对象分析OOA（Object-Oriented Analysis）
2. 领域分析
3. OOA的一般步骤



OOA的目标和任务

- OOA的目标
  - 完成对所求解问题的分析，确定待建系统要做什么
  - 建立系统的模型
- OOA的任务
  ① 在客户和软件工程师之间沟通基本的用户需要
  ② 标识类（包括定义其属性和操作）
  ③ 刻画类的层次结构
  ④ 表示类（对象）之间的关系
  ⑤ 为对象行为建模
  ⑥ 递进地重复任务①至任务⑤，直至完成建模



3.1 面向对象分析

- 传统方法和面向对象方法
- 面向对象分析概述

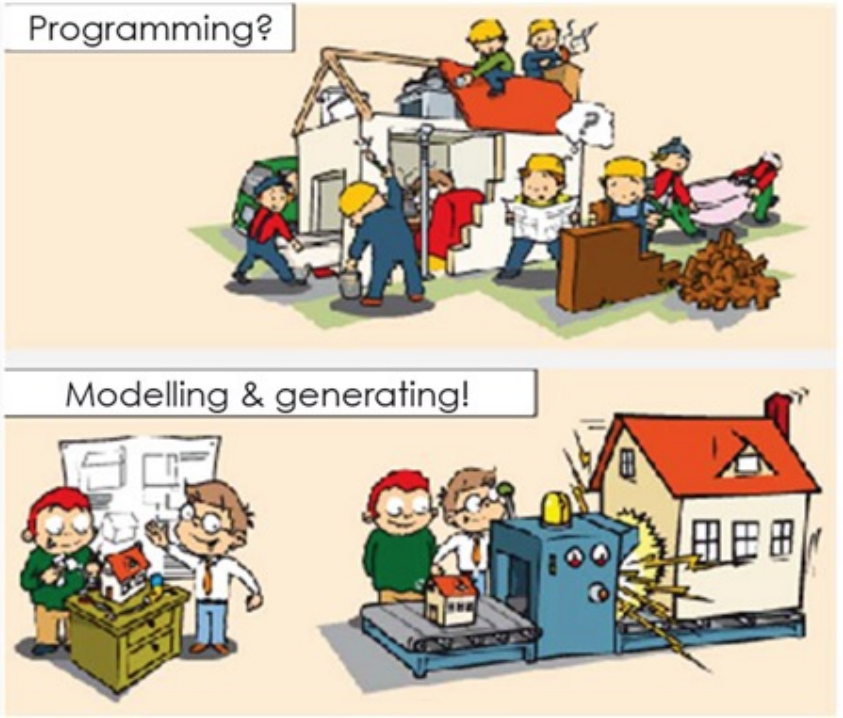


模型（model）

- 模型是现实的抽象。
- 对同一个事物，可以从不同的视角描述这个事物，从而得到不同的模型。
- 模型可以有不同的描述方法或表现形式。

Restraining factors to construct a model

- *Assumptions（假设）*
- *Simplifications（简化）*
- *Limitations（限制）*
- *Constrains（约束）*
- *Preferences（偏爱）*

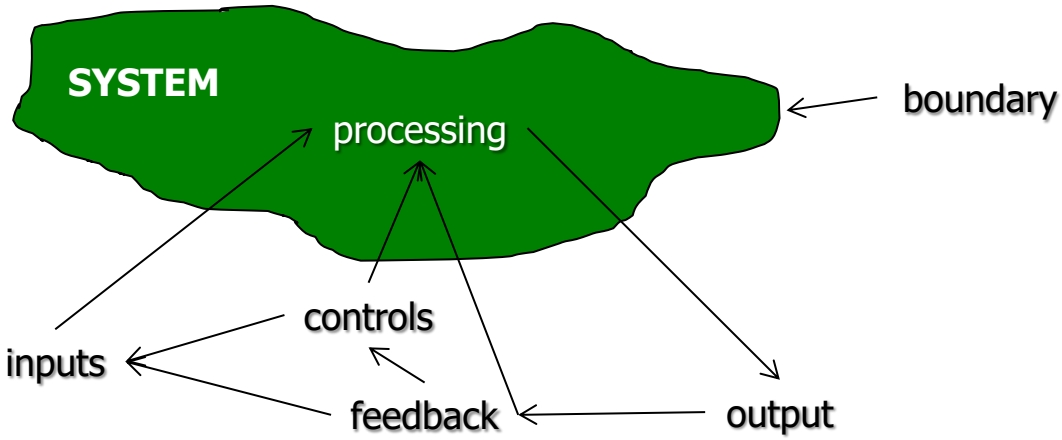


Systems Model with 6 Components

Model

- *Define the processes that serve the needs of the view under consideration.*
- *Represent the behavior of the processes and the assumptions on which the behavior is based.*
- *Explicitly define both exogenous and endogenous input to the model.*
- *Represent all linkage (including output) that will enable the engineer to better understand the view.*

-- Mortamarrin, 1992



3.2 领域分析

- 领域分析与复用
- 领域分析过程



什么是领域？

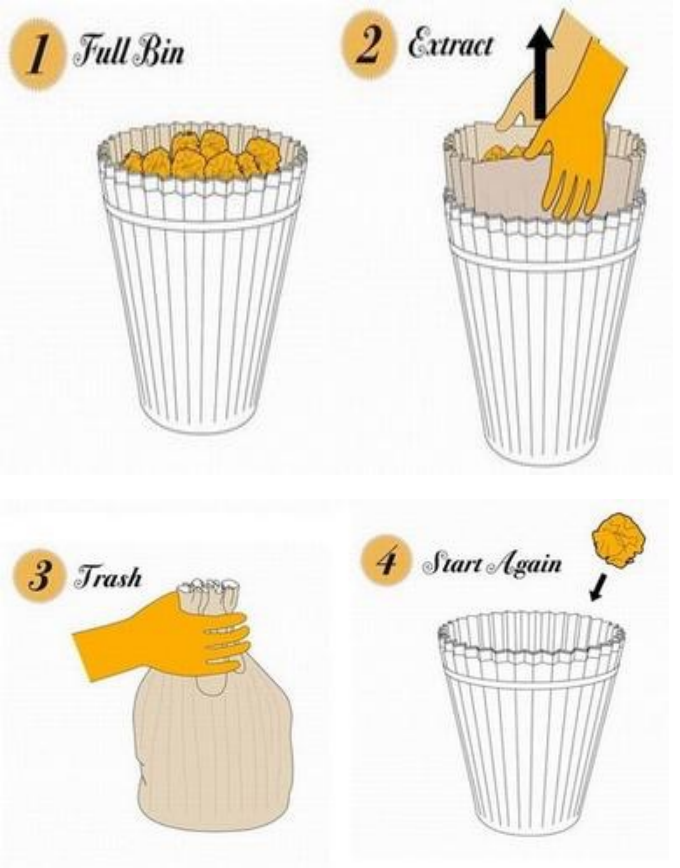
- **领域**（Domain）是指一组具有相似或相似软件需求的应用系统所覆盖的功能区域。一个领域是一组相关的系统，相关的方面是公共设计、公共服务、公共技术和公共信息体系结构。领域内的不同系统，相同系统的不同版本之间由于共享领域知识，因此具有相似的设计、体系结构、服务和技术。
- 领域工程是为领域产品族中相似应用系统建立核心资产的过程。这些核心资产包括领域需求、领域框架、以及领域可复用构件，领域工程覆盖了开发可利用软件构件的所有活动。
- 应用工程则是根据领域工程中所生产的核心可复用软件构件进行基于可复用软件构件的应用系统开发的过程和活动。

软件复用的效益
- 质量：复用构件错误较少
- 生产率：30-50%的复用导致生产提高25-40%
- 成本节省



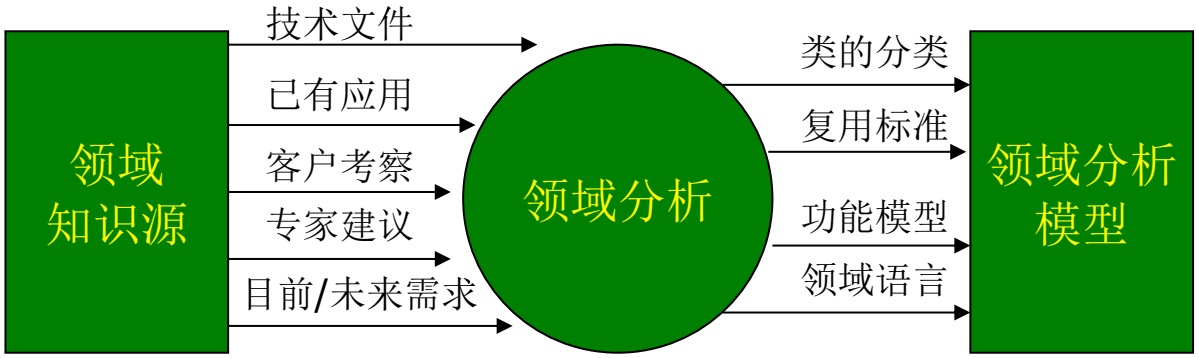
什么是领域分析？

- 领域分析指对特定问题空间内的知识进行提取、整理和建模，它为以后涉及该问题空间的系统开发提供**复用基础**。
- 领域分析的目标是为领域产品族中相似应用系统建立核心资产的过程，用于应用工程的复用。
- 领域工程可以分为以下三个主要阶段：**领域分析、领域设计和领域实现**。



领域分析与复用

- 领域分析是为了建立复用的类库。
- 以公共对象、类、子集合和框架等形式，在特定的应用领域中标识、分析和规约公共的可复用的能力。
- 领域分析是软件过程的一个全程活动，不和任何软件项目联系，它类似于制造环境中的工具制造者。



领域分析过程

- 定义领域范围
- 领域中项目的分类
- 收集有代表性的应用

面向对象复用

- 软件成分的复用级别
  - 代码复用
  - 设计复用
  - 分析复用
- 可复用构件的特点：
  - 模块独立性强
  - 具有高度可塑性：提供扩充和修改机制
  - 接口清晰、简明、可靠，详尽的文档说明
- 类构件的重用方式
  - 实例复用
  - 继承复用
  - 多态复用

什么是代码重用？

- **代码重用**（Code Reuse），又称为代码复用，也称为软件重用，是对现有（已经写好的）代码进行重复使用。重用的代码可以来自外部资源，也可以来自以往的项目，甚至是当前项目，并用其开发新的软件或代码逻辑。
- 代码重用的好处：
  - 提高开发效率
  - 降低开发成本
  - 使软件更简洁
- 代码重用弊端：
  - 不可控
  - 增加外部依赖

代码重用方式

① **代码片段重用**
- 将重用的代码段以函数、静态库、动态库、类等方式在代码中调用都可算是代码片段重用，这是最广泛也是最直观的重用方式。

② **组件重用**
- 相比代码片段重用，组件重用粒度更大。

③ **框架重用**
- 框架重用也被称为模式重用，框架本身并不是完整的代码，而是为使用者提供了编码的规范。
- 例如：被广泛应用的Spring MVC框架等，程序员在框架内填充自己的业务逻辑实现特定功能，而框架帮助程序员组织整个软件架构，提升了整个软件的效率和可维护性等。

3.3 OOA的一般步骤

① 获取客户对系统的需求：包括标识场景（scenario）和用例（use case），以及建造需求模型

② 用基本的需求为指南来选择类和对象

③ 定义类的结构和层次

④ 建造对象-关系模型

⑤ 建造对象-行为模型

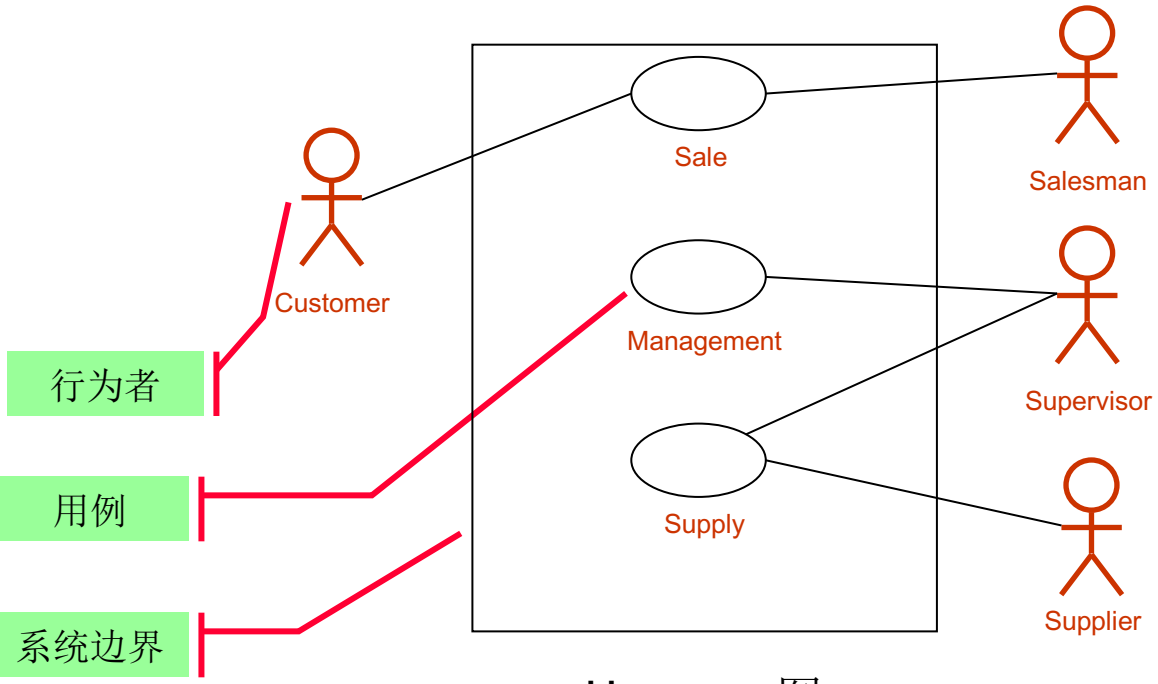
⑥ 利用用例/场景来复审分析模型



1）获取客户对系统的需求

必须让客户与开发者充分地交流。

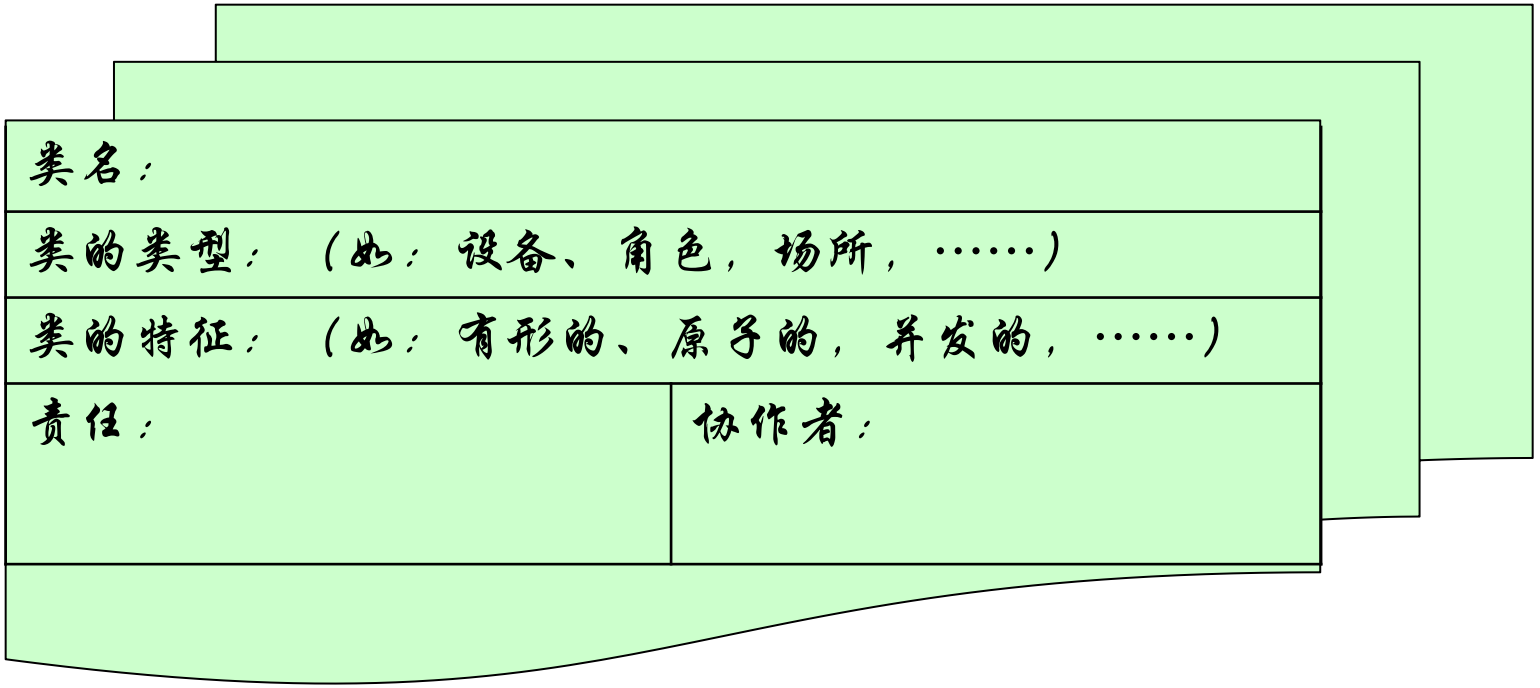
- 用例（user case）
- 行为者（actor）
- 场景（scenario）
- 功能（function）



Use-case图

2）标识类和对象

- CRC索引卡，Class-Responsibility-Collaborator，类-责任-协作者。



识别类对象

- 外部实体
  - 他们生产或消费计算机系统所使用的信息，
  - 如：其它系统、设备、人员
- 角色
  - 他们由与系统交互的人扮演，
  - 如：管理者、工程师、销售员
  - 也可以是其它的系统。
- 构造物
  - 它们定义一类对象，或者定义对象的相关类
  - 如：交通工具、计算机
- 发生的事件
  - 它们出现在系统运行的环境中
  - 如，完成一系列遥控动作
- 组织单位
  - 他们与一个应用有关
  - 如：部门、小组、小队

- 场所
  - 它们建立问题和系统所有功能的环境
  - 如：制造场所、装载码头
- 事物
  - 它们是问题信息域的一部分
  - 如：报告、显示、信函、信号

潜在对象成为最终的对象：

- 保留的信息
- 需要的服务
- 多个属性
- 公共属性
- 公共操作
- 必需的需求

责任与协作者

责任（Responsibility）

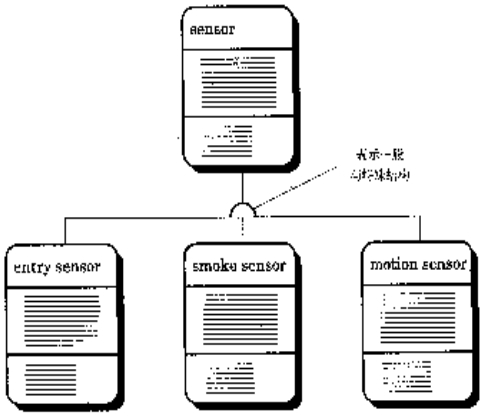
- 是类所知道的或要做的任何事情。
- 是与类相关的属性和操作。
  - 属性定义了对象，阐明了问题空间中对象的作用和地位。
  - 操作定义了对象的行为并以某种方式修改对象的属性值。操作分为三类：
    - 操纵数据的操作
    - 计算的操作
    - 为控制事件的发生而监控对象的操作

协作者（Collaborator）

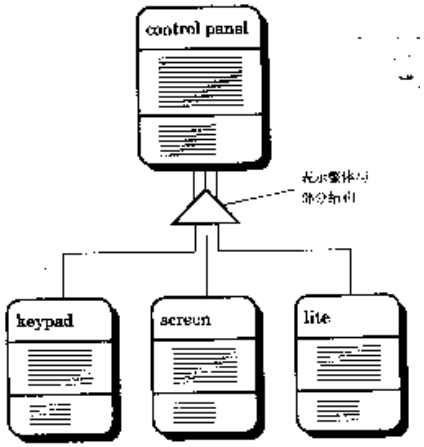
- 如果一个对象为完成某个责任需要向其它对象发送消息，则该对象和另一个对象协作。
- 协作反映了类之间的关系。
- 一个类的协作可以通过确定该类是否能独立完成每个责任来标识，如果不能，则它需要和另一个类交互，从而可识别一个协作。

3）定义类的结构和层次

- 类的结构主要有两种：
  - 一般-特殊（generalization-specialization）： “is a ”的关系
  - 整体-部分（whole-part）： “has a ”的关系
- 可以将互相协作的类的集合定义为主题（subject）或子系统（subsystem）。
- 类结构和层次图(Coad &Yourdon) ：



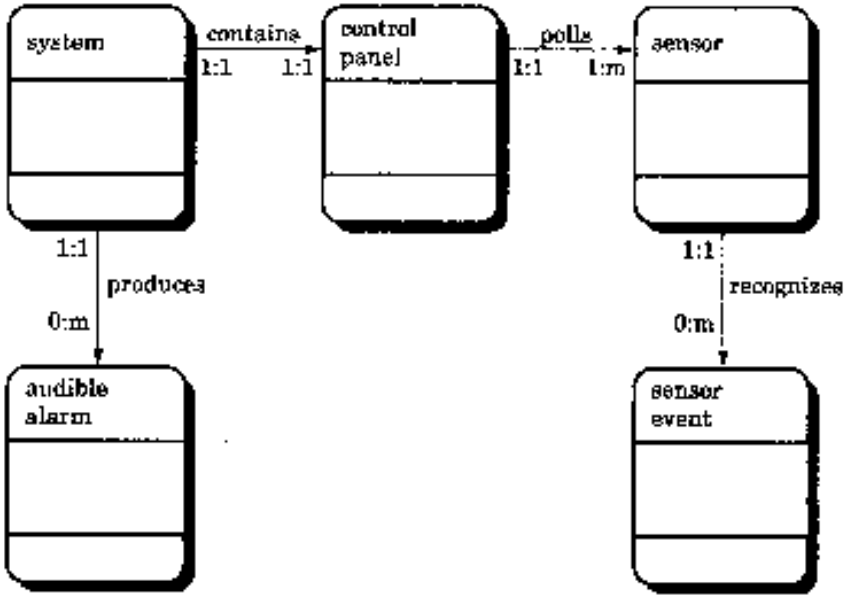
一般与特殊结构



整体与部分关系

4）建造对象-关系模型

- 对象-关系模型描述了系统的静态结构，它指出了类间的关系（relationship）及对象之间的消息路径。
- 三个步骤：
  - ① 用CRC画协作者对象网络
  - ② 复审CRC，标出箭头
  - ③ 确定关系基数（ 1对1，1对多，多对1，多对多）

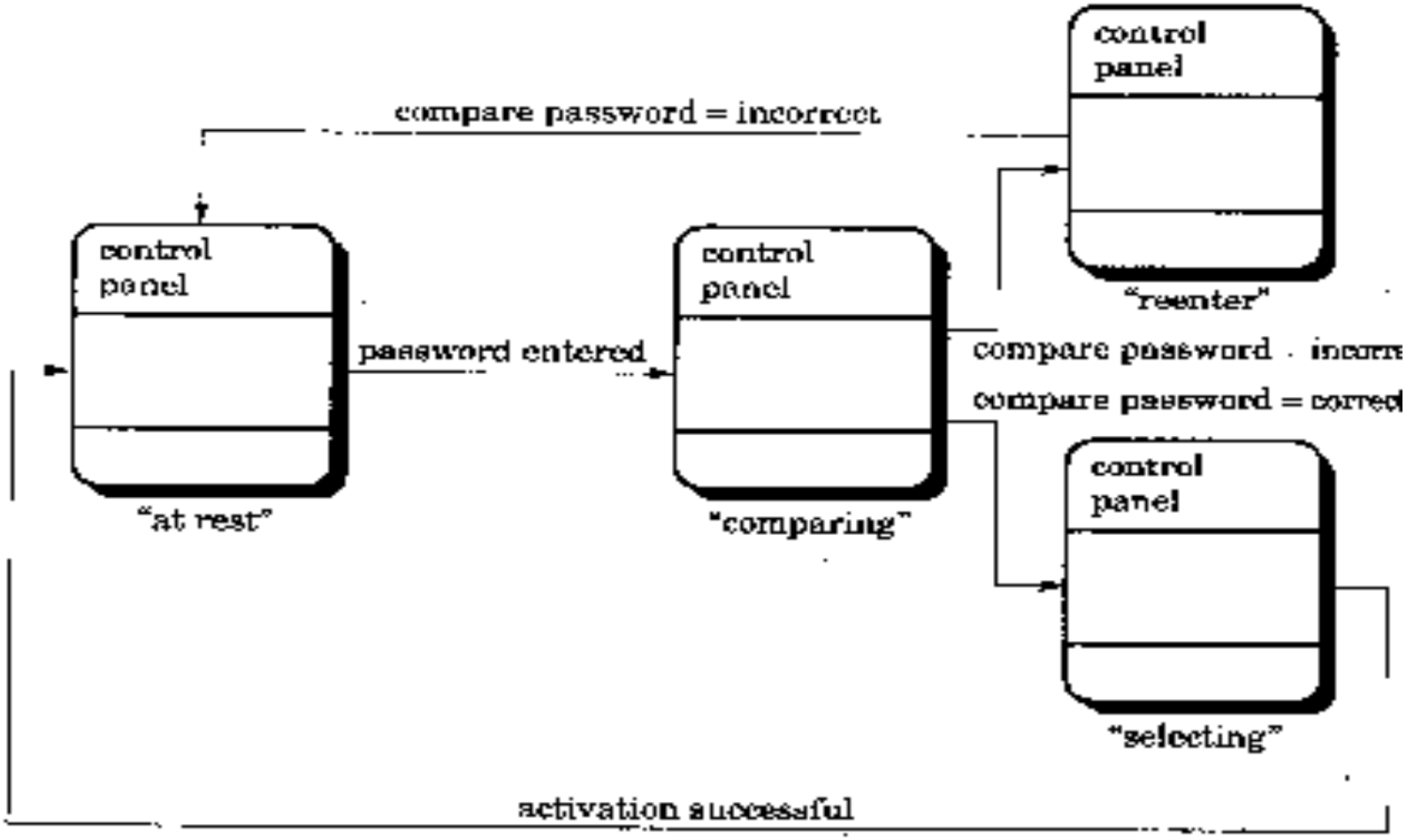


类间的连接（完成责任的协作者类），最常见是二元关系如：**1对1，1对多，多对1，多对多。**

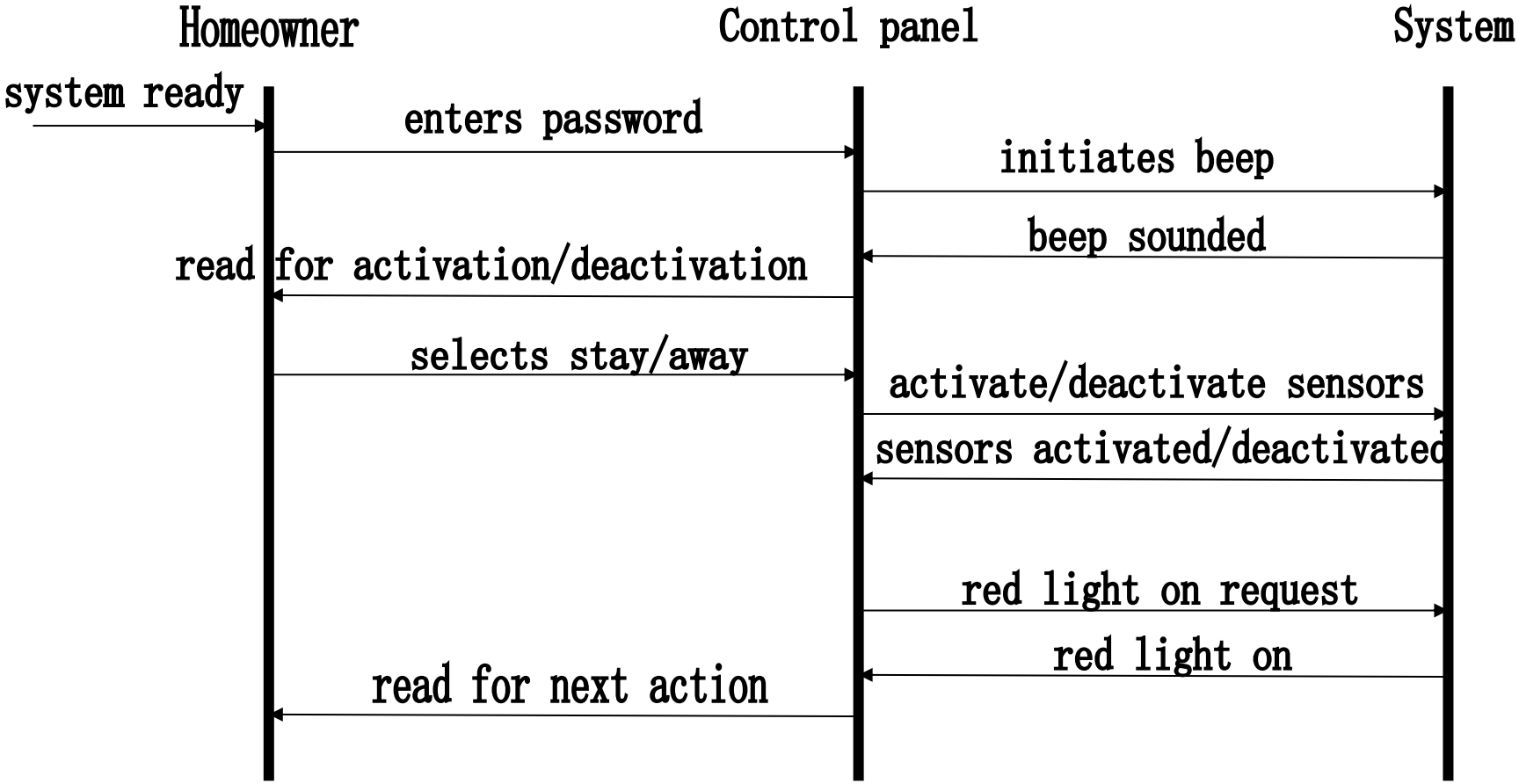
5）建立对象-行为模型

- 对象-行为模型描述了系统的动态行为，它们指明系统如何响应外部的事件或激励（stimulus）。
- 目的是描述系统的动态行为。
- 对所有实例要完全理解系统中交互的序列
- 找出驱动交互序列的事件
- 创建事件轨迹
- 创建状态-变迁（转换）图
- 检查模型的正确
- 状态的表示要考虑两个方面：
  - 每个对象的状态变化（状态图）；
  - 系统中各对象的状态关系（轨迹、时序图）

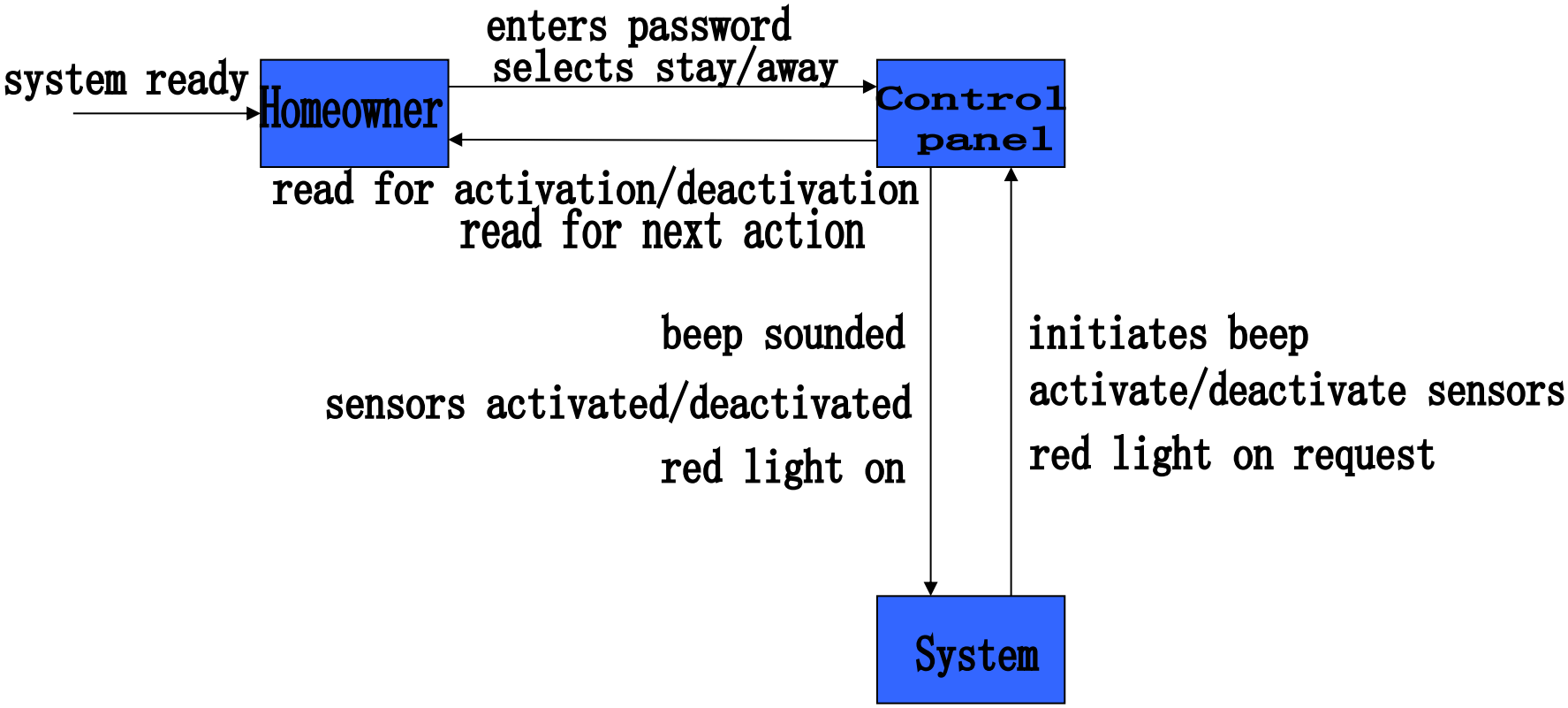
对象行为模型（状态迁移）

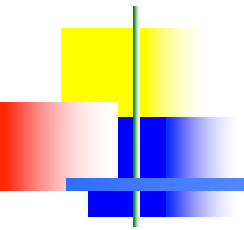

control
panel
compare password = incorrect
control
panel
"reenter"
control
panel
password entered
compare password · incorr
compare password = correct
"at rest"
"comparing"
control
panel
"selecting"
activation successful

对象行为模型（轨迹图）


Homeowner
Control panel
System
system ready
enters password
initiates beep
beep sounded
read for activation/deactivation
selects stay/away
activate/deactivate sensors
sensors activated/deactivated
red light on request
red light on
read for next action

对象行为模型（事件流图）


system ready
Homeowner
enters password
selects stay/away
Control panel
read for activation/deactivation
read for next action
beep sounded
sensors activated/deactivated
red light on
initiates beep
activate/deactivate sensors
red light on request
System



面向对象方法

4 面向对象设计

面向对象的设计——主要内容

1. 传统与OOD方法比较
2. OOD模型的体系成分
3. OOD设计方法
4. OOD步骤



4.1 结构化方法与OOD方法比较

	结构化方法	OOD方法
相同点	数据设计	属性设计
	接口设计	消息模型
	过程设计	操作设计
区别 （总体结构设计）	层次化	对象间的协作

面向过程分析方法的特点

（1）面向用户，更注重思维逻辑
（2）强调调查工作的重要性
（3）对管理业务中的各种数据进行分解
（4）采用了层次分解的系统思想
（5）用图形工具来分析和构建新方案

面向对象系统分析方法的特点

（1）强调系统开发的整体性与全局性
（2）侧重于数据转换的过程而不是数据本身
（3）系统的开发周期长
（4）封装性：突破了传统数据与操作分离的模式。
（5）继承性：继承是面向对象特有的
（6）多态性

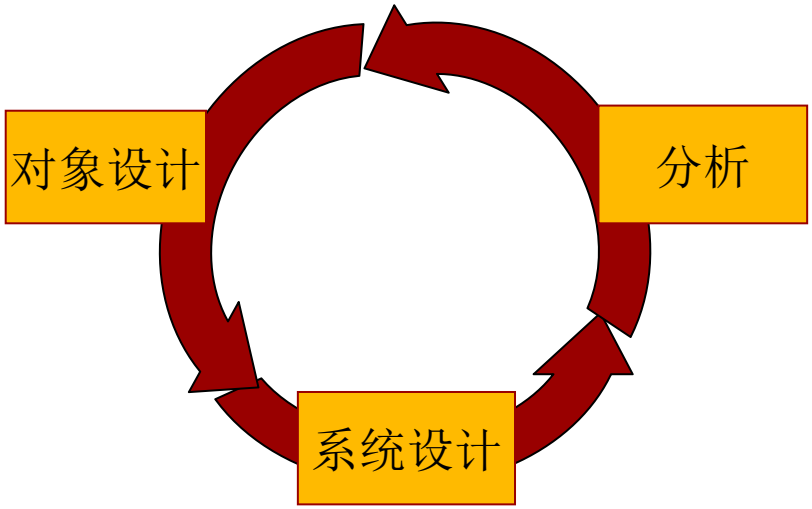
系统设计的十个方面

1. 模块层次的表示
2. 数据定义的规约
3. 过程逻辑的规约
4. 端点间处理序列的指明
5. 对象状态和转变的表示
6. 类及层次的定义
7. 类的操作
8. 详细的操作定义
9. 消息连接的规约
10. 独有的服务标识



结构化方法不支持5~10条

5.2 OOD模型的体系成分

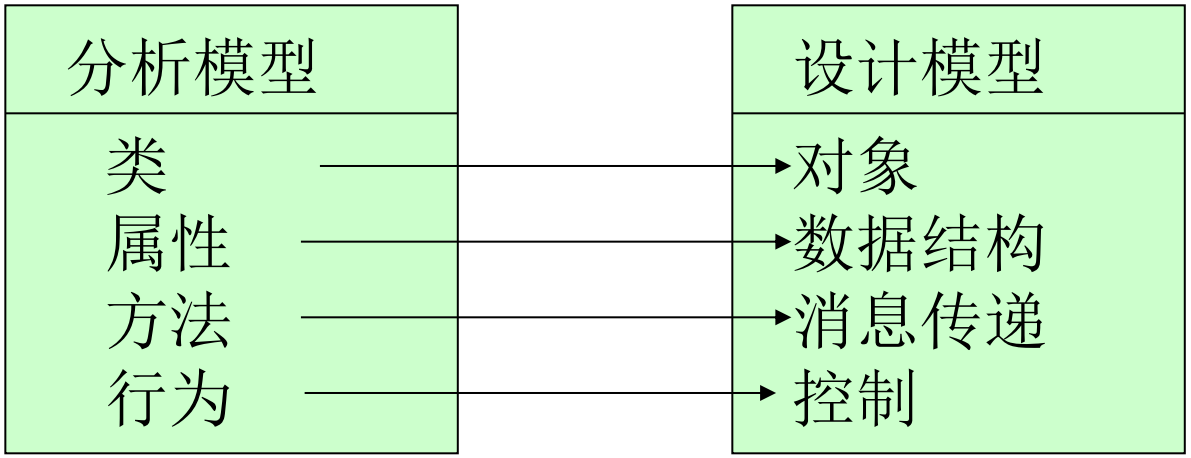


- 面向对象设计（Object-oriented Design，OOD）是将OOA所创建的分析模型转化为设计模型。
- OOD与OOA之间没有明显的分界线，两阶段采用相同的符号表示。
- **在OOA时，主要考虑系统做什么，而不关心系统如何实现；在OOD时，主要解决系统如何做。**
- OOD要在OOA的模型中为系统补充一些新的类，或在原有的类中补充一些属性和操作。
- OOD时应能从类中导出对象，以及对象的关联，描述对象的关系、行为以及对象通讯等。

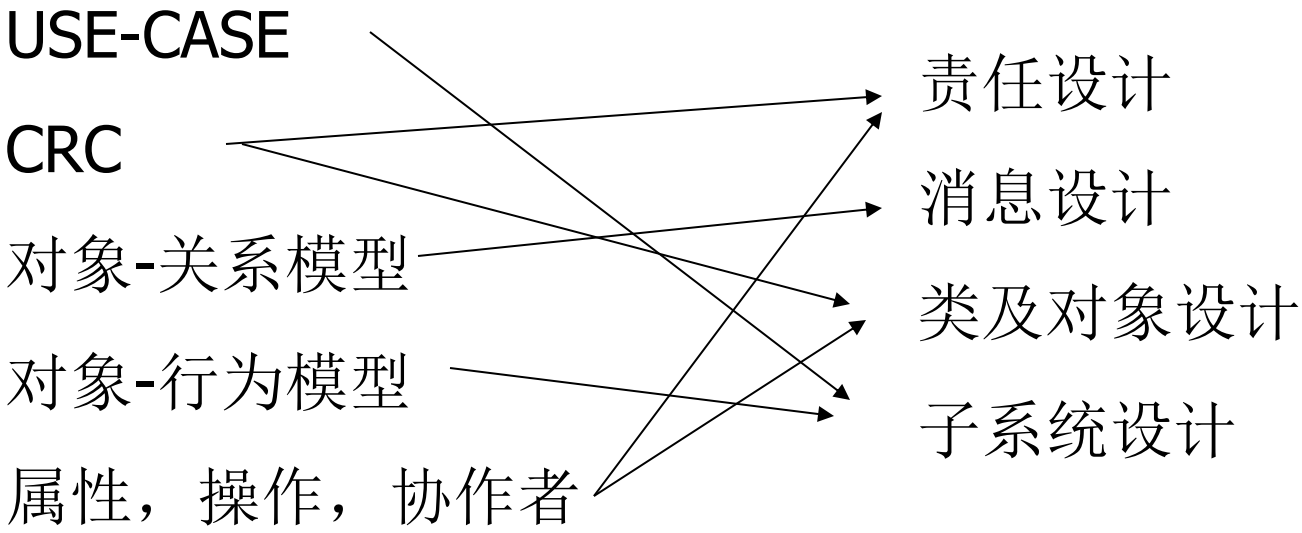
OO设计模型的体系成分

系统设计要定义四种重要构件：

- 问题域：负责实现需求的子系统
- 人机交互：实现用户界面子系统
- 任务管理：负责控制和协调并发任务的子系统
- 数据管理：负责对象的存储和检索子系统



4.3 OO设计方法



设计中的问题

Bertrand Meyer的设计方法模块化能力评判：

- 分解性（方法能提供帮助设计者分解大型问题）
- 组装性（方法能支持设计的构件可复用）
- 易理解性（使模块可独立被理解）
- 连贯性（使修改后对系统影响小）
- 保护性（出现问题时对系统的影响小）

从而对设计提出基本设计原则：语义模块单元；很少接口；接口简单；显式接口；信息隐蔽等

4.4 OOD的步骤

① 系统设计
- 将子系统分配到处理器；
- 选择实现数据管理、界面支持和任务管理的设计策略；
- 为系统设计合适的控制机制
- 复审并考虑权衡

② 对象设计
- 在过程级别设计每个操作
- 定义内部类
- 为类属性设计内部数据结构

③ 消息设计
- 使用对象间的协作和对象-关系模型，设计消息模型。

④ 复审
- 复审设计模型，并在需要时迭代。



系统设计

1）将分析模型划分成子系统

- 子系统的设计准则是：
  - 子系统应该具有定义良好的接口，通过接口和系统的其它部分通信；
  - 除了少数的通信类外，子系统中的类应只和该子系统中的其它类协作；
  - 子系统的数量不宜太多；
  - 可以在子系统内部再次划分，以降低复杂度。

2）标识问题本身的并发性，并为子系统分配处理器

- 通过对对象-行为模型的分析，找出系统的并发性。
- 并发的子系统可以分配在不同的处理器上，也可以分配在同一个处理器上，由操作系统提供并发处理。

（续）

3）任务管理设计

- 设计策略：
  - 确定任务的特征
  - 定义协调者任务和关联的对象
  - 集成协调者任务和其它任务
- 可以通过任务如何被启动的，来确定任务的特征。

4）数据管理设计

- 数据管理设计包括：
  - 系统中各种存储对象的存储方式，如：内存数据结构、文件、数据库
  - 设计相应的服务，即为要存储的对象增加所需的属性和操作。

基本任务模板：

- 任务名
- 描述
- 优先级
- 服务
- 由...协调
- 通过...通信。

（续）

5）资源管理设计

- 设计一套控制机制和安全机制，以控制对资源的访问，避免对资源使用的冲突。

6）人机界面的设计

7）子系统间的通信

- 子系统之间可以通过建议C/S连接进行通信，也可以通过端到端（peer to peer）连接进行通信。

对象设计

- 对象设计是为每个类的属性和操作作出的详细设计，并设计连接类与它的协作者之间的消息规约。

1）对象描述

- 对象描述的形式有：
  - 协议描述：描述对象的接口
  - 实现描述：每个操作的实现细节

2）设计算法和数据结构

- 为对象中的属性和操作设计数据结构。

对象描述—协议描述

- 协议描述
  - 通过接收的消息和完成的操作来建立对象的接口
- 例：
  - 读传感器时所需消息：

Message(motion sensor) read: RETURNS sensor ID, sensor status;

  - 设置和复位传感器：

Message(motion sensor) set: SENDS sensor ID, sensor status;

对象描述—实现描述

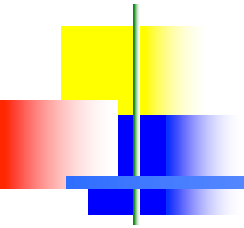
- 实现描述
  - 从消息蕴涵的操作来表明对象数据结构的内部细节和操作过程。
- 实现描述如下信息：
  - 对象名字的定义和类的引用；
  - 表明数据项和类型的私有数据结构定义；
  - 每个操作的过程描述或指针。
- 例：
  - 对象名：smoke sensor
  - 类名：sensor
  - 私有数据结构：ID: string;
  - 操作过程：set/reset/read/get…

算法设计

- 系统中类的操作通常有三类：
  - 操纵数据的操作；
  - 执行计算的操作；
  - 监控对象的操作。

设计模式（pattern）

- 在许多面向对象的系统中，存在一些类和通信对象的重复出现模式。
- 一个设计模式通常用四类信息描述：
  - 模式名
  - 应用模式时必须存在的环境条件
  - 设计模式的特征
  - 应用设计模式的结果



面向对象方法

5 面向对象方法介绍

OOA与OOD方法—主要内容

- Coad & Yourdon方法
- Rumbaugh的OMT方法
- Booch方法
- OOSE方法

5.1 Coad & Yourdon方法

- Coad & Yourdon方法比较简单，也是最容易学习的一种OO方法。
- Coad & Yourdon的OOA模型由五个层次和五个活动组成。
- Coad & Yourdon的OOD模型由四个部件和四个活动组成。

OOA的五个层次和五个活动

OOA的五个层次：

① 主题层： 控制一次分析所考虑的范围， 即对相关的类进行归并。

② 类及对象层：在分析范围内找出全部的对象。

③ 结构层：分析对象的分类结构和组装结构。

④ 属性层：描述每个对象的状态特征。

⑤ 服务层：描述每个对象所具有的操作。

OOA五个活动：

① 确定主题

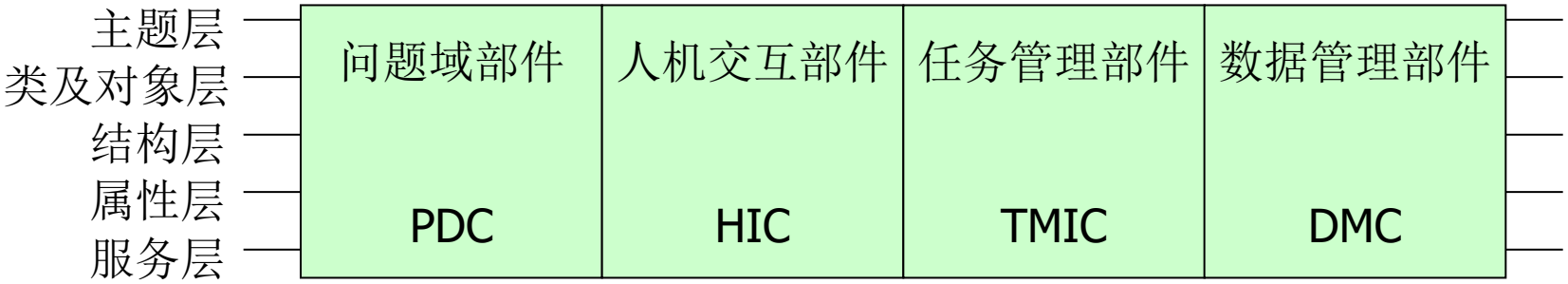
② 识别类及对象

③ 识别类的结构

④ 定义属性

⑤ 定义服务

OOD模型及组成



Coad & Yourdon OOD模型

- 四个部件
  - ① 问题域部件
  - ② 人机交互部件
  - ③ 任务管理部件
  - ④ 数据管理部件

- 四个活动
  - ① 设计问题域部件
  - ② 设计人机界面部件
  - ③ 设计任务管理部件
  - ④ 设计数据管理部件

问题域部件（problem domain component）

- 组合所有的领域特定类
- 为应用类设计适当的类层次
- 必要时简化继承
- 细化设计以改善性能
- 开发与数据管理部件的接口
- 按要求细化并加入低层次的对象
- 复审设计并审查对分析模型的增补

人机交互部件（human interface component）

- 定义参与人员
- 开发任务场景
- 设计用户命令的层次
- 细化用户交互序列
- 设计相关的类和类层次
- 必要时集成图形用户界面

任务管理部件（task management component）

- 标识任务的类型
- 建立优先级
- 标识作为其它任务协作者的任务
- 为每任务设计合适的对象

数据管理部件（data management component）

- 设计数据结构
- 设计管理数据结构所需要的服务
- 标识可以协助实现数据管理工具
- 设计适当的类和类层次

5.2 OMT方法

- OMT（Object Modeling Technique）方法是由Rumbaugh等人提出的。
- OMT的三种模型
  - 对象模型
    - 描述一个系统中对象的结构，它表示静态的、结构上的、系统的数据特征。
  - 动态模型
    - 描述与时间和操作顺序有关的系统特征，它表示瞬时的、行为的、系统的控制特征。
  - 功能模型
    - 描述与值的变换有关的系统特征，它表示变换、系统的功能特征。

OMT的OOA过程

① **问题陈述**

- 问题陈述为记下或获取对问题的初步描述。

② **构造对象模型**

- 确定对象类。
- 编制描述类、 属性及关联的数据词典。
- 在类之间加入关联。
- 给对象和链加入属性。
- 使用继承来构造和简化对象类。
- 将类组合成模块， 这种组合在紧耦合和相关功能上进行。
- 最后得到： 对象模型=对象模型图+数据词典。

（续）

③ **构造动态模型**

- 准备典型交互序列的脚本。
- 确定对象间的事件并为各脚本安排事件跟踪。
- 准备系统的事件流图。
- 开发具有重要的动态行为的各个类的状态图。
- 检查状态图中共享事件的一致性和完整性。
- 最后得到： 动态模型=状态图+全局事件流图。

④ **构造功能模型**

- 确定输入、 输出值。
- 需要时使用数据流图来表示功能依赖关系。
- 描述各功能“干什么”。
- 确定约束。
- 详细说明优化标准。
- 最后得到： 功能模型=数据流图+约束。

OMT的OOD过程

① **系统设计**

- 将系统分解为各子系统。
- 确定问题中固有的并发性。
- 将各子系统分配给处理器及任务。
- 根据数据结构、 文件及数据库来选择实现存储的基本策略。
- 确定全局资源和制定控制资源访问的机制。
- 选择实现软件控制的方法。
- 考虑边界条件。
- 最后得到： 系统设计文档=系统的基本结构+高层次决策策略。

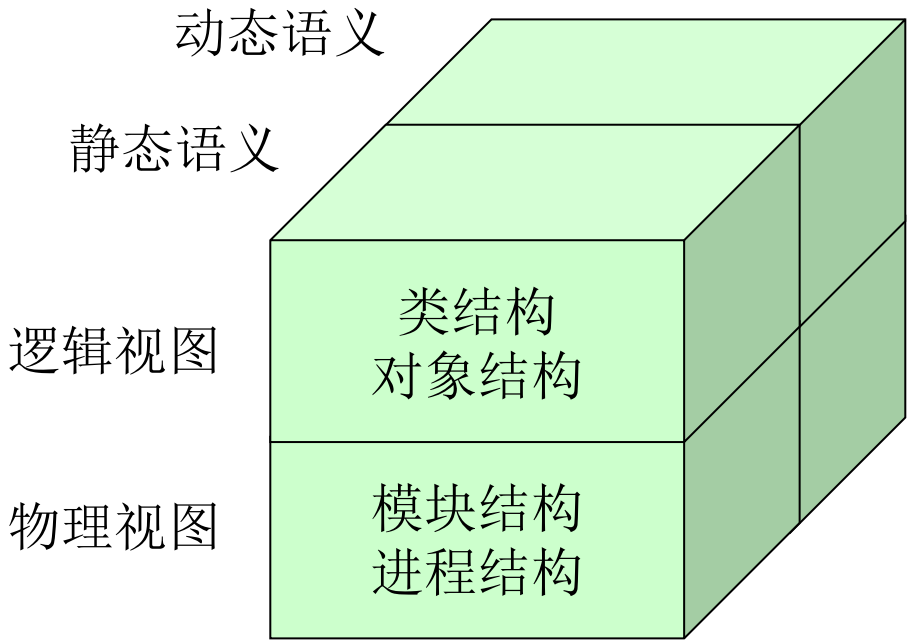
（续）

② 对象设计

- 从其他模型获取对象模型上的操作：在功能模型中寻找各个操作，为动态模型中的各个事件定义一个操作，与控制的实现有关。
- 设计实现操作的算法: 指选择开销最小的算法， 选择适合于算法的数据结构，定义新的内部类和操作。给那些与单个类联系不太清楚的操作分配内容。
- 优化数据的访问路径：指增加冗余联系以减少访问开销，提高方便性，重新排列运算以获得更高效率。为防止重复计算复杂表达式，保留有关派生值。
- 实现系统设计中的软件控制。
- 为提高继承而调整类体系：是为提高继承而调整和重新安排类和操作，从多组类中把共同行为抽取出来。
- 设计关联的实现。分析关联的遍历， 使用对象来实现关联或者对关联中的 1、 2 个类增加值对象的属性。
- 确定对象属性的明确表示：是将类、关联封装成模块。
- 最后得到： 对象设计文档=细化的对象模型+细化的动态模型+细化的功能模型。

5.3 Booch方法

- Booch方法的一般过程：
  - ① 在一定的抽象层次上标识类和对象
  - ② 标识类和对象的语义
  - ③ 标识类和对象间的关系
  - ④ 实现类和对象



Booch方法的开发模型图

- Booch认为软件开发是一个螺旋上升过程，他强调过程的多次重复。

5.4 OOSE方法

- OOSE（Object-Oriented Software Engineering）是Jacobson提出的一种用例（use case）驱动的OO方法。



OOSE的OOA过程

① 标识对象的用户和他们的责任

② 建造需求模型

- 定义行为者（actor）及其责任
- 为每个行为者标识用例
- 制定系统对象及其关系的初始视图
- 利用用例去复审模型以确定其有效性

③ 建造分析模型

- 用行为者交互的信息来标识界面对象
- 创建界面对象的结构视图
- 表示对象的行为
- 分离出子系统和模型
- 利用用例作为场景去复审模型 以确定其合法性

OOSE的OOD过程

① 修改理想化的分析模型以适合现实世界环境

② 创建块作为主要的设计对象

③ 创建一个显示激励如何在块间传送的交互图

④ 把块组织成子系统

⑤ 复审设计工作

结构化方法与面向对象方法的比较

- 内在联系
  - 二者在分解和抽象原则上一致
  - 局部化和重用性设计上的一致
- 区别
  - 处理问题的出发点不同
  - 处理问题的基本单位和层次逻辑关系不同
  - 数据处理方式与控制方式不同

课堂讨论：

(1) 使用自己的话和例子，定义术语“类”、“封装”、“继承”和“多态”。

(2) 设计者如何识别必须并发的任务？

(3) 论述面向对象方法对传统方法的优势和问题。

(4) 如何从需求文档中识别类和对象?

练习题：打车系统的OOA与OOD

1. 用面向对象方法为案例中的“好的打车系统”建模，要求：
   ① 识别出主要的对象
   ② 定义类的结构和层次
   ③ 建造对象-关系模型
   ④ 建造对象-行为模型
2. 用面对象方法设计案例中“好的打车系统”，要求完成主要部分的：
   ① 系统设计
   ② 对象设计
   ③ 消息设计

课堂练习：用OOA为“共享单车App”建模

共享单车App作为日常出行租用自行车的软件，旨在给人们提供一个优雅的出行方式。共享单车App希望帮人们解决最后一公里的出行问题，让哪都有自行车可用，而且停放自由。

共享单车App的具体功能如下：

1.实时车辆地图：车辆实时定位，导航引导到达。

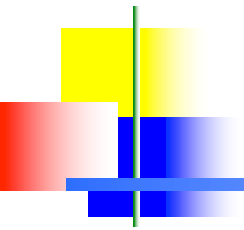
2.享受提前预约：提前预约周边单车，掌握行程安排。

3.一键扫码解锁：扫描车身二维码，即刻解锁用车。

4.随时随地轻松还车：无固定车桩，停放域即可还车。

5.在线支付：提供多种电子支付方式。





面向对象方法

6 系统建模

什么是系统？

系统（system），是指存在于一定环境中，由其部件按照一定的方式组成，实现一定功能，有形或者无形的…（东西）

- 存在于环境中
  - 与环境交互
- 由其部件构成
  - 包含一个或者多个部件或者子系统
- 实现一定的功能
  - 一个有意义的系统必须实现一些功能
- 有形或者无形
  - 有形的：硬件、生态系统、气候系统
  - 无形的：社会组织

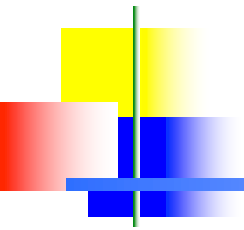
系统思维

- 从环境、功能、结构、性能等方面去看待一个系统，系统思维的四个任务：
  - ① 确定系统及其形式与功能
  - ② 确定系统中的实体及其形式与功能，以及系统的边界和外部环境
  - ③ 确定系统中各实体之间的关系，关系的形式和功能
  - ④ 根据实体的功能和它们之间的交互，确定系统的涌现（功能）
- 系统思维的目标：对系统进行结构化的思考，让复杂的系统看起来不那么复杂。

系统建模

系统建模是用形式化的方法，通过高效的工具，对系统的架构开展设计、迭代、验证等，并最终通过模型指导系统的实现。

- 把系统思维的过程形式化
- 利用工具把系统的功能、结构、行为等进行描述
- 对系统的特性进行验证
- 系统建模的两种场景
  - 系统设计（产品或者流程设计）
    - 已知系统的功能（Function），设计系统的形式（Form）
    - 自顶向下？
  - 系统分析（反向工程）
    - 已知系统的形式（至少是部分的），推断系统的功能、运行方式或系统的设计目标
    - 自底向上？



面向对象方法

7 统一建模语言UML

内容提要

① UML的基本概念

② 静态建模

③ 动态建模

④ 物理架构

⑤ 建模步骤



7.1 UML的基本概念

- UML的语义
- UML表示法



UML的语义

- 元模型为UML的所有元素在语法和语义上提供了简单、一致及通用的定义性说明。 使开发者能在语义上取得一致，消除因人而异的表达方法。
- UML语义还支持对元模型的扩展定义。
- UML的元素是基本构造单位，其中模型元素用于构造系统，视图元素用于构成系统的表示部分。
- UML定义的各种机制的语义，保持了UML的简单和概念上的一致。这些机制是依赖关系、约束、注记、标记值和定制等。
- UML支持的各种类型的语义，如布尔、表达式、列表、 值、名字、坐标、字符串和时间等，还允许用户自定义类型。

UML表示法

- UML的图形符号，为建模者和建模工具的开发者提供了标准的图形符号和正文语法。
- UML表示法的构成：
  - 视图（view）
  - 图（diagram）
  - 模型元素（model element）
  - 通用机制（general mechanism）
- UML表示法分为两部分：
  - 通用表示
  - 图形表示



通用表示

① 字符串：用于表示有关模型的信息。

② 名字：用于表示模型元素。

③ 标号：用于表示缚于图形符号的字符号。

④ 特定字符串：用于表示缚于模型元素的特性。

⑤ 类型表达式：用于声明属性变量和参数。

⑥ 定制：是一种机制，用已有的模型元素来定义新的模型元素。

图形

① 用例图（use-case diagram）
- 表示各类外部参与者与系统所提供的用例之间的连接。
- 一个用例是系统所提供的一个功能的描述。
- 给出了用户所感受到的系统行为，但不描述系统如何实现该功能。

② 类图（class diagram）
- 表示系统中类的静态结构，即类与类之间的相互联系。
- 一个系统可以有多张类图，一个类也可以出现在几张类图中。

③ 对象图（object diagram）
- 对象图是类的实例，它表示系统执行在某一时间点的一个可能的快照（snapshot）。

④ 状态图（state diagram）
- 状态图通常是对类描述的补充，它说明该类的对象所有可能的状态以及哪些事件将导致的改变。

（续）

⑤ 时序图（ sequence diagram）
- 时序图表示几个对象之间的动态协作关系。
- 它主要用来分析对象之间发送消息的顺序。

⑥ 协作图（collaboration/communication diagram）
- 与时序图一样，协作图表示对象间的动态协作关系。
- 它除了说明消息的交互外，还显示对象及其间的关系。

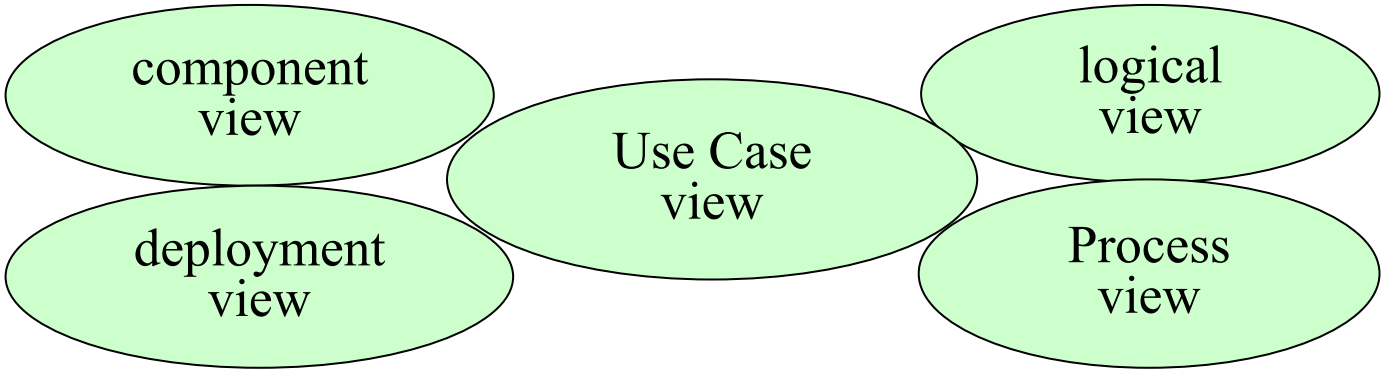
⑦ 活动图（activity diagram）
- 表示连续的活动流，通常用来描述一个操作所需要的活动。

⑧ 构件图（component diagram）
- 以代码构件为单位表示代码的物理结构，及构件之间的依赖关系。

⑨ 部署图（deployment diagram）
- 表示系统中硬件和软件的物理结构，及各部分之间的连接方式。

视图

- 一个系统应从不同的角度进行描述，从一个角度观察到的系统称为一个视图（view）。
- 视图由多个图（diagram）构成，它不是一个图表（graph），而是在某一个抽象层上，对系统的抽象表示。
- 如果要为系统建立一个完整的模型图，需定义一定数量的视图，每个视图表示系统的一个特殊的方面。另外，视图还把建模语言和系统开发时选择的方法或过程连接起来。



视图

① 用例视图（use-case view）
- 表示外部参与者看到的系统将提交的功能。

② 逻辑视图（logical view）
- 表示系统内部如何提供系统的功能。
- 静态结构用类图和对象图来描述，动态行为用状态图、时序图、协作图、活动图来描述。

③ 构件视图（component view）
- 构件视图表示代码构件的组织，描述了实现模块及它们之间的依赖关系。
- 构件视图针对开发者。

③ 进程视图（process view）
- 表示系统的并发性，包括进程的通信和同步。
- 用动态图（即状态图、时序图、协作图和活动图）和实现图（即构件图和部署图）来描述。
- 针对开发者和系统集成者。

⑤ 部署视图（deployment view）
- 表示系统的物理部置。
- 构件被部署到物理结构中的映射，用部署图来描述。
- 它针对开发者、集成者和测试者。

建模工具

- Visual Paradigm for UML
- Microsoft Visio Professional
- PowerDesigner
- Select Component Architect
- Visual UML
- Borland Together
- Gentleware Poseidon
- astah
- OmniGraffle

7.2 UML的基本元素

- 关系
- 基本图形



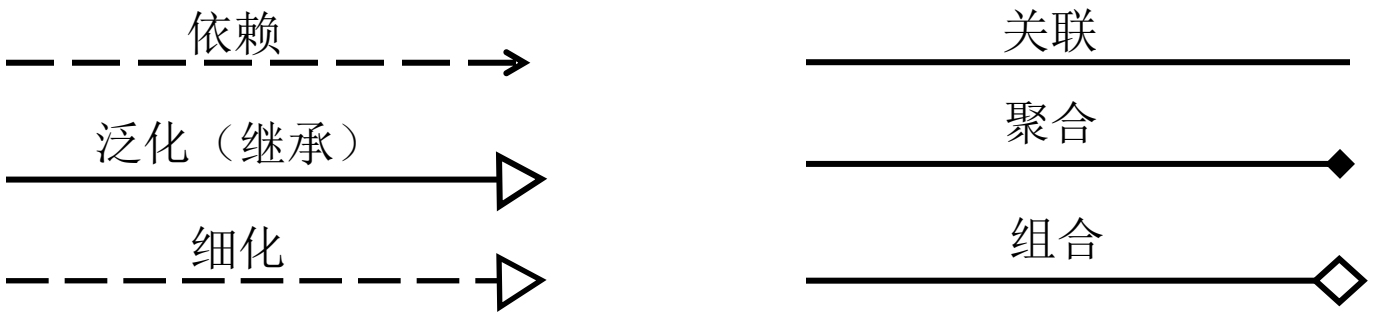
基本元素

- 分类：
  - 参与者（ator）
  - 用例（use case）
  - 类（class）
  - 对象（object）
  - 接口（interface）
  - 子系统（subsystem）
  - 包（package）
  - 组件（component）
  - 结点（node）
  - 注释（comment)
- 关系：
  - 关联（association）
  - 泛化（generalization）
  - 依赖（dependency）
  - 实现（realization）
  - 约束（constraint)



基本元素—关系

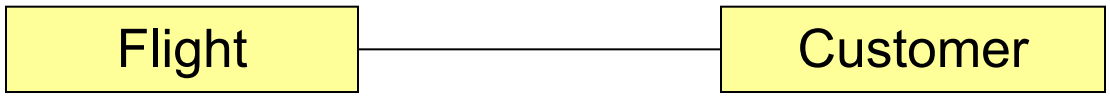
模型元素与模型元素之间的连接关系也是模型元素，常见的关系有关联（association）、泛化（generalization）、依赖（dependency）和聚合（aggregation），其中聚合是关联的一种特殊形式。



- 关联：连接（connect）模型元素及链接（link）实例。
- 依赖：表示一个元素以某种方式依赖于另一种元素。
- 泛化：表示一般与特殊的关系，即“一般”元素是“特殊”关系的泛化。
- 聚合：表示整体与部分的关系。

基本元素—关系—关联

- 关联（association）是类之间的语义连接（link），使一个类知道另一个类的公共属性和操作。
- 关联可以是单向的也可以是双向的。

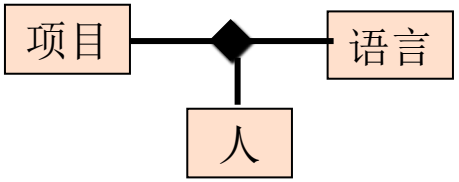


关联的表示

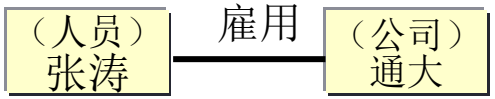
- 关联有二元关联(binary)、三元关联(ternary)、多元关联(higher order)。



二元关联的例

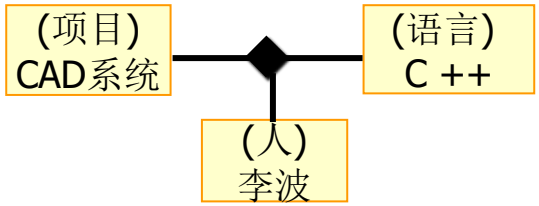


三元关联的例



链的例子

（a） 二元关联

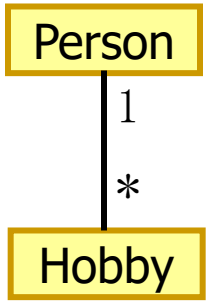


链的例子

（b）三元关联

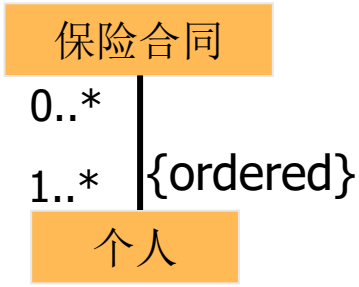
关联的重数

- 重数（multiplicity）表示多少个对象与对方对象相连接，它的默认值为1，常用的重数表示符号有：
  - 0..1　　表示零或1
  - 0..* 或*　　表示零或多个
  - 1..*　　表示1或多个
  - 1,3,7　　表示1、3或7（枚举型）

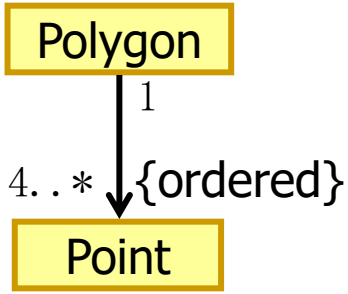


带有多重性关联

- 在关联的多端标注{ordered}指明这些对象是有序的。
- 关联可以用箭头，表示该关联使用的方向（单向或双向），称为导引或导航（navigation）。



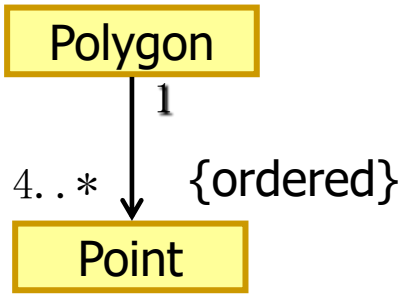
(a)指定链接之间有明确的顺序

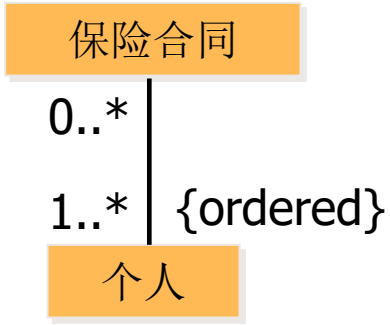


(b)单向关联

关联的约束

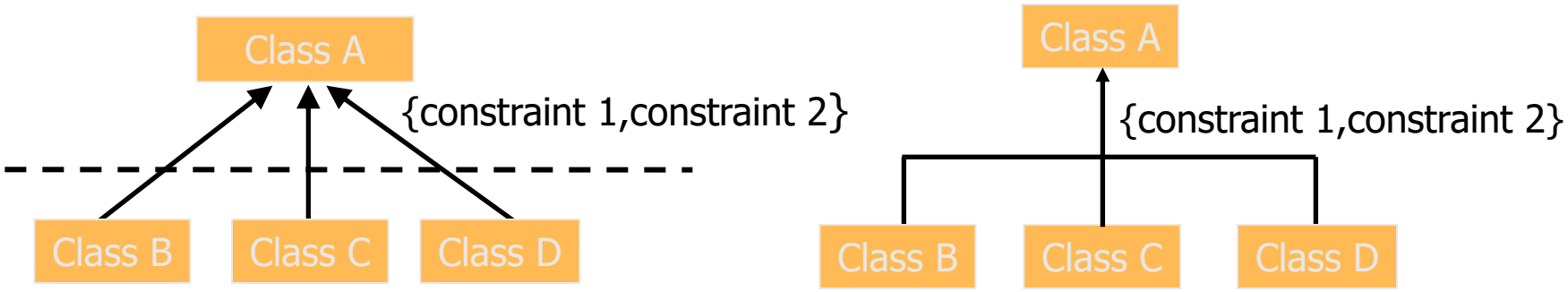
- 约束（constraint）是各种模型元素的一种语义条件或限制。一条约束只能应用于同一类的元素。
- 如果约束应用于一种具有相应视图元素的模型元素，它可以出现在它所约束元素视图元素的旁边。
- 通常一个约束由一对花括号括起来（{constraint}），花括号中为约束内容。
- 如果一条约束涉及同一种类的多个元素，则要用虚线把所有受约束的元素框起来，并把该约束显示在旁边（如或约束）。





泛化的约束

- 约束可分为：泛化的约束、关联的约束。
- 对泛化的约束
  - 显示在大括号里，若有多个约束，用逗号隔开。
  - 如果没有共享，则用一条虚线通过所有继承线，并在虚线的旁边显示约束。



对泛化的约束的两种表示方法

对泛化有以下常用的约束：

① complete：泛化中所有子元素都已在模型中说明，不允许再增加其它子元素。

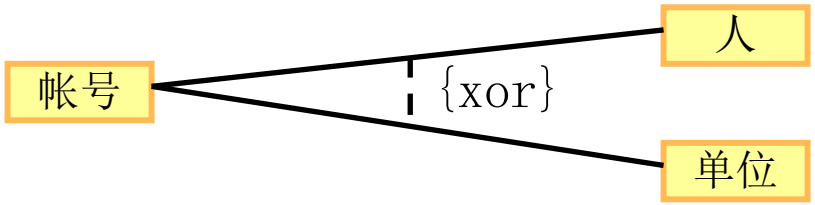
② incomplete：不是泛化中所有子元素都已说明，允许再增加其它子元素。

③ disjoint：父类对象不能有多于一个型的子对象。

④ overlapping：给定父类对象可有多于一个型的子对象，表示重载。

关联的约束

- 对关联有以下常用的约束：
  ① implicit：该关联只是概念性的，在对模型进行精化时不再用。
  ② ordered：具有多重性的关联一端的对象是有序的。
  ③ changeable：关联对象之间的链（link）是可变的（添加、修改、删除）。
  ④ addonly：可在任意时刻增加新的链接。
  ⑤ frozen：冻结已创建的对象，不能再添加、删除和修改它的链接。
  ⑥ xor：“或约束”，某时刻只有一个当前的关联实例。

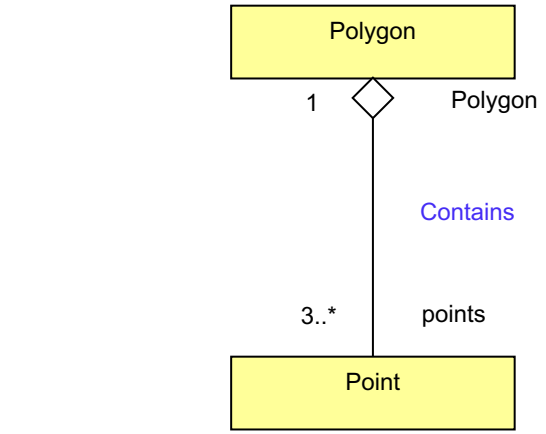


对象类的xor关联

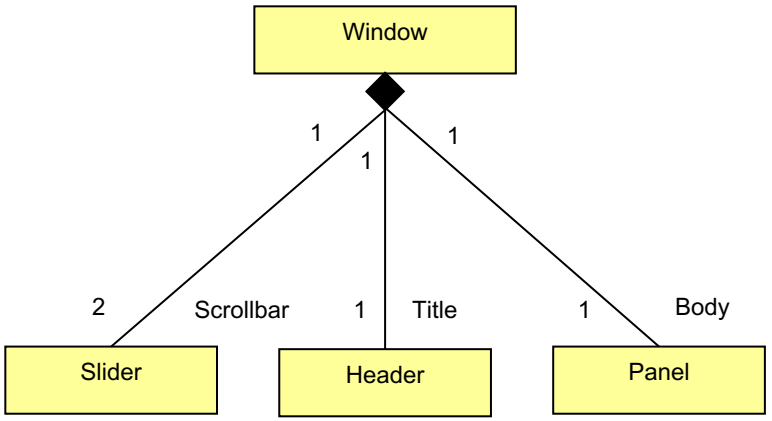
- 对消息，链接角色和对象的约束
- 自定义约束

基本元素—关系—聚集

- 聚集（Aggregation）用来表达整体——部分关系的关联。
- 聚集是特殊的关联，它可以有重数、角色、限制等。聚集分为：
  - ◇ 共享（shared）聚集，它的“部分”对象可以是任意“整体”对象的一部分。
  - ◆ 组合（Composition）聚集，是指它的“整体”拥有它的“部分”，是关联更强的形式。



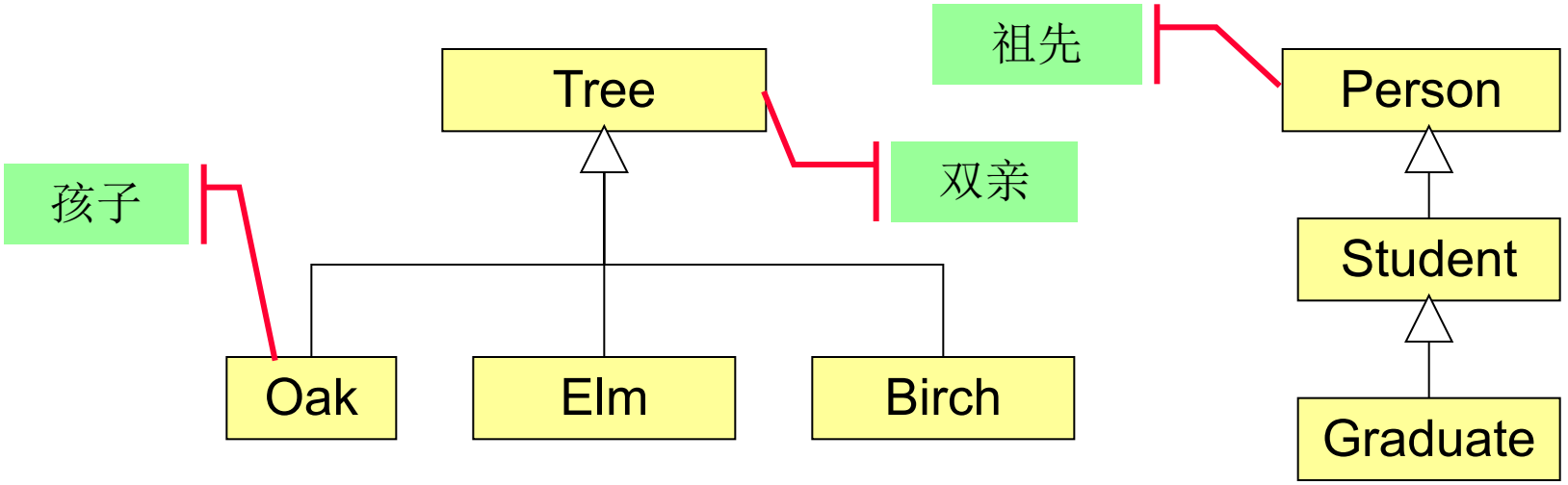
共享聚集



组合聚集

基本元素—关系—泛化

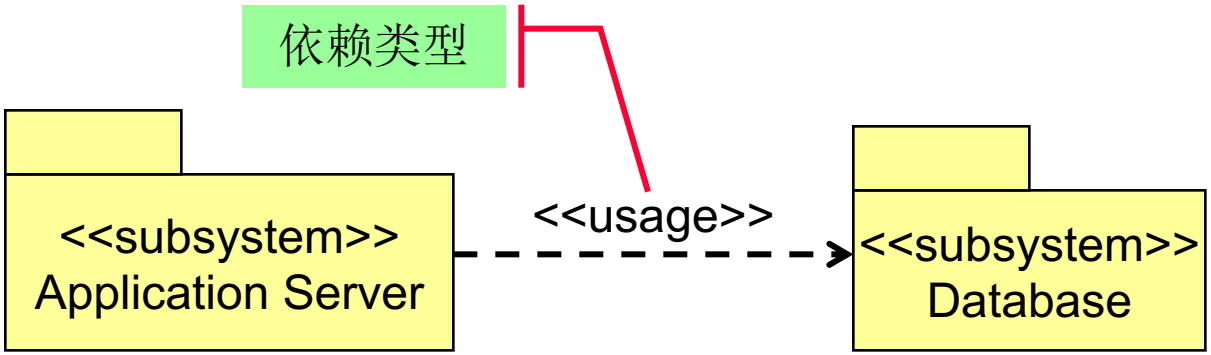
- 泛化（generalization）是一般和具体之间的一种关系。
- 继承（inherit）就是一种泛化关系，更一般化的描述称为双亲，双亲的双亲称为祖先，更具体化的描述称为孩子，在类的范畴，双亲对应超类，孩子对应派生类。



基本元素—关系—依赖

- 依赖（dependency）指明一个元素引用另一个元素的关系，它可分为类与类之间的依赖和包与包之间的依赖。
- 依赖有很多种类，比如：

实现（realize）	使用（usage）	实例化（instantiate）
调用（call）	派生（derive）	访问（access）
引入（import）	友元（friend）	……



依赖的形式

依赖的形式可能是多样的，针对不同的依赖的形式，依赖关系有不同的变体(varieties)：

① 抽象(abstraction)：从一个对象中提取一些特性，并用类方法表示。

② 绑定(binding)：为模板参数指定值，以定义一个新的模板元素。

③ 组合(combination)：对不同类或包进行性质相似融合。

④ 许可(permission)：允许另一个对象对本对象的访问。

⑤ 使用(usage)：声明使用一个模型元素需要用到已存在的另一个模型元素，这样才能正确实现使用者的功能（包括调用、实例化、参数、发送）。

⑥ 跟踪(trace)：声明不同模型中元素的之间的存在一些连接。

⑦ 访问或连接(access)：允许一个包访问另一个包的内容。

（续）

⑧ 调用(call)：声明一个类调用其他类的操作的方法。

⑨ 导出(derive)：声明一个实例可从另一个实例导出。

⑩ 友元(friend)：允许一个元素访问另一个元素，不管被访问的元素是否具有可见性。

⑪ 引入(import)：允许一个包访问另一个包的内容，并为被访问组成部分增加别名。

⑫ 实例(instantiation)：关于一个类的方法创建了另一个类的实例声明。

⑬ 参数(parameter)：一个操作和它参数之间的关系。

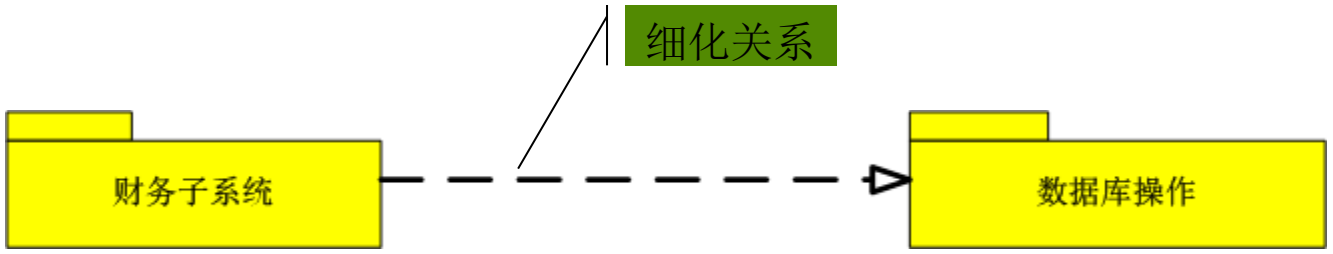
⑭ 实现(realize)：说明和其实之间的关系。

⑮ 精化(refine)：声明具有两个不同语义层次上的元素之间的映射。

⑯ 发送(send)：信号发送者和信号接收者之间的关系。

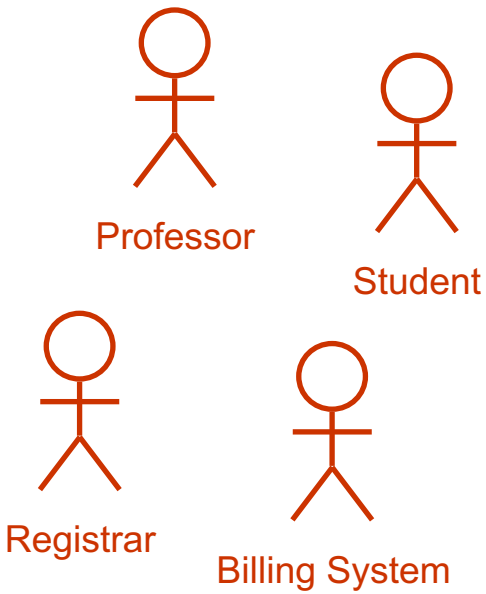
基本元素—关系—细化

- 细化（refinement）是对同一事物不同级别的两种描述之间的一种关系。
- 例如：对同一事物建模的分析类与设计类之间的就是细化关系。



基本元素—参与者（actor）

- 参与者是与系统、子系统或类交互的外部人员，进程或事务。在运行时，具体人员会充当系统的多个参与者，不同用户可能会成为一个参与者。



- **如何确定参与者：**

① 谁使用系统的主要功能(主参与者)?

② 谁需要从系统获得对日常工作的支持和服务？

③ 需要谁维护管理系统的日常运行（副参与者）？

④ 系统需要控制哪些硬件设备？

⑤ 系统需要与其它哪些系统交互？

⑥ 谁需要使用系统产生的结果（值）？

基本元素—用例（use case）

- 用例是系统提供的外部可感知的功能单元，它的作用是定义清晰的系统行为，但不解释系统的内部结构。

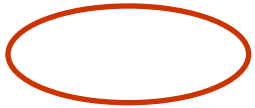
- **如何确定用例：**

① 与系统实现有关的主要问题是什么？

② 系统需要哪些输入/输出？这些输入/输出从何而来？到哪里去？

③ 参与者需要系统提供哪些功能？

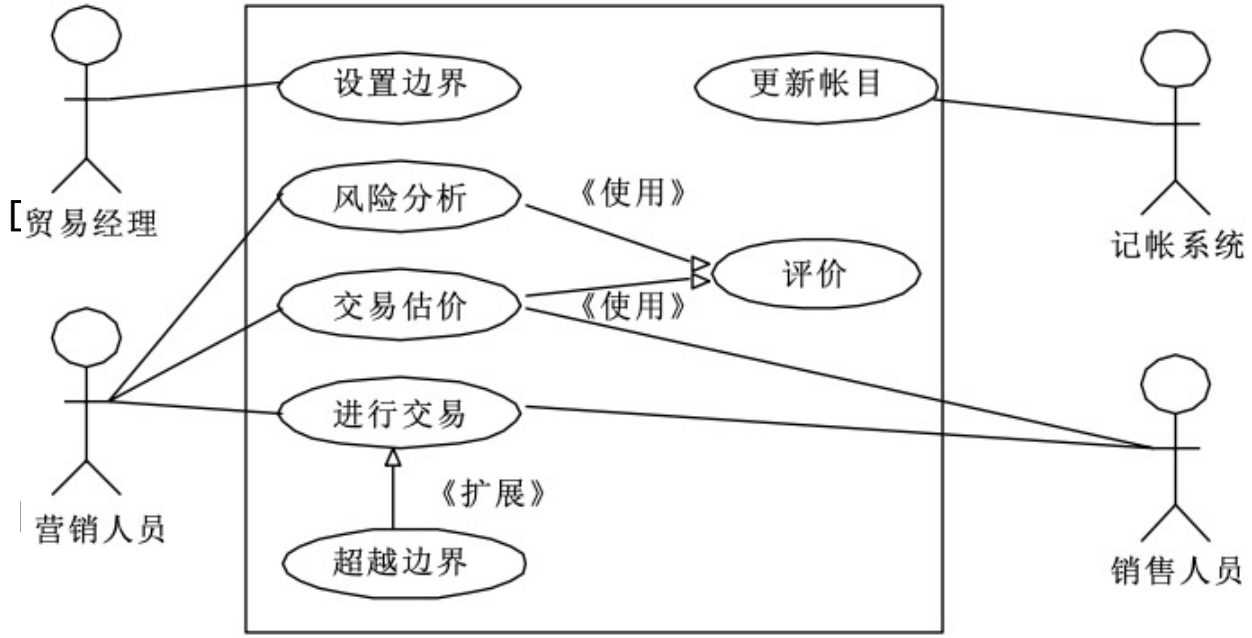
④ 参与者是否需要对系统中的信息进行读、创建、修改、删除或存储？



Maintain Curriculum

用例用椭圆来表示，用例名标在椭圆下方，用实线与同自身通信的用户相连。

例：金融贸易系统的用例图



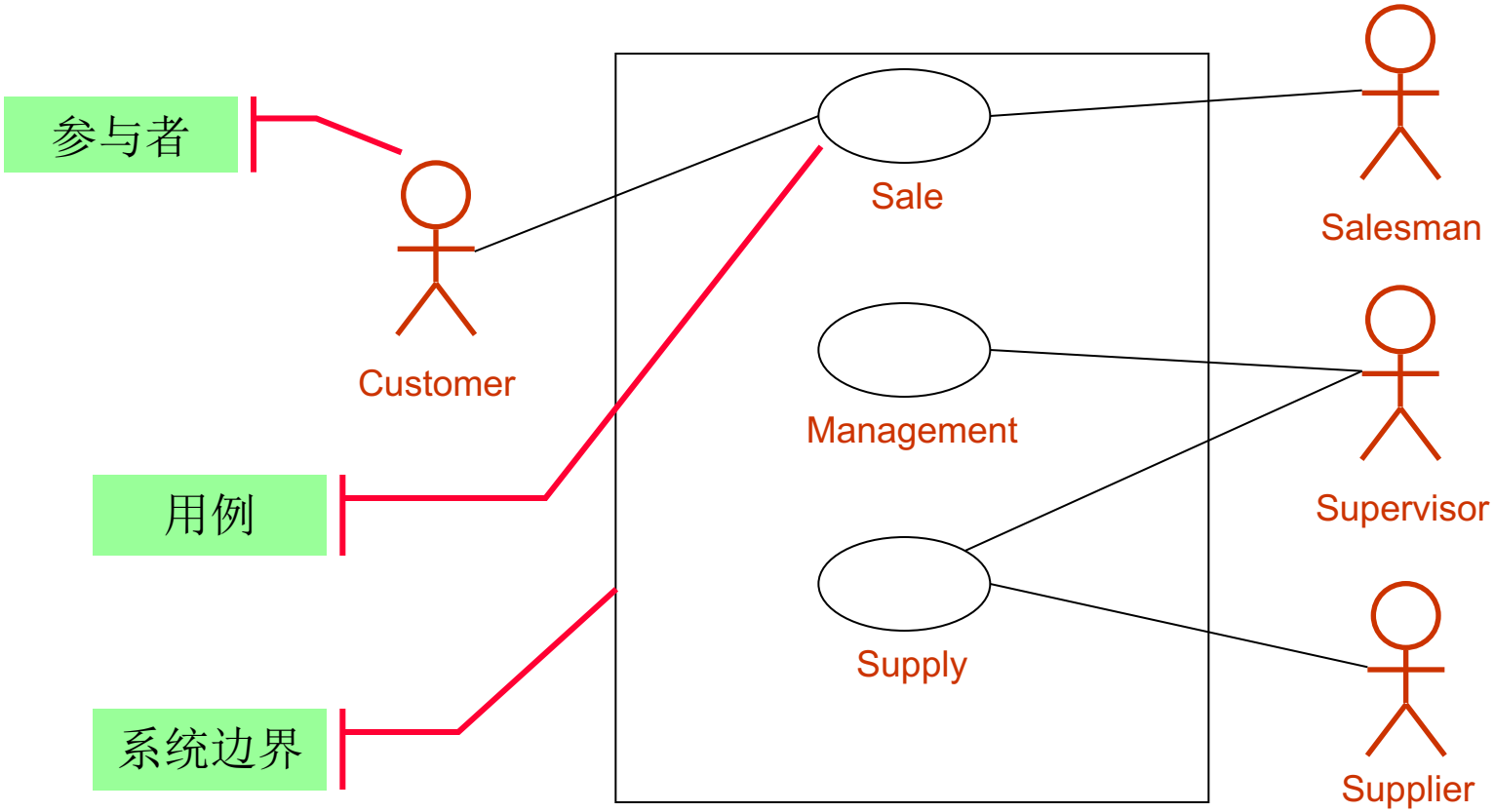
- 用例可以与参与者关联，也可以参与其他多种关系，比如扩展、泛化和包含等。
- 用例有以下特点:
  - 用例捕获某些用户可见的需求，实现一个具体的用户目标。
  - 用例由参与者激活，并将结果值反馈给参与者。
  - 用例必须具有功能上的完整描述。

用例图（use-case diagram）

- 用例（Use Case）：在不展现一个系统或子系统内部结构的情况下，对系统或子系统的某个连贯的功能单元的定义和描述。
- 用例图描述各个参与者在各个用例中的参与情况，描述系统为用户所感知的外部视图。
- 用例图的功能：
  - 捕获用户需求
  - 描述系统边界
  - 指明系统外部行为
  - 指导系统开发者的功能开发
  - 系统建模的起点，指导所有的类图和交互图的设计
  - 产生测试用例，用户文档
  - 估计项目大小和进度。

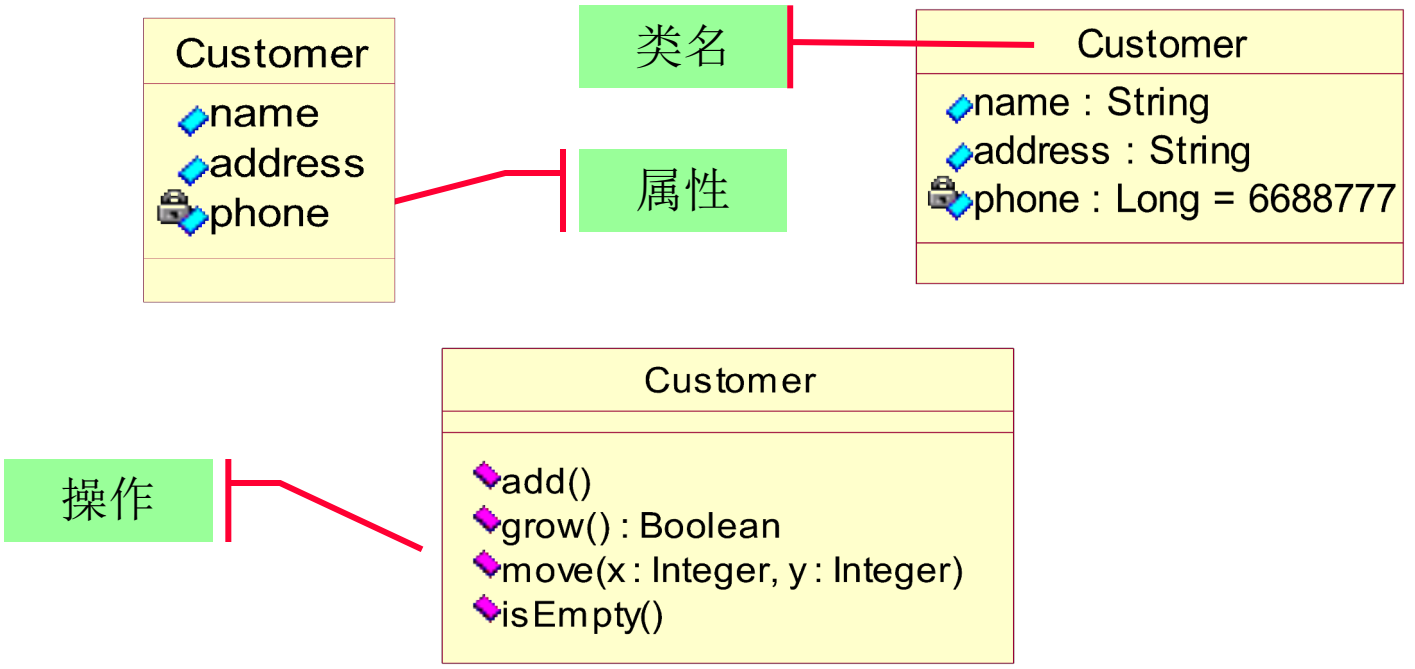
用例图示例

- 用例可以参与多种关系：关联、扩展、泛化和包含。



基本元素—类（class）

- 类是具有相同属性、操作和关系的对象集合的总称。
- 通常在UML中类被画成矩形，包括三个部分：
  - 名称：
  - 属性：
  - 操作：

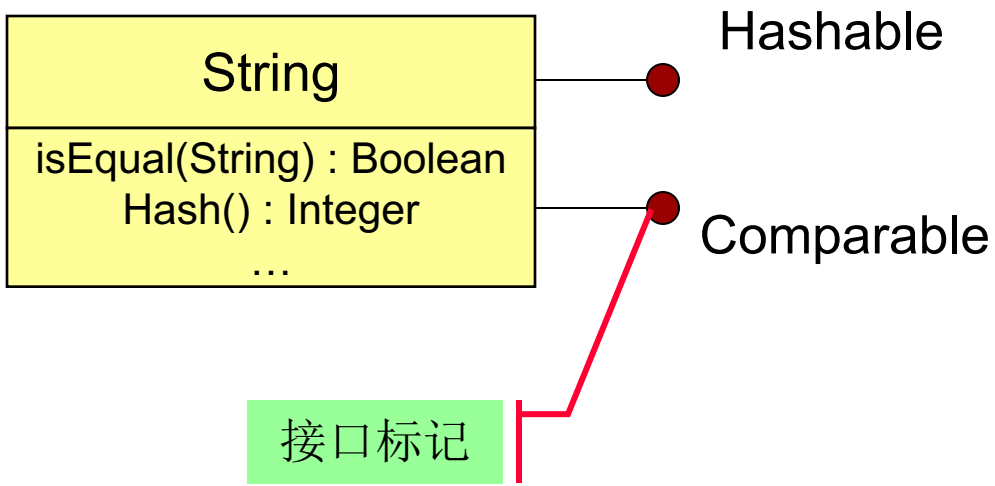


类图符的构成

- 名称：
  - 每个类都必须有一个名字，用于区别其它类。类名是一个字符串，称为简单名字。路径名字是在类名前加包含类的包名为前缀。例如Wall、java::awt::Wall都是合法的类名。
- 属性：
  - 类可以有任意多个属性，也可以没有属性。在类图中属性只要写上名字就可以了，也可以在属性名后跟上类型甚至缺省取值 。
- 操作：
  - 操作是类的任意一个实例对象都可以调用的，并可能影响该对象行为的实现。

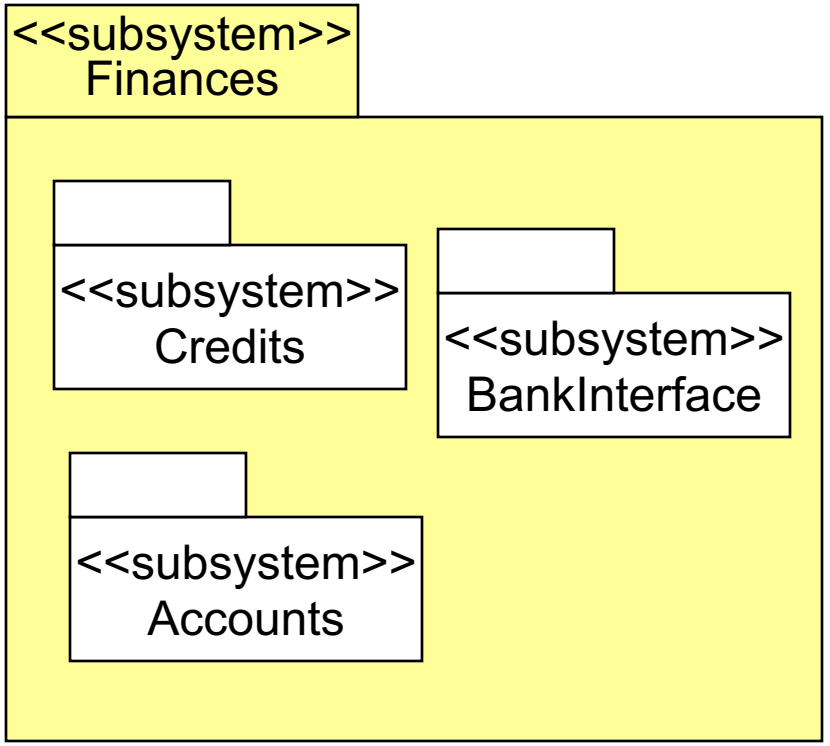
基本元素—接口（interface）

- 接口是未给出实现的对象行为的描述，接口包含操作，但没有属性，一个或多个类可以实现接口，每个类实现接口的操作。



基本元素—子系统（subsystem或package）

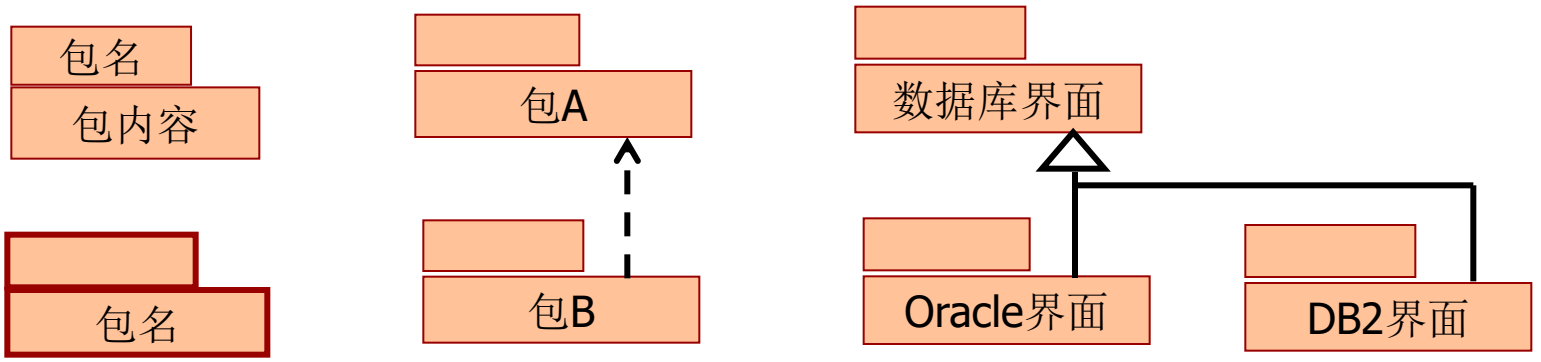
- OO解决问题的思路之一是将许多类集合成一个更高层次的单位，形成一个高内聚、低耦合的类的集合。UML中这种分组机制叫子系统（subsystem）或包(package)。
- 引入包是为了降低系统的复杂性，以便人们在某一时刻可以只处理有限的信息，使团队的工作不相互影响。



包的构成

- 包是一种组合机制，把各种各样的模型元素通过内在的语义连在一起成为一个整体就叫包，构成包的模型元素称为包的内容，包通常用于对模型的组织管理。
- 包可以包含各种模型元素和其它的包，如：可以是类的列表，也可以是另一个包图，还可以是一个类图。
- 包之间的关系有依赖和泛化（继承）。
- 包拥有自己的模型元素，包与包之间不能共用一个相同的模型元素，包的实例没有任何语义（含义）。仅在模型执行期间包才有意义。

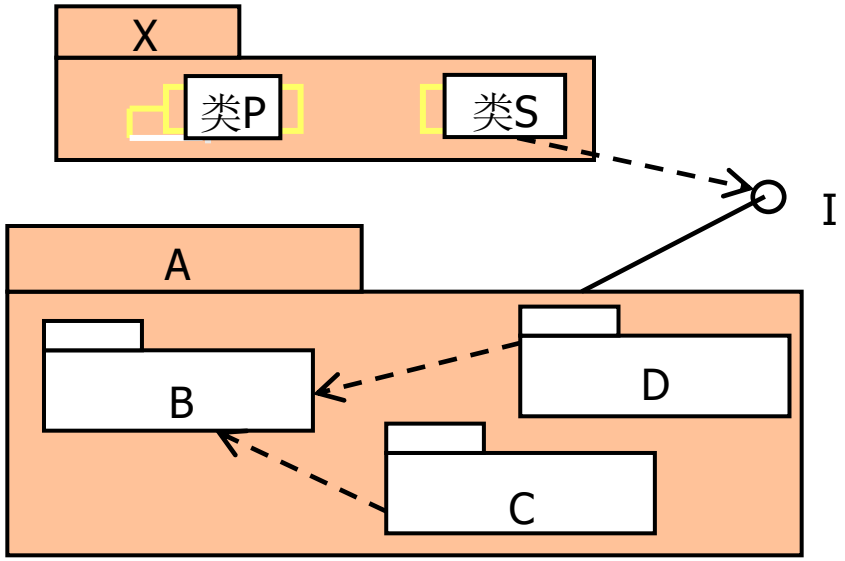
包之间的关系



（a）包的表示　(b)包的依赖关系　(c) 包的泛化关系

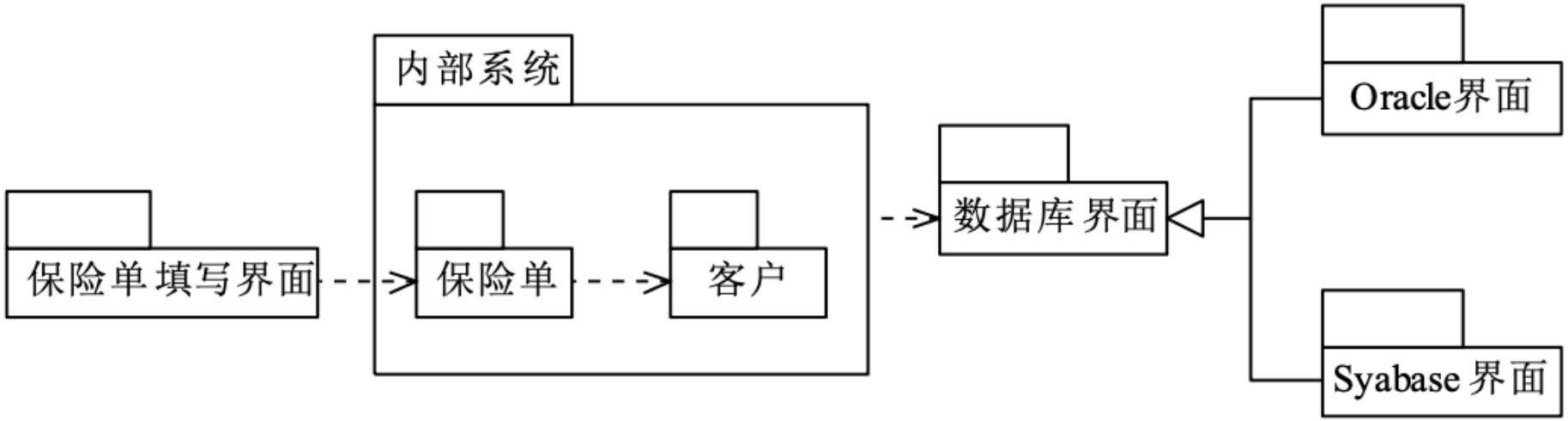
- 依赖关系：两个包中的任意两个类存在依赖关系，则包之间存在依赖关系。表示为：
- 泛化关系：使用继承中通用和特例的概念来说明通用包和专用包之间的关系。例如，专用包必须符合通用包的界面，与类继承关系类似。表示为：

包的可见性

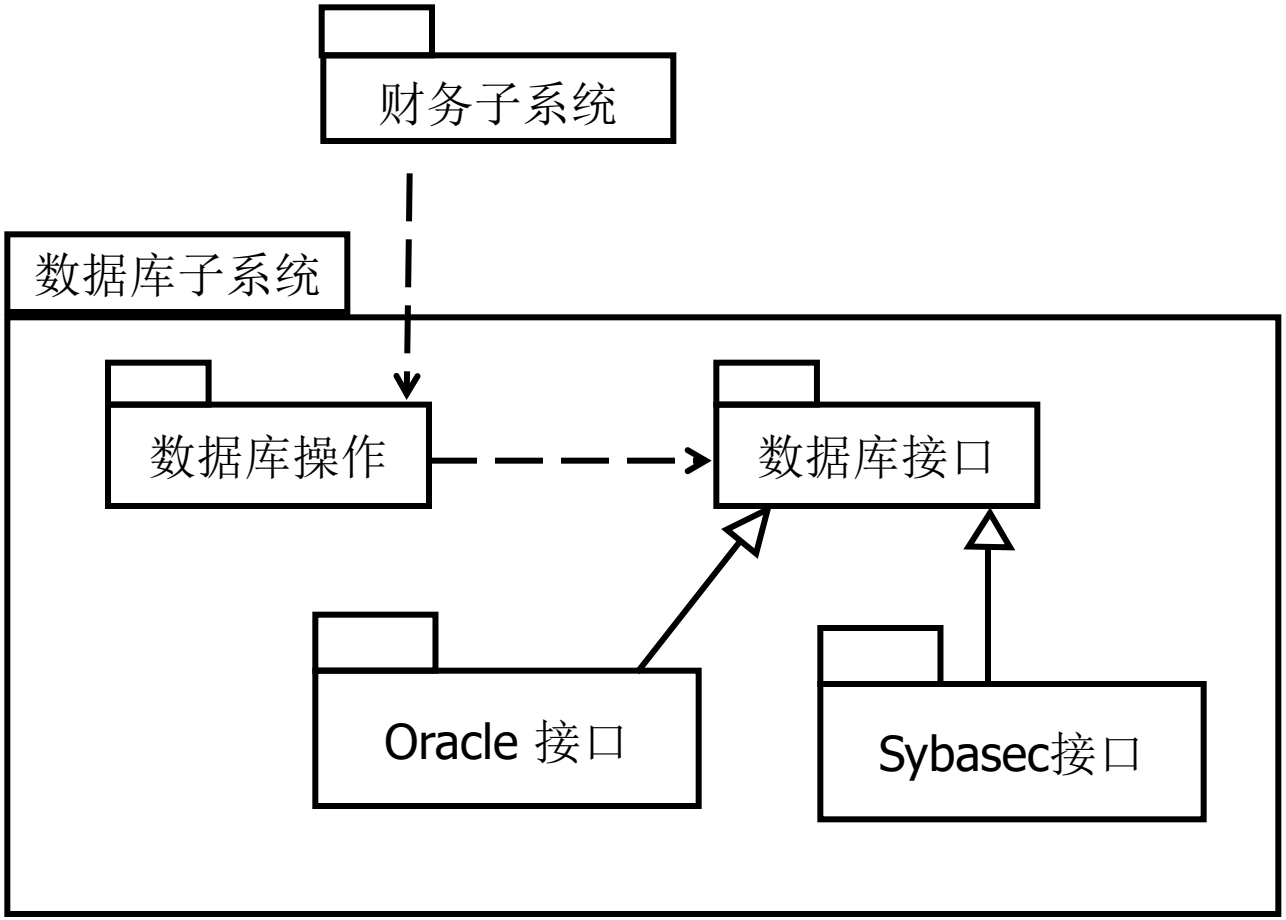


- 和类一样包也有可见性，利用可见性控制外部包对包的内容的存取方式，UML中定义了四种可见性：私有（private），公有（public），保护（protected）和实现（implementation）。缺省值为公有。
- 包也可以有接口，接口与包之间用实线相连，接口通常由包的一个或多个类实现。

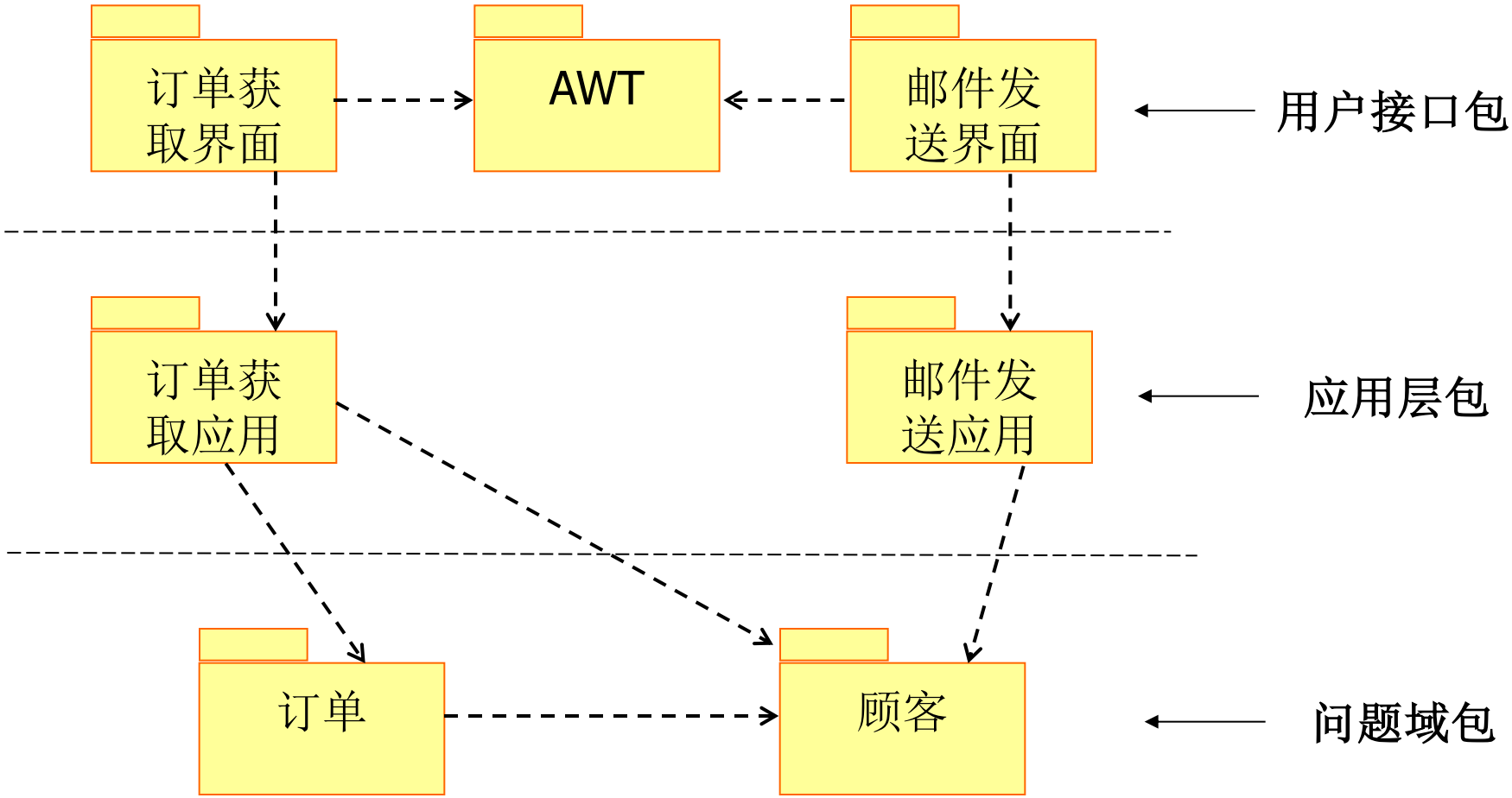
保险信息管理系统的包图



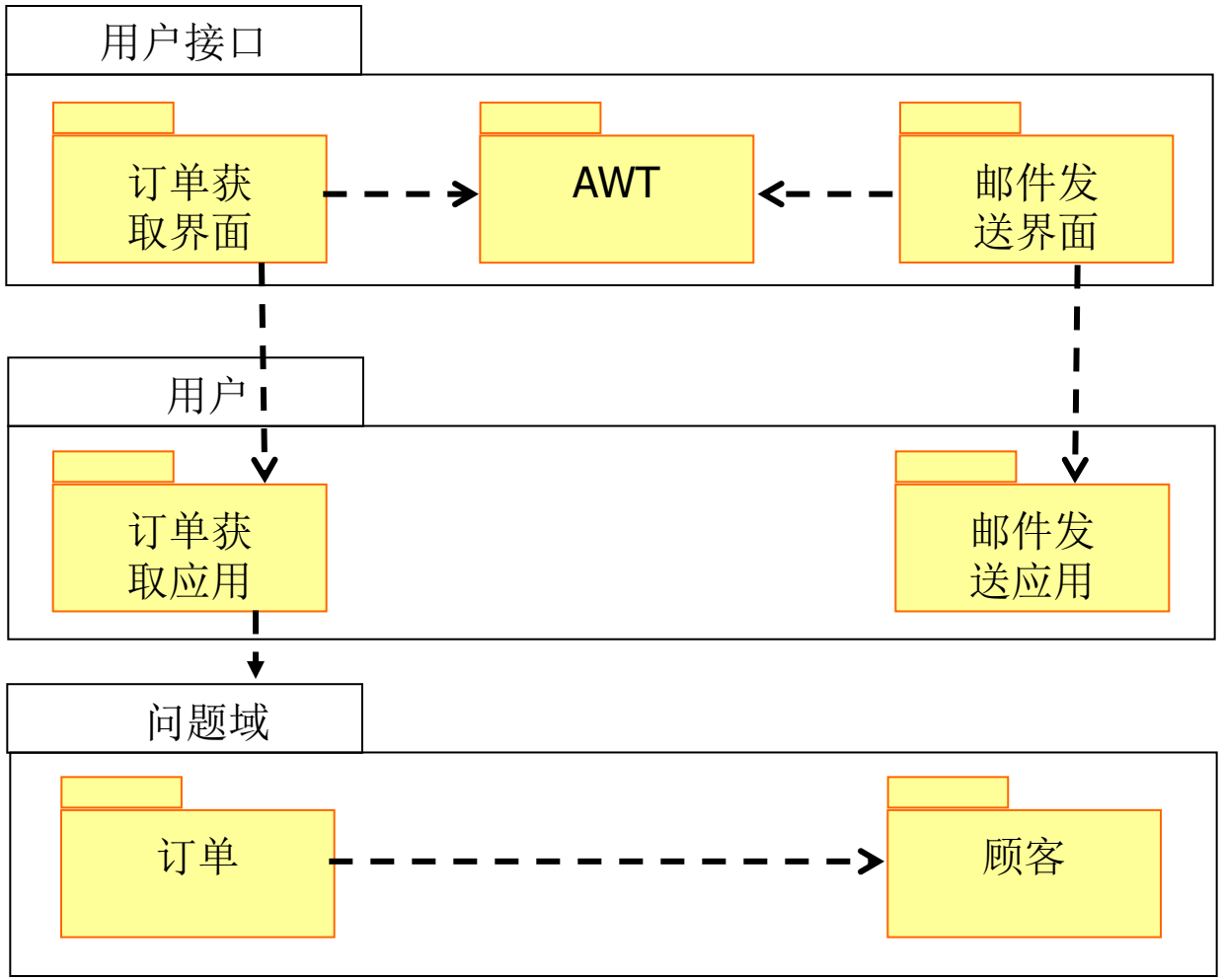
包图举例



（续）

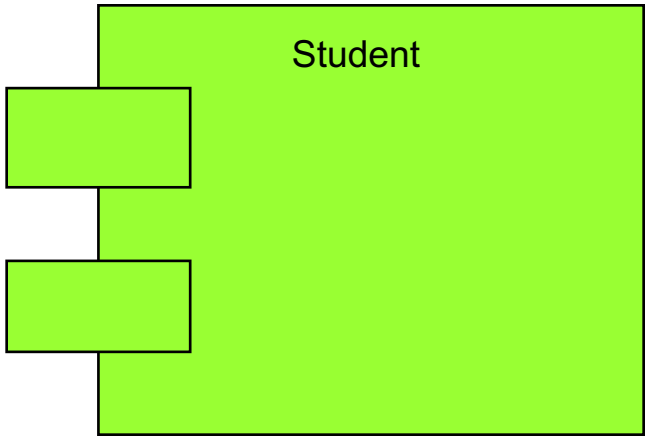


包图另一种划分方式



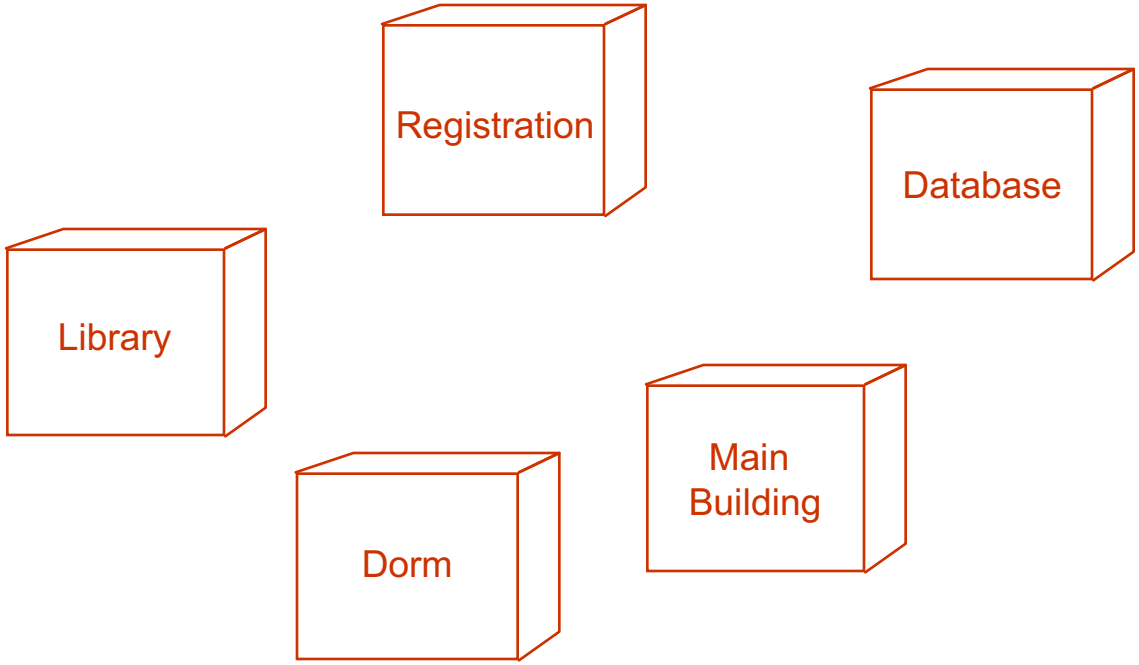
基本元素—组件（component）

- 组件是可重用的系统片段，具有良好接口定义的物理实现单元。每个组件包含了系统设计中某些类的实现。
- 组件设计的原则：良好的组件不直接依赖于其它组件的实现，只依赖于其它组件的接口。这样的好处是系统中的组件可以被支持相同接口的组件所取代。
- 组件可能是源代码、可执行程序或动态库。



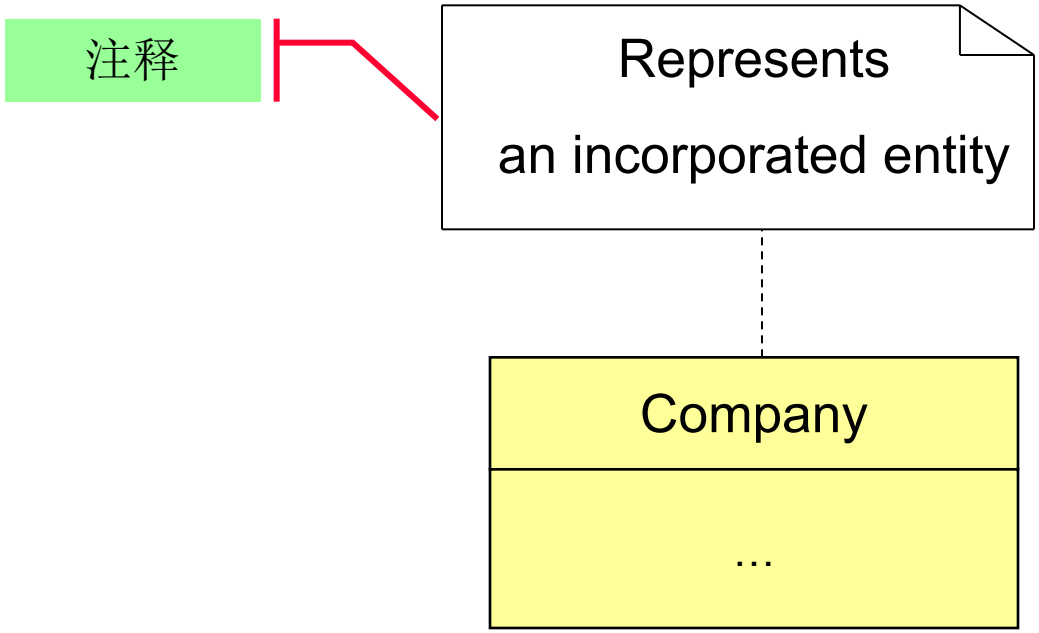
基本元素—结点（node）

- 结点代表系统运行时的物理对象，结点通常拥有运算能力，它可以容纳对象和组件实例。



基本元素—注释（note）

- 注释用于解释设计的思路，便于理解。
- 一个好的模型应该有详尽的注释。



7.3 UML对静态建模的支持

- 静态模型表示
- 静态建模



静态建模

- 所谓静态建模是指对象之间通过属性互相联系，而这些关系不随时间而转移。类和对象的建模，是UML建模的基础。
- 熟练掌握基本概念、区分不同抽象层次以及在实践中灵活运用，是三条最值得注意的建模基本原则。
- UML的静态建模机制包括：
  - 用例图(Use case diagram)
  - 类图(Class diagram)
  - 对象图(Object diagram)
  - 包图(Package diagram)
  - 构件图(Component diagram)
  - 配置图(Deployment diagram)

用例模型(use-case model)

- 用例模型描述了外部参与者（Actor）所理解的系统功能。
- 它描述了待开发系统的功能需求，它驱动了需求分析之后各阶段的开发工作 。
- 用例模型由若干个用例图构成，主要描述参与者和用例之间的关系。

创建用例模型的工作包括：

定义系统、确定执行者和用例、描述用例、定义用例间的关系、确认模型。

例：建立项目与资源管理系统的用例图

PRMS系统的主要功能是：

- 项目管理：包括项目的增加、删除、更新。
- 资源管理：包括对资源和技能的添加、删除和更新。
- 系统管理：包括系统启动和关闭，数据存储和备份等功能。

1.分析确定系统的执行者（角色）

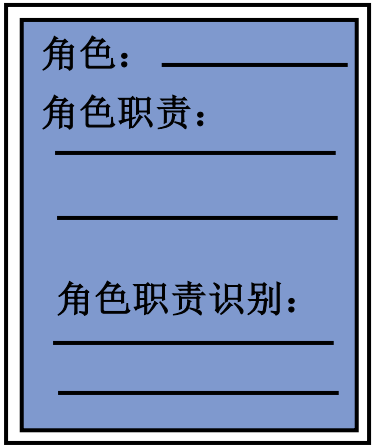
项目管理员、资源管理员、系统管理员、备份数据系统。

2.确定用例

项目管理、资源管理和系统管理。

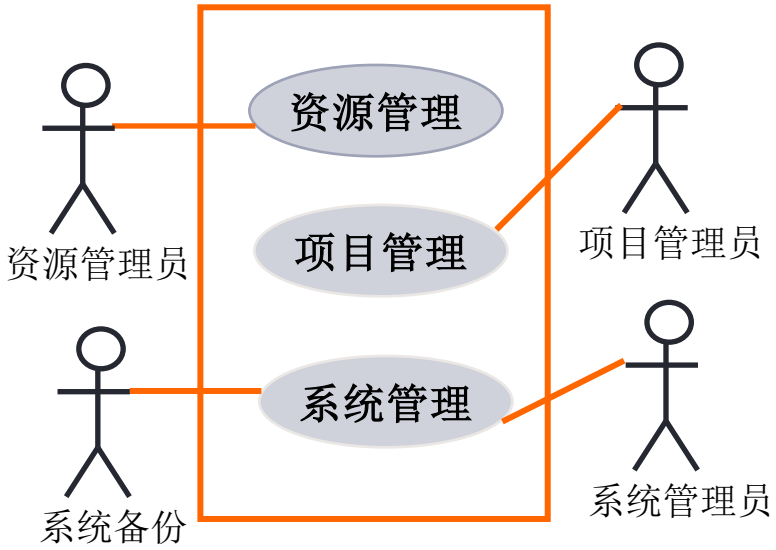
3.对用例进行分解，画出下层的Use case图

对上层的用例进行分解，并将执行者分配到各层次的用例图中。还应画出相应的执行者描述模板及用例描述模板。



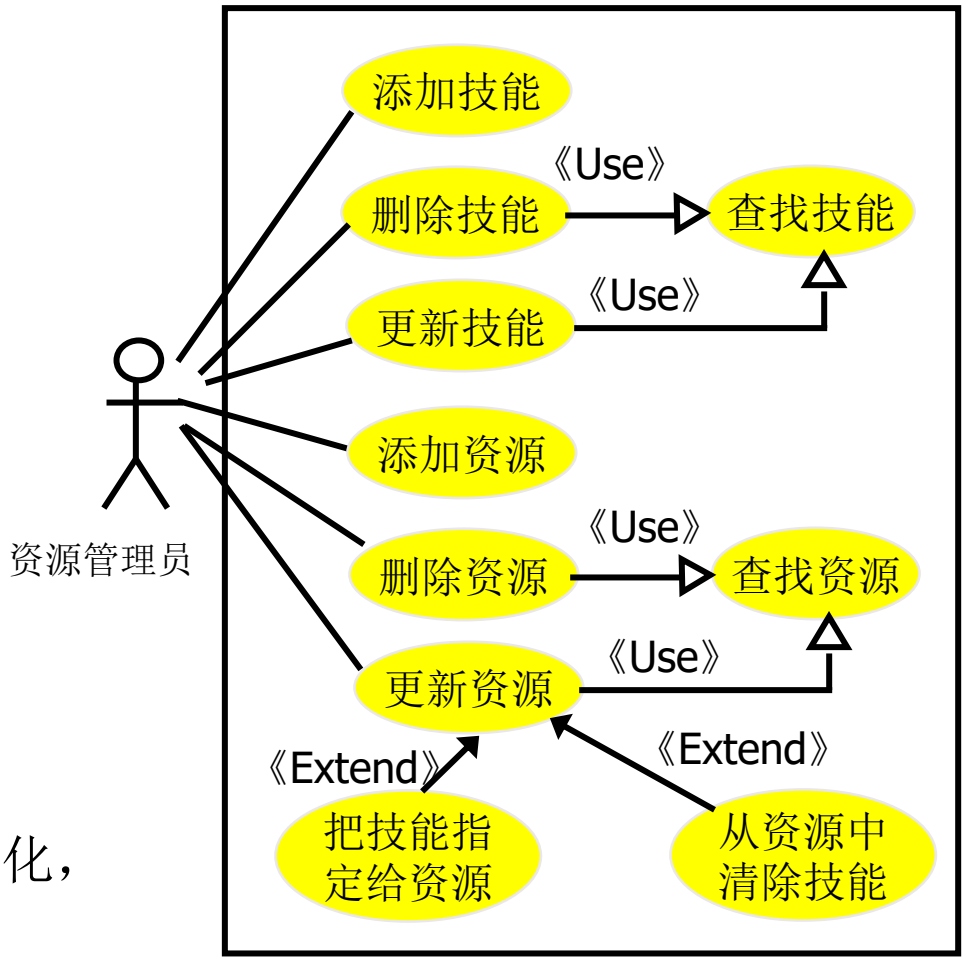
角色描述模板

例续：项目与资源管理系统（PRMS）



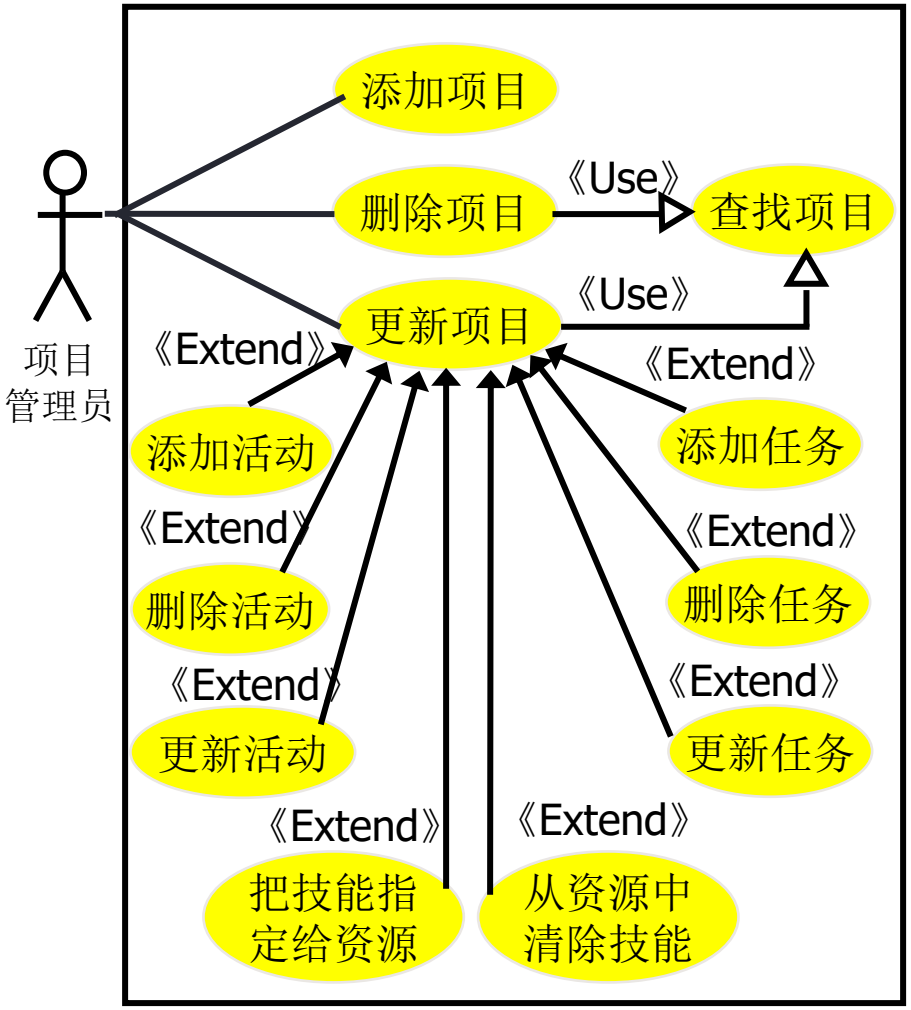
PRMS高层Use Case图

Use Case图可以自顶而下不断精化，抽象出不同层次的Use Case图。

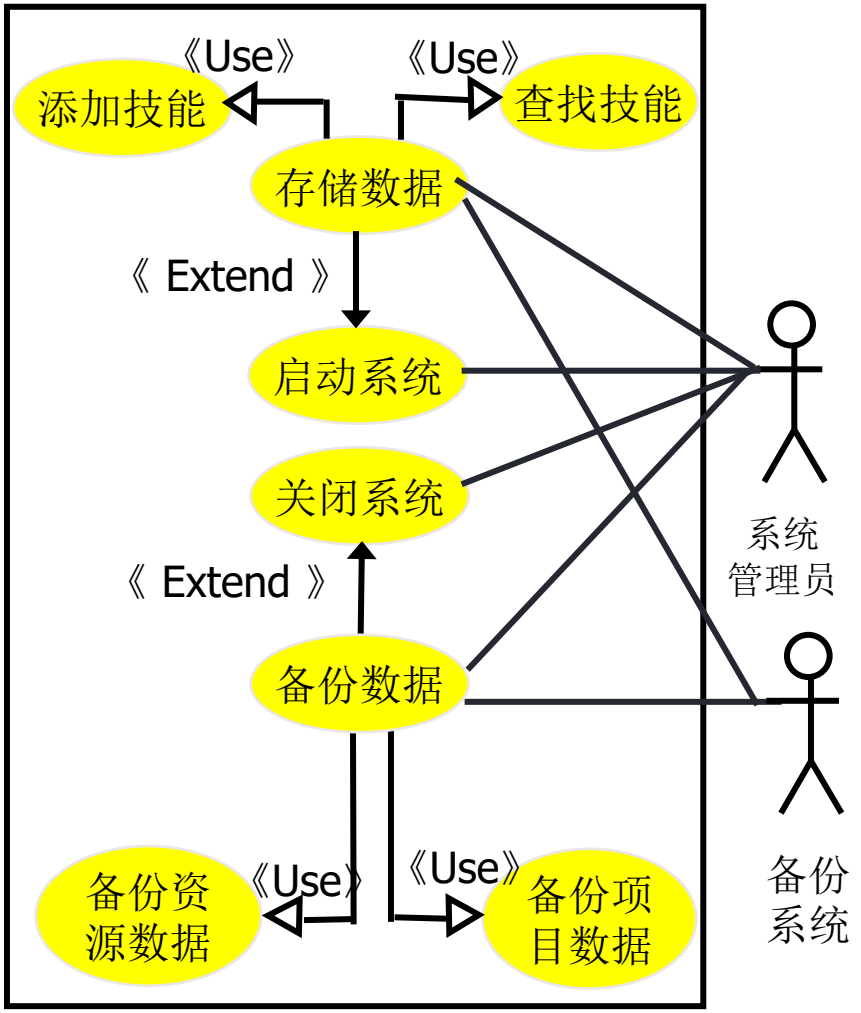


资源管理Use Case图

例续：项目与资源管理系统（PRMS）



项目管理Use Case图



系统管理Use Case图

案例——医院病房监护系统

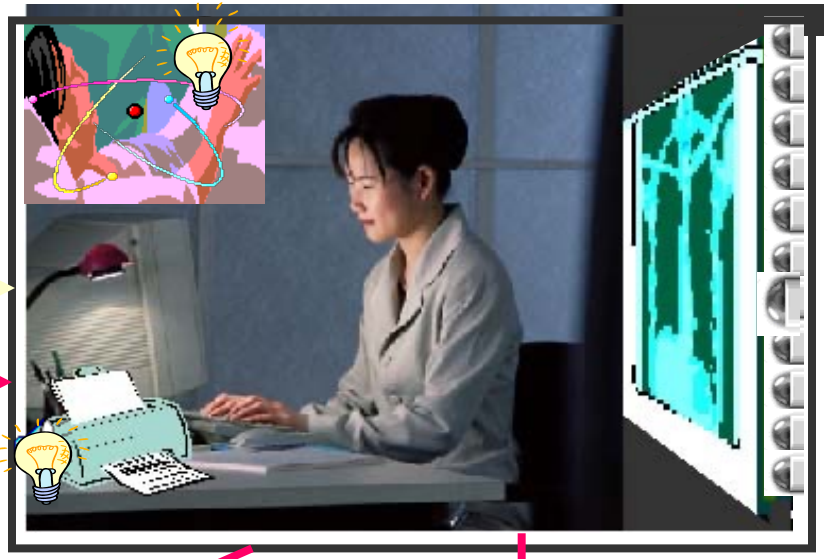
现有一医院病房监护系统，病症监视器安置在每个病房，将病人的病症信号实时传送到中央监视系统进行分析处理。在中心值班室里，值班护士使用中央监视系统对病员的情况进行监控，根据医生的要求随时打印病人的病情报告，定期更新病历，当病症出现异常时，系统会立即自动报警，并实时打印病人的病情报告，立及更新病历。

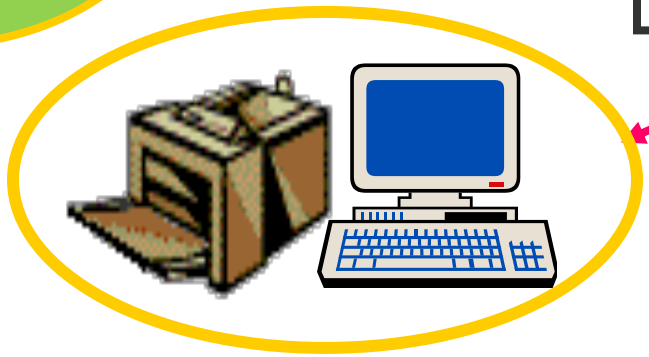
要求根据现场情景，对医院病房监护系统进行需求分析， 建立系统的用例模型

医院病房监护系统



监视病情





产生
病情报告



更新病历

经过初步的需求分析，得到系统功能要求：

① 监视病员的病症（血压、体温、脉搏等）

② 定时更新病历

③ 病员出现异常情况时报警。

④ 随机地产生某一病员的病情报告。

案例——简单的需求分析说明

- 系统名称：**医院病房监护系统**
- 根据分析，系统主要实现以下功能：

① 病症监视器可以将采集到的病症信号（组合），格式化后实时的传送到中央监护系统。

② 中央监护系统将病人的病症信号开解后与标准的病症信号库里的病症信号的正常值进行比较，当病症出现异常时系统自动报警。

③ 当病症信号异常时，系统自动更新病历并打印病情报告。

④ 值班护士可以查看病情报告并进行打印。

⑤ 医生可以查看病情报告，要求打印病情报告，也可以查看或要求打印病历。

⑥ 系统定期自动更新病历。

案例——系统的静态建模

建立系统的用例图

- 通过以下六个问题识别角色

① 谁使用系统的主要功能？

② 谁需要系统的支持以完成日常工作任务？

③ 谁负责维护，管理并保持系统正常运行？

④ 系统需要应付（或处理）哪些硬设备？

⑤ 系统需要和哪些外部系统交互？

⑥ 谁（或什么）对系统运行产生的结果（值）感兴趣？

案例——系统的静态建模（续）

通过回答这六个问题以后，再进一步分析可以识别出本系统的四个角色：值班护士、医生、病人、标准病症信号库。

角色描述模板：

角色：病人
角色职责：
提供病症信号

角色职责识别：

负责生成、实时提供各种病症信号。

角色：医生
角色职责：
对病人负责，负责处理病情的变化
角色职责识别：
(1)需要系统支持以完成其日常工作
(2)对系统运行结果感兴趣

角色：值班护士
角色职责：
负责监视病人的病情变化
角色职责识别：
(1)使用系统主要功能
(2)对系统运行结果感兴趣

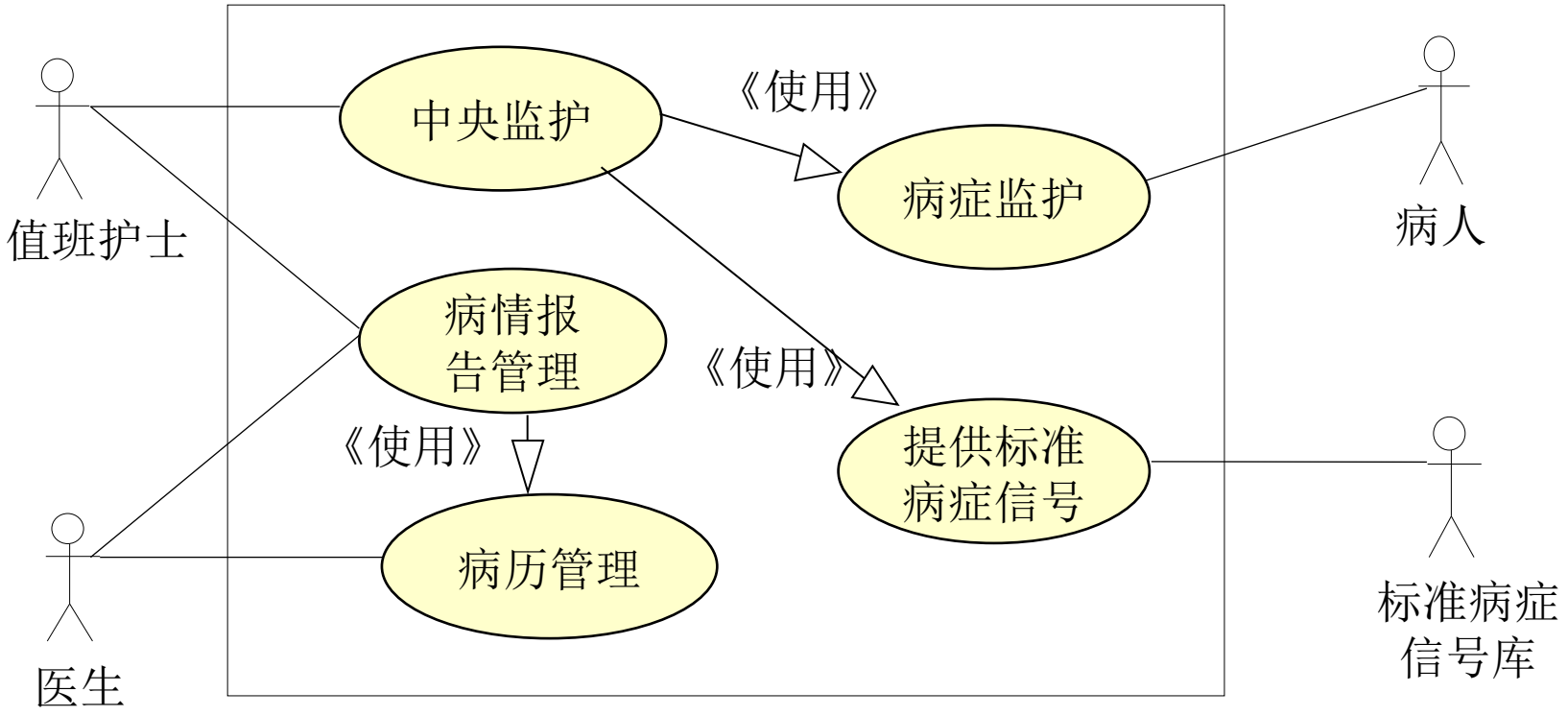
角色:标准病症信号库
角色职责：
负责向系统提供病症信号的正常值

角色职责识别：
(1)负责保持系统正常运行
(2)与系统交互

案例——系统的静态建模（续）

初步识别出系统的用例为：中央监护、病症监护、提供标准病症信号、病历管理、病情报告管理。

顶层用例图为：



案例——系统的静态建模（续）

将用例细化，可以得到分解的用例：

① 中央监护

a) 分解信号：将从病症监护器传送来的组合病症信号分解为系统可以处理的信号。
b) 比较信号：将病人的病症信号与标准信号比较。
c) 报警：如果病症信号发生异常（即高于峰值），发出报警信号。
d) 数据格式化：将处理后的数据格式化以便写入病历库。

② 病症监护

e) 信号采集：采集病人的病症信号。
f) 模数转化：将采集来的模拟信号转化为数字信号。
g) 信号数据组合：将采集到的脉搏，血压等信号数据组合为一组信号数据。
h) 采样频率改变：根据病人的情况改变监视器采样频率。

案例——系统的静态建模（续）

③ 提供标准病症信号

i) （此用例不分解）

④ 病历管理

j) 生成病历

k) 查看病历

l) 更新病历

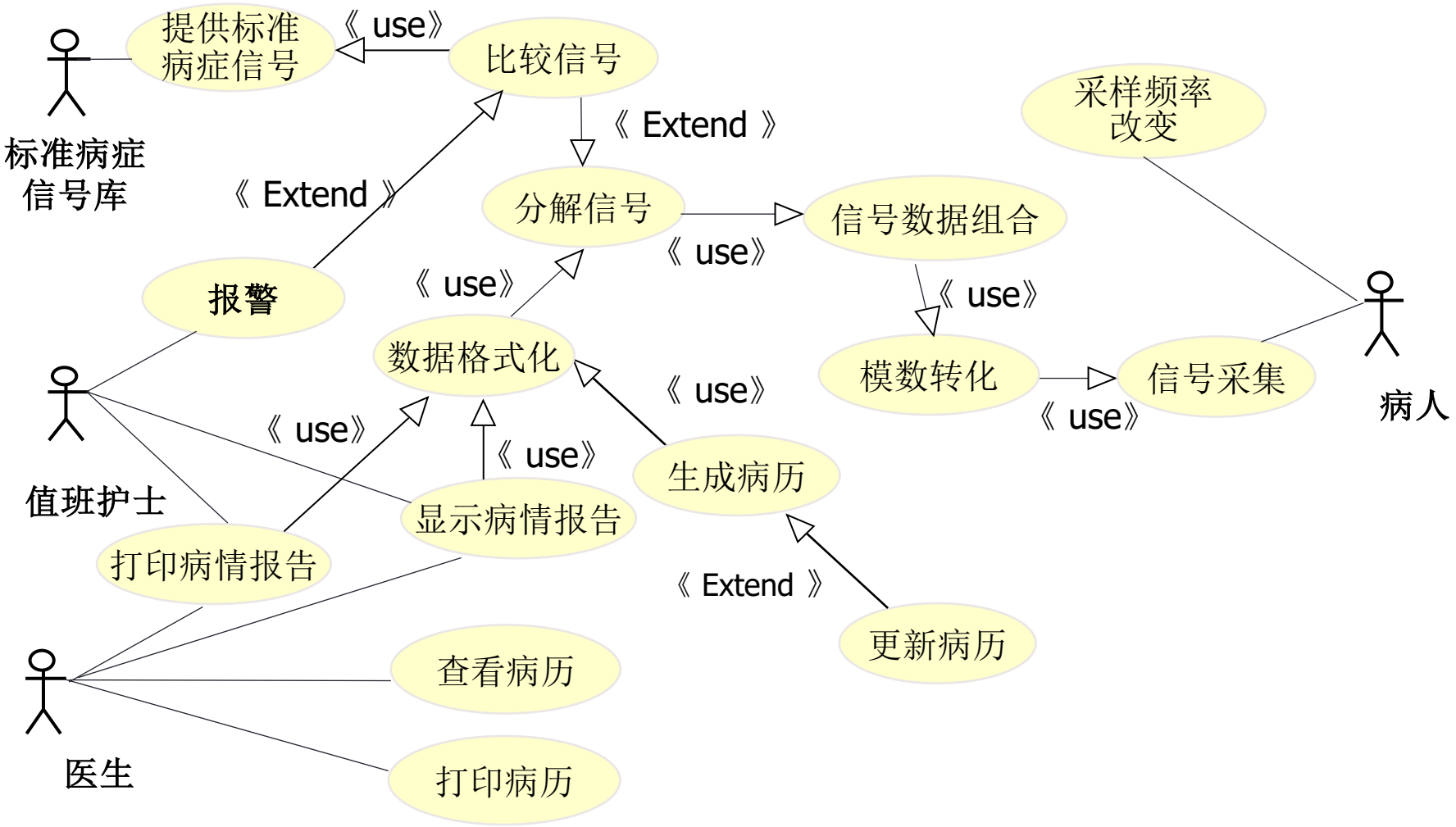
m) 打印病历

⑤ 病情报告

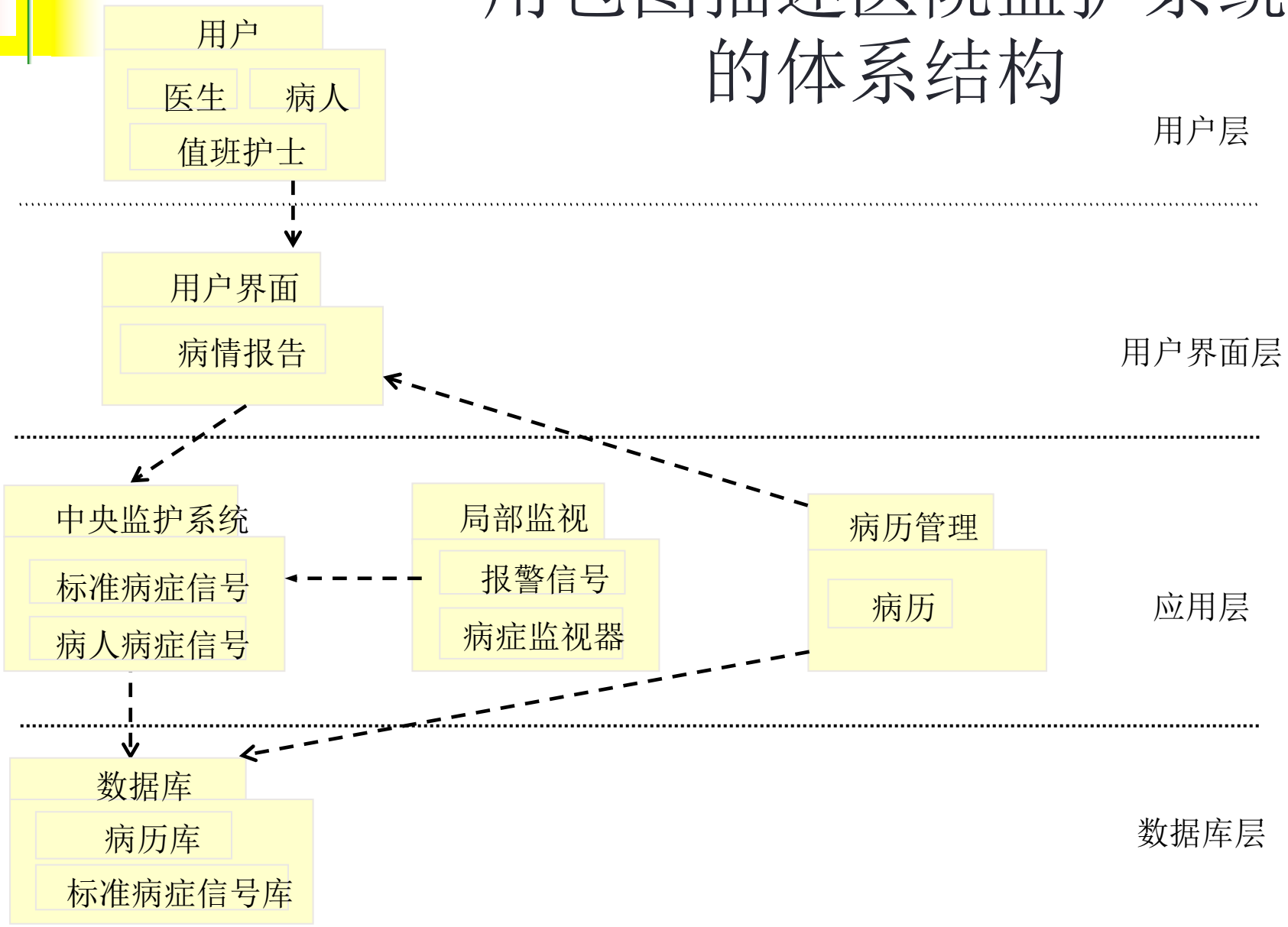
n) 显示病情报告：在显示器上显示病情

o) 打印病情报告：在打印机打印病情报告

案例——细化出的用例图

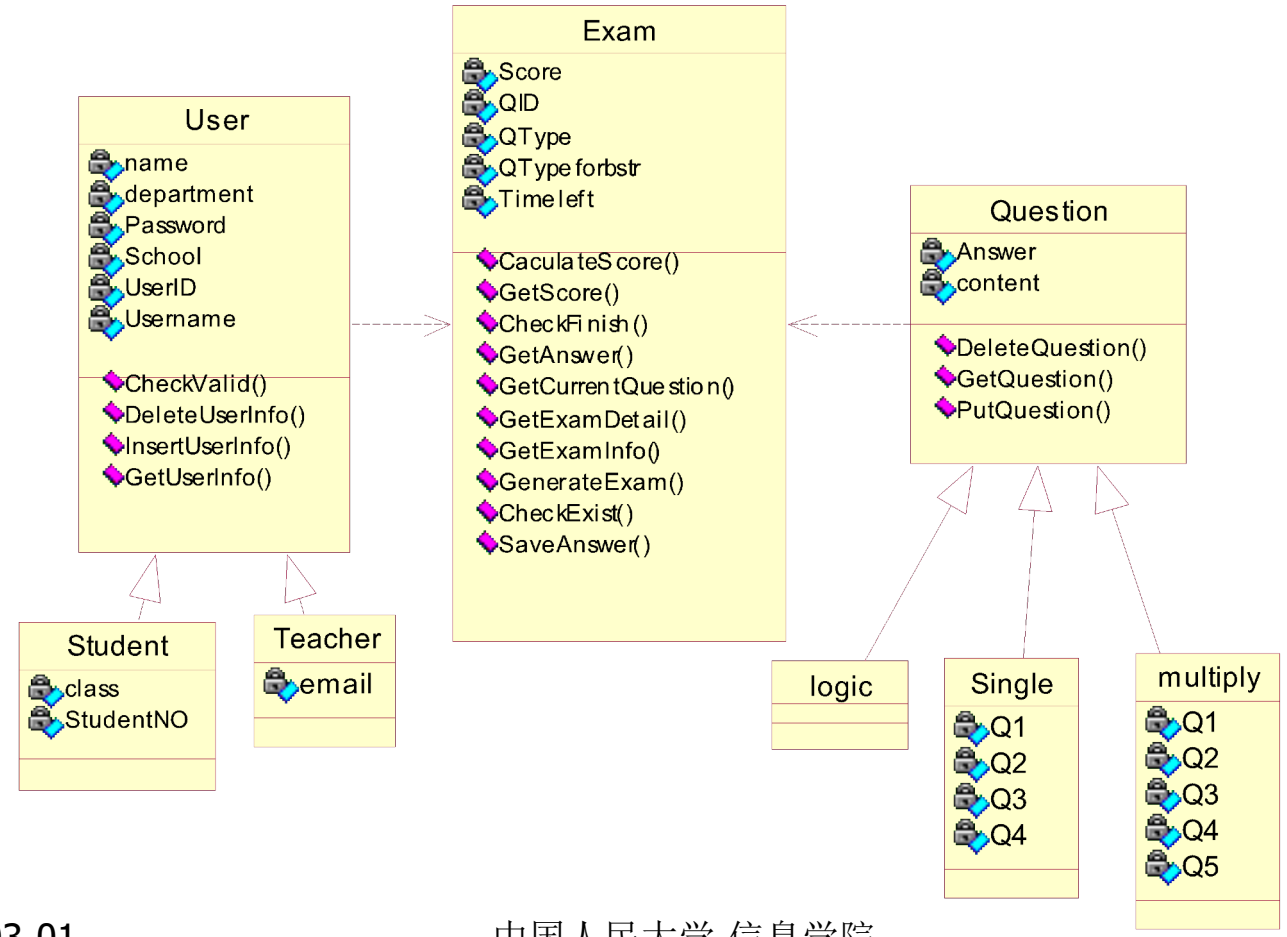

提供标准病症信号
《 use》
比较信号
标准病症信号库
《 Extend 》
《 Extend 》
分解信号
信号数据组合
采样频率改变
《 use》
《 use》
《 use》
报警
数据格式化
模数转化
信号采集
病人
《 use》
《 use》
《 use》
值班护士
显示病情报告
生成病历
打印病情报告
《 Extend 》
更新病历
查看病历
医生
打印病历

用包图描述医院监护系统的体系结构



类图（class diagram）

静态视图是UML的基础，静态视图表示为类图和对象图，主要是描述类和类之间的关系。



类图的描述

(1) 属性（attribute）

- 属性用来描述类的特征，表示需要处理的数据。
- 属性定义：

  visibility attribute-name : type = initial-value {property-string}

  可见性　　属性名：类型 = 缺省值 { 约束特性 }

- 其中：可见性(visibility)表示该属性对类外的元素是否可见。如果可见性未申明，表示其可见性不确定。分为：
  - public（+） 公有的，即模型中的任何类都可以访问该属性。
  - private（-） 私有的，表示不能被别的类访问。
  - protected（#） 受保护的，表示该属性只能被该类及其子类访问。

类图的描述（续）

(2) 操作

- 对数据的具体处理方法的描述则放在操作部分，操作说明了该类能做些什么工作。操作通常称为函数，它是类的一个组成部分，只能作用于该类的对象上。
- 操作定义：

visibility operating-name(parameter-list): return-type {property - string}

可见性 操作名（参数表）：返回类型{约束特性}

- 其中：可见性同上。
- 参数表：参数名：类型，…

Parameter-name : type = default - value

返回类型：操作返回的结果类型。

对象图（object diagram）

对象图是系统在某一时刻的快照（snapshot）。

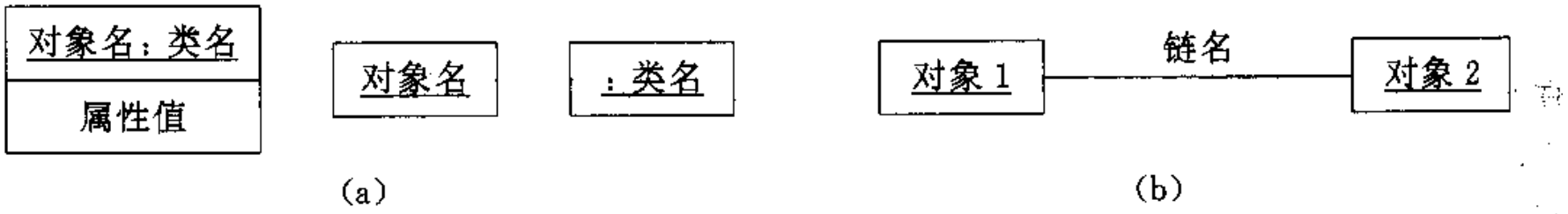
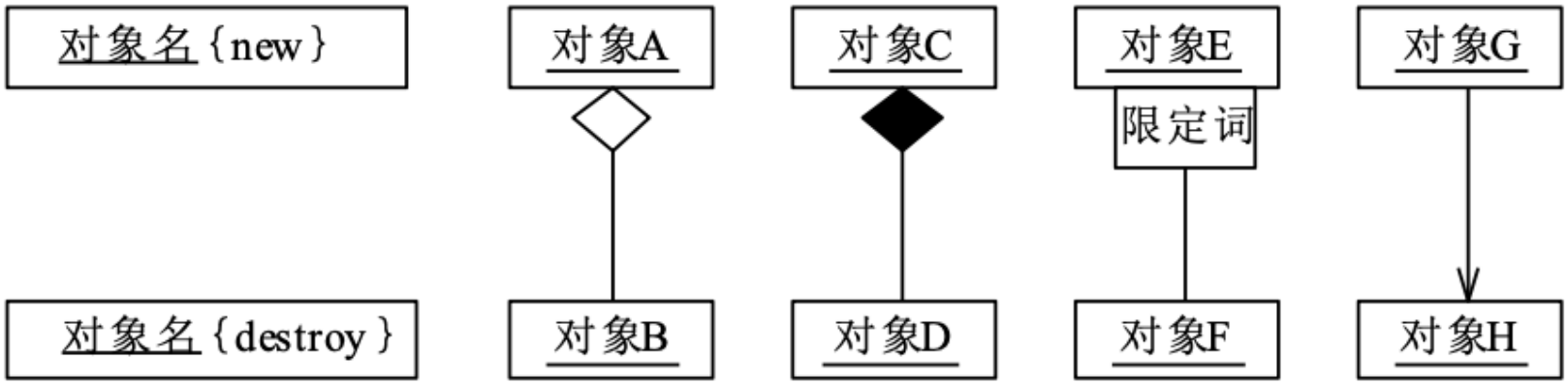


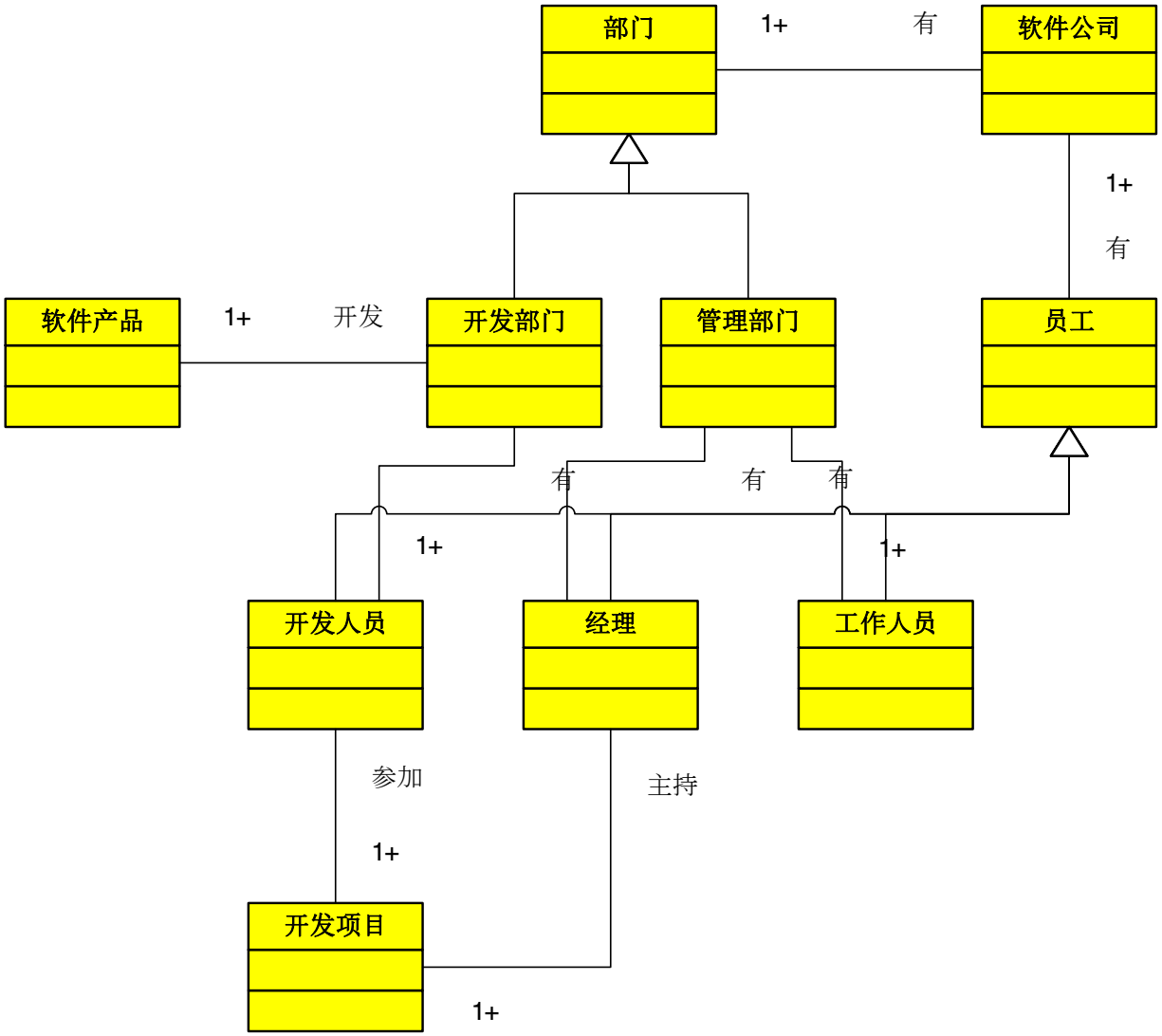
图 13.7　对象图的元素

(a) 对象的表示；(b) 链的表示



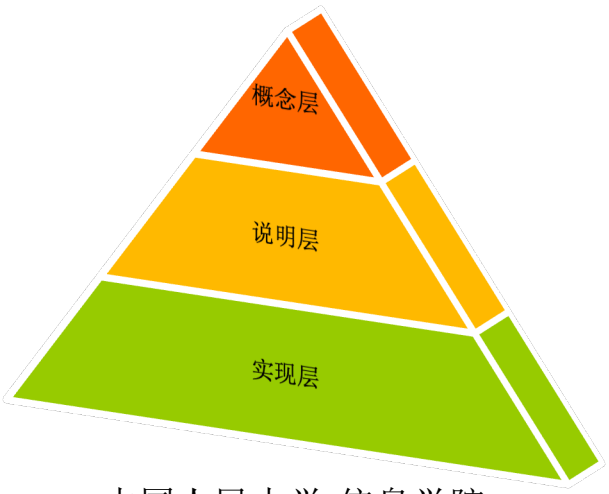
(a) 对象的创建和删除；　　(b) 各种关系的连接

例：软件公司的类图


部门
1+
有
软件公司
1+
有
软件产品
1+
开发
开发部门
管理部门
员工
有
有
有
1+
1+
开发人员
经理
工作人员
参加
1+
主持
开发项目
1+

类图的抽象层次和细化(Refinement)关系

- 在软件开发的不同阶段都使用类图，但这些类图表示了不同层次的抽象：
  - 在需求分析阶段，类图是研究领域的概念；
  - 在设计阶段，类图描述类与类之间的接口；
  - 在实现阶段，类图描述软件系统中类的实现。
- 按照Steve Cook和John Daniels的观点，类图分为三个层次：
  - 概念层
  - 说明层
  - 实现层



实例：识别系统的类

通过名词识别法和系统实体识别法等方法可以识别出系统的12个类，以下用类图的方法分别表示出类的名称，属性/操作。

值班护士	医生	病人	病症监视	中央监护系统	报警信号
标准病症信号库	病历	病历库	病人病症信号	病情报告	标准病症信号

值班护士	医生	病人	病症监视
用户名 密码	用户名 密码	姓名 性别 年龄 病症	采集频率 病症信号
查看病情报告（） 打印病情报告（）	查看病情报告（） 要求打印病情报告（） 查看病历（） 要求打印病历（）	提供病症信号（）	格式化信号数据（） 采集信号（） 信号组合（）

（续）

中央监护系统
输入 输出
分解信号（） 比较信号（） 报警（） 数据格式化（）

报警信号
声音 灯光 文字
报警（） 数模转化（）

标准病症信号库
类型 大小 容量
提供标准信号（）

病历库
类型 大小 容量
生成病历（） 更新病历（） 查看病历（） 打印病历（）

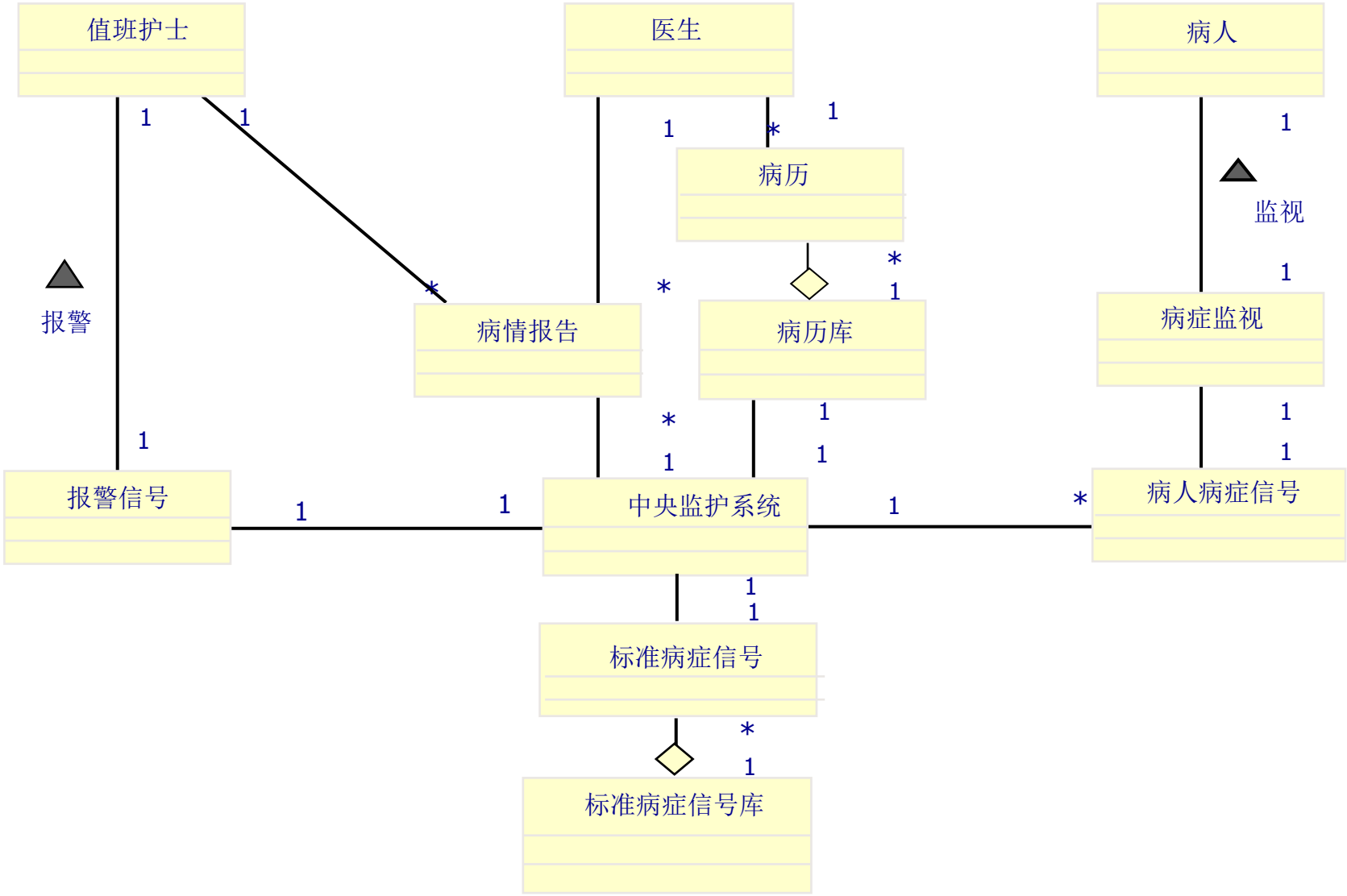
病人病症信号
脉搏 血压 体温
生成病症信号（）

病情报告
标题 格式
生成病情报告（） 查看病情报告（） 打印病情报告（）

病历
格式 病人基本情况 打印时间
生成病历（） 查看病历（） 打印病历（）

标准病症信号
脉搏 血压 体温
生成标准信号（）

在类图中标明类之间的关系：



7.4 UML对动态建模的支持

- 动态建模
- 状态图（state diagram）
- 活动图（activity diagram）
- 顺序图（sequence diagram）
- 协作图（collaboration diagram）



动态建模

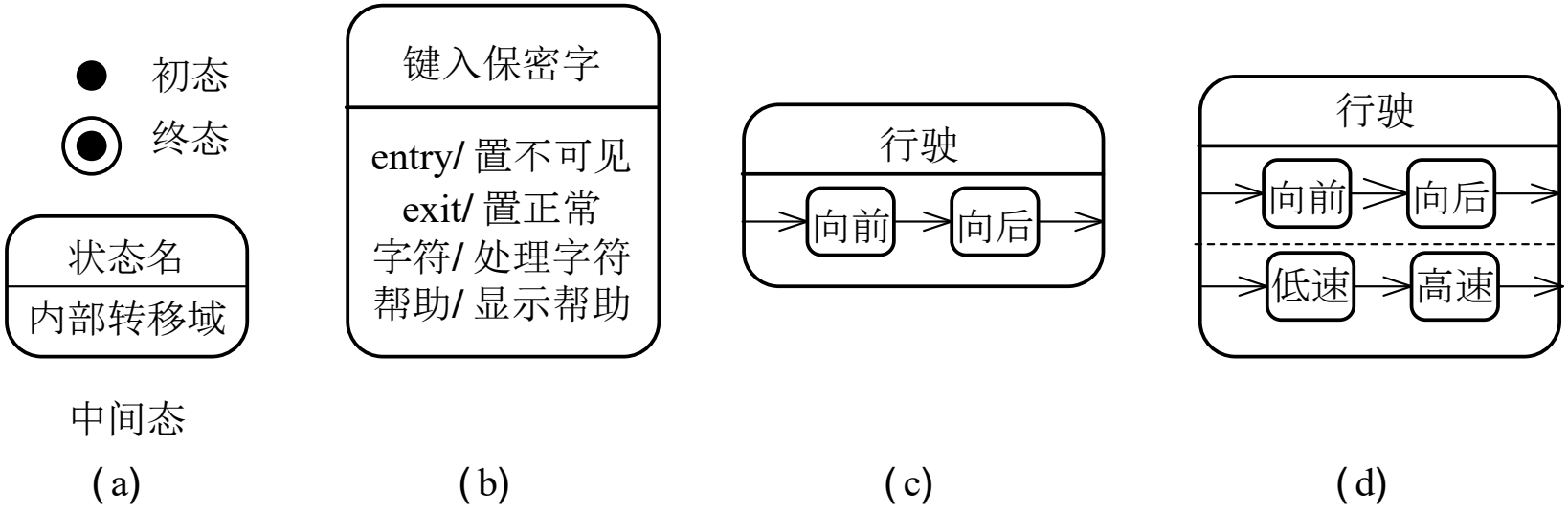
- 系统中的对象在执行期间的不同时间点如何通信以及通信的结果如何，就是系统的动态行为，也就是说，对象通过通信相互协作的方式以及系统中的对象在系统生命期中改变状态的方式，是系统的动态行为。
  - 状态图（state diagram）
  - 活动图（activity diagram）
  - 顺序图（sequence diagram）
  - 协作图（collaboration diagram）



状态图（state diagram）

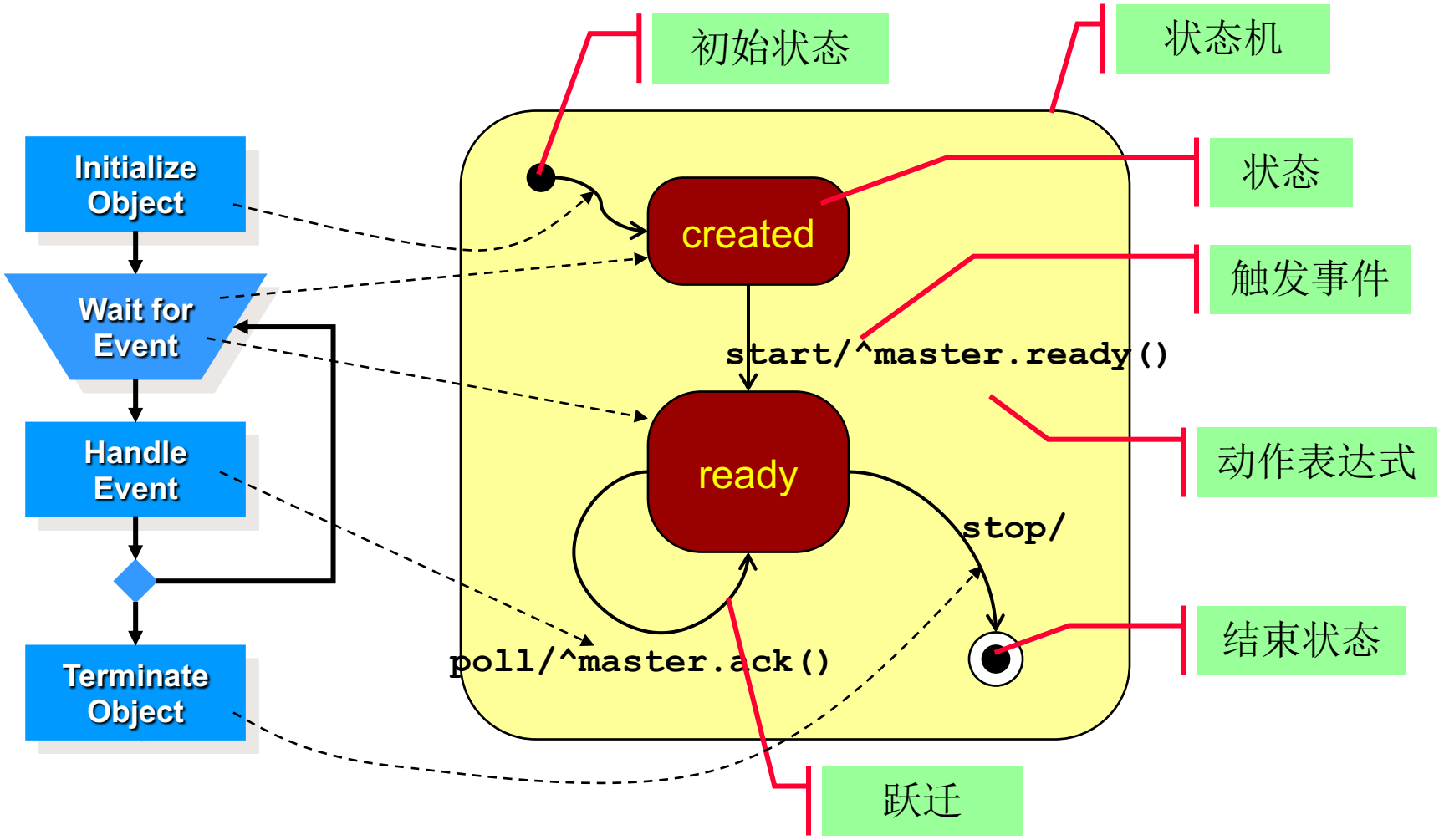
- 状态图是对单个类的对象的生命周期进行建模，描述了对象时间上的动态行为，每个对象被认为是事件驱动的孤立实体。
- 状态图是由状态（state）和跃迁（transition）组成的图，通常状态机附属于类，描述类实例对接受事件的响应。
- 事件（event）表达对象间的调用、显式信号、值的改变或时间的推移。
  - 调用事件、变更事件、信号事件、时间事件
- 状态（state）描述对象生命周期的一段时间，可以是等待其它事件时所处的时间，或是执行某一活动时所处的时间，状态分为简单状态和复合状态。
- 跃迁定义对象对某一事件的反应，通常，跃迁具有触发事件、跃迁条件、动作和目标状态。
- 跃迁的种类有外部跃迁和内部跃迁。外部跃迁是最普通的跃迁，会导致状态改变；内部跃迁不发生状态改变。
- 跃迁有两个隐式动作：进入动作（entry）和退出动作（exit）。无论何时进入和退出时都要执行，这方便进入时进行初始化工作，退出时进行资源的释放工作。

状态的表示

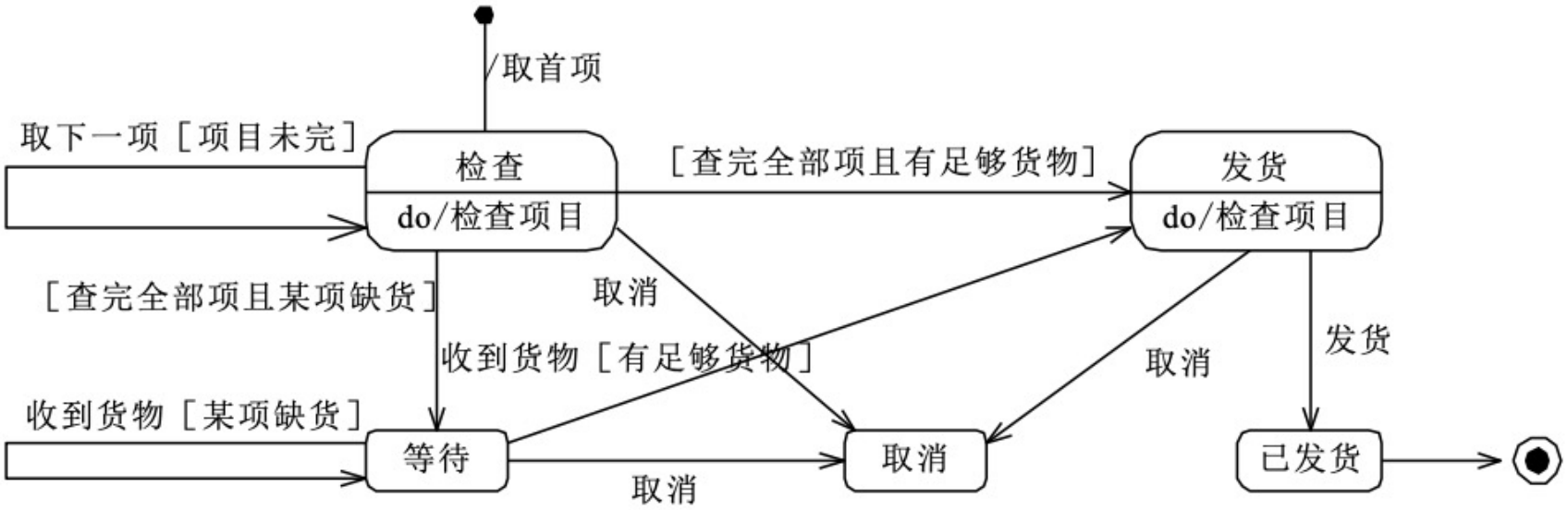


(a) 状态类型； (b) 内部转移域； (c) 或关系子状态； (d) 与关系子状态

状态图示例


初始状态
状态机
状态
触发事件
动作表达式
结束状态
跃迁
Initialize Object
Wait for Event
Handle Event
Terminate Object
created
ready
start/^master.ready()
stop/
poll/^master.ack()

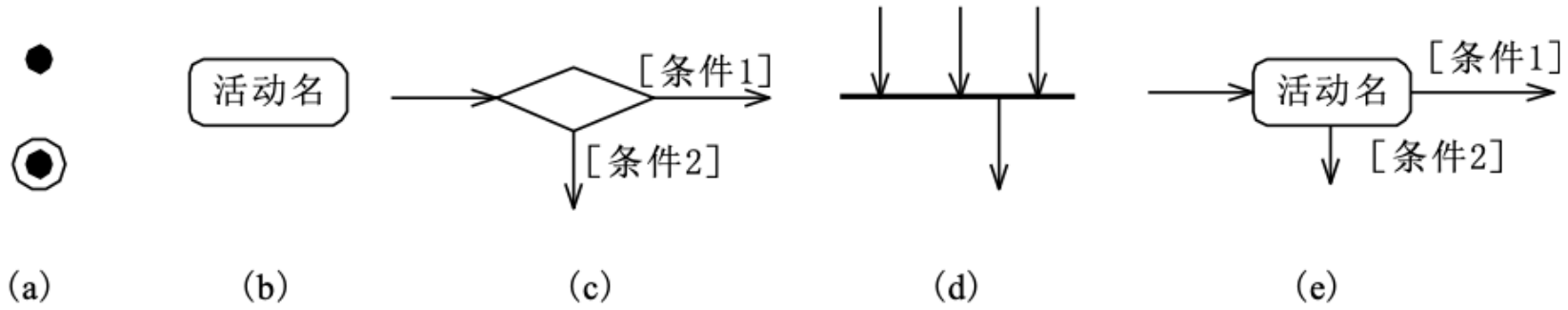
例：订单类的状态图


/取首项
取下一项［项目未完］
检查
do/检查项目
［查完全部项目且有足够货物］
发货
do/检查项目
［查完全部项目且某项缺货］
取消
收到货物［有足够货物］
取消
发货
收到货物［某项缺货］
等待
取消
取消
已发货

活动图（activity diagram）

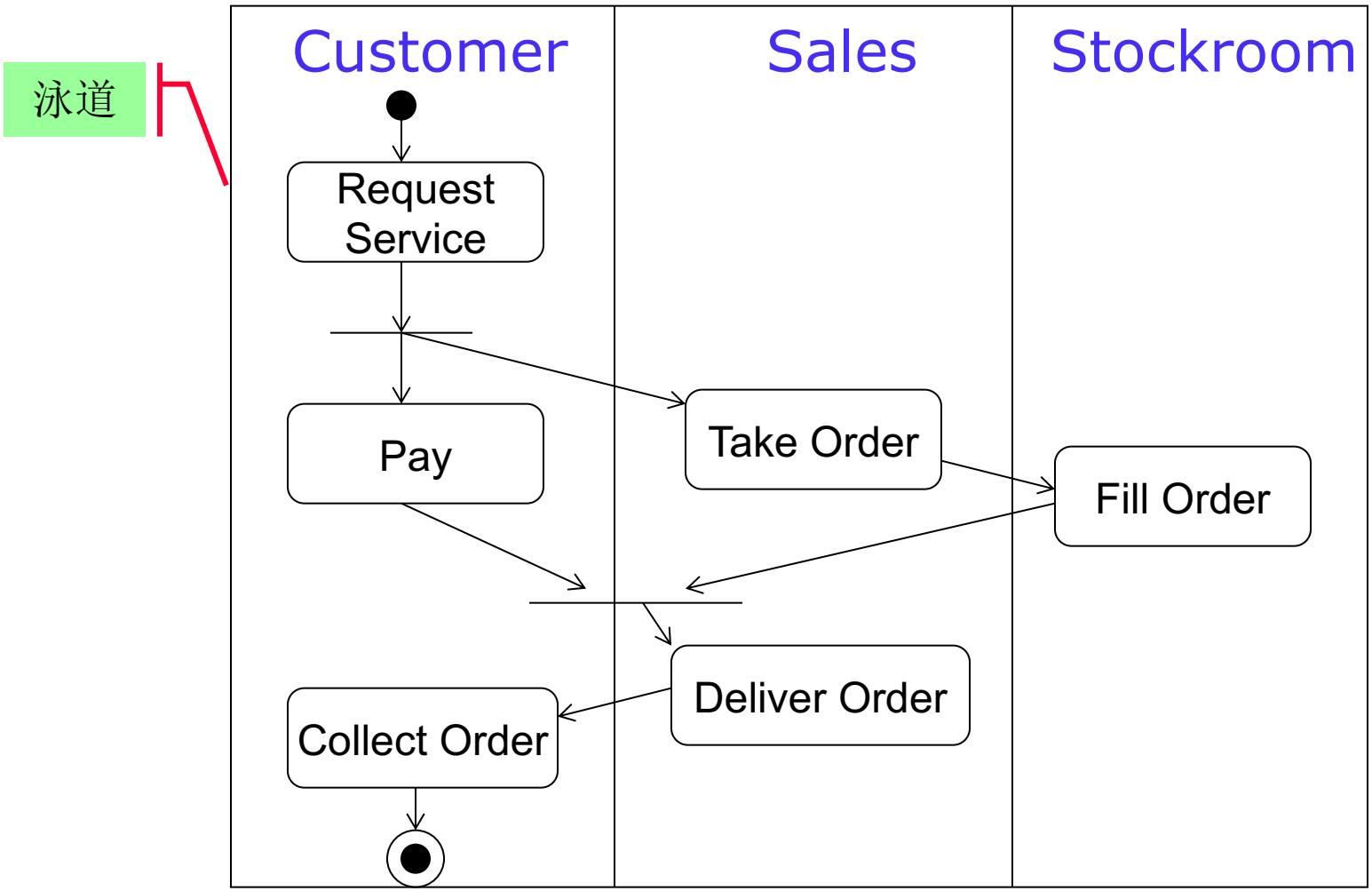
- 活动图是UML中的一种行为建模工具，主要用于描述系统或业务中的一系列活动或操作，是用状态机对工作流进行建模的特殊形式。
- 活动图描述用例中的行为，展现从一个活动到另一活动的控制流，它和流程图很类似，不过它支持并发控制。
- 活动图一般不描述所有的运算细节，它显示活动的流，但不显示执行活动的对象。
- 活动图处于系统的外部和内部视图之间，所以它可以作为设计的起点，为了完成设计，每个活动必须扩展成一个和多个操作，每个操作被指派给特定的对象来实现。
- 将商业组织控制的活动划分在一起，这类划分可以通过分隔的区域来表达，由于它们的外观，每个区域称为泳道（swim lane）。

活动图的元素



(a) 起点、终点； (b) 活动； (c) 判断； (d) 同步线； (e) 转移

活动图示例


泳道
Customer
Sales
Stockroom
Request Service
Pay
Take Order
Fill Order
Deliver Order
Collect Order

例：用活动图描述“找饮料”

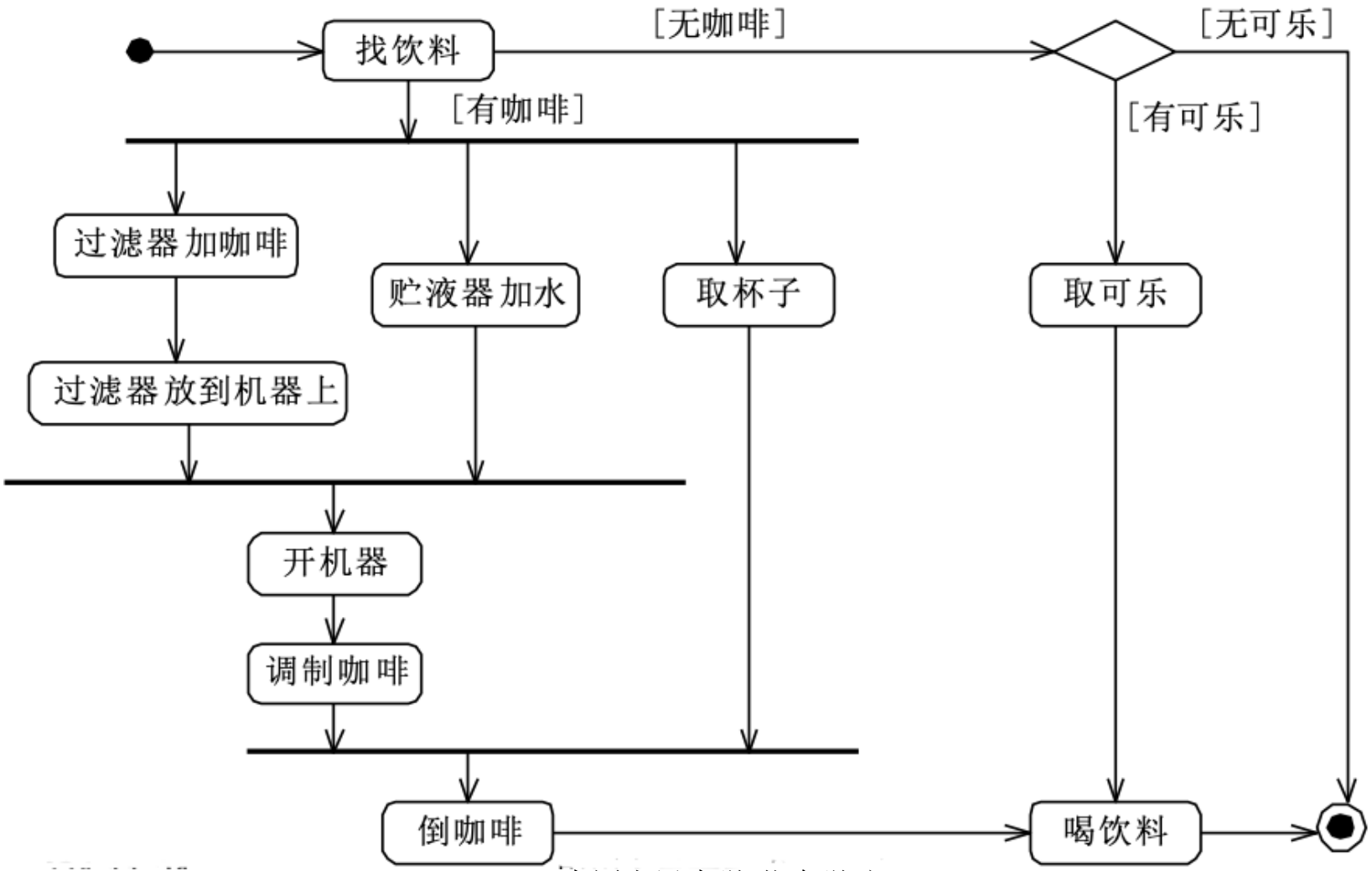
- 在“人”的类中关于“找饮料喝”方法的活动图。
- 从开始顺序转移到“找饮料”活动，该活动有两个转移。
  - ［有咖啡］时， 转移到一个同步线，同时开始 3 个活动：
    - 第一个是“过滤器加咖啡”活动，然后顺序转移到“过滤器放到机器上”活动;
    - 第二个是“贮液器加水”活动;
    - 第三个是“取杯子”活动。
  - ［无咖啡］时，则转移到有无可乐的判断。
  - ［有可乐］条件时，则执行：
    - “取可乐”活动
    - “喝饮料”活动
  - ［无可乐］条件时，则转移到结束。







找饮料活动图示例（续）


找饮料
[无咖啡]
[无可乐]
[有咖啡]
[有可乐]
过滤器加咖啡
贮液器加水
取杯子
取可乐
过滤器放到机器上
开机器
调制咖啡
倒咖啡
喝饮料

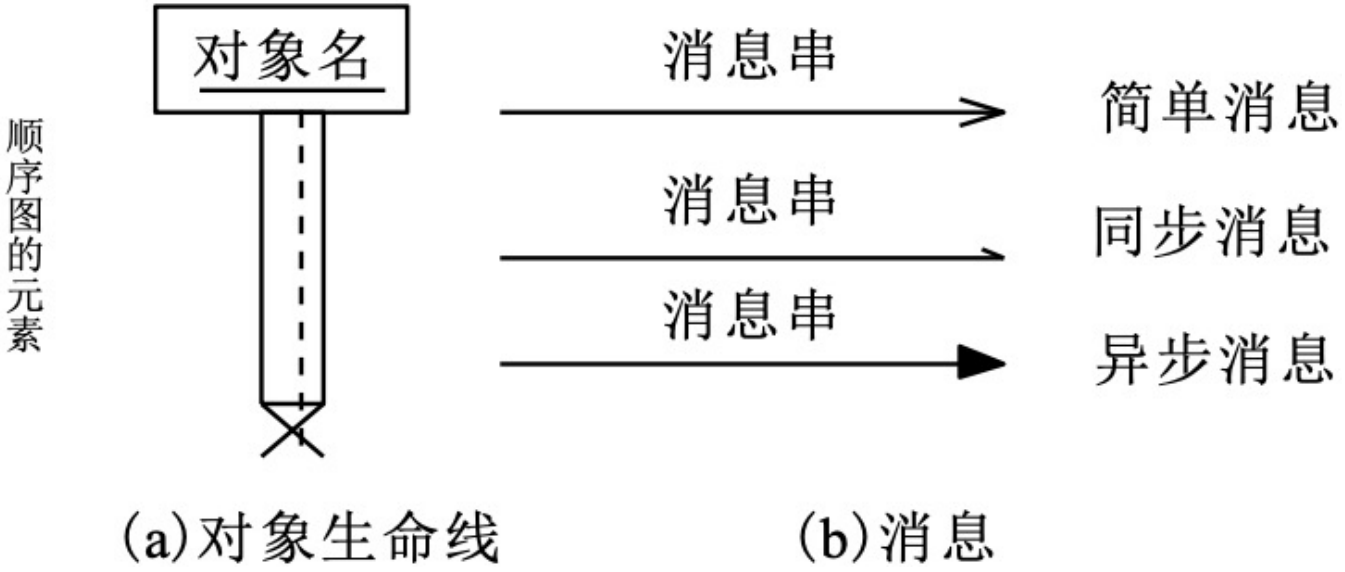
交互视图（interactive view）

- 对象行为是通过交互来实现的，交互是对象间为完成某一目标而进行的一系列消息交换。
- 消息（message）是对象间的单向通信，从发送者到接受者的携带信息的控制流。消息可能带有值参。
- 消息序列可用两种图表示：
  - 顺序图：重点在消息的时间顺序。
  - 协作图：重点在交换消息与对象间的关系。



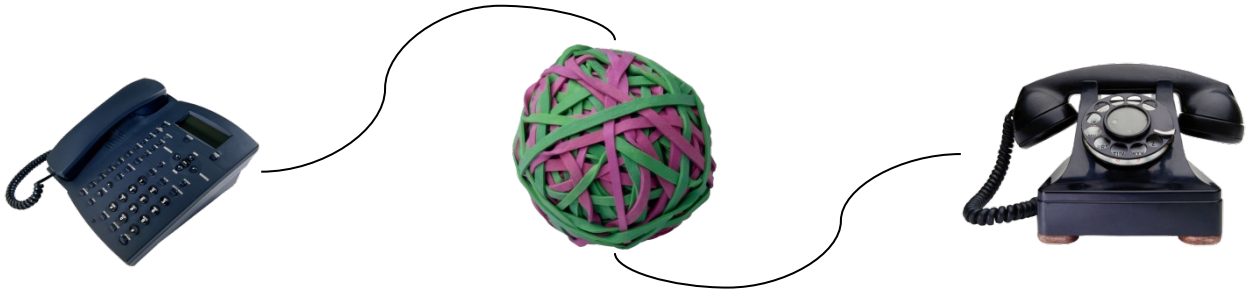
顺序图（sequence diagram）

- 顺序图用二维表来表示交互，纵向是时间轴，横向是参与的角色以及它们交换的消息。
- 角色的生命周期表现为生命线，一条垂直的线，在激活的时间段里是双线，在状态保持的时间里是虚线。
- 消息表示为从一条生命线出发到另一条生命线的有向线，从上而下，表示消息的时间顺序。

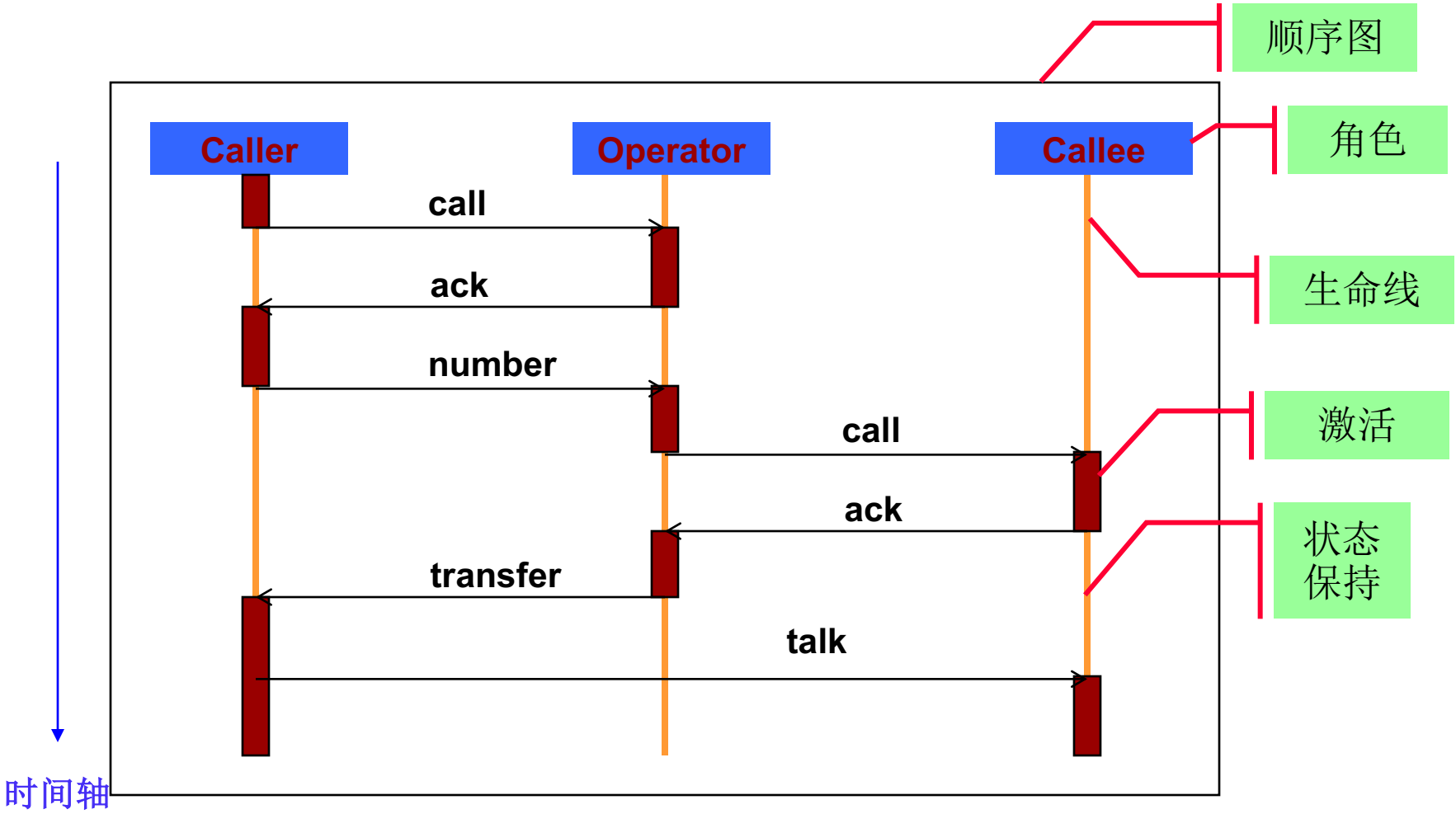


例：顺序图的示例

- 在一次电话通话中，涉及到的对象有：
  - Caller 呼叫者
  - operator 接线员
  - callee 被叫者
- 这些对象之间的消息发送和接受可以用顺序图描述。



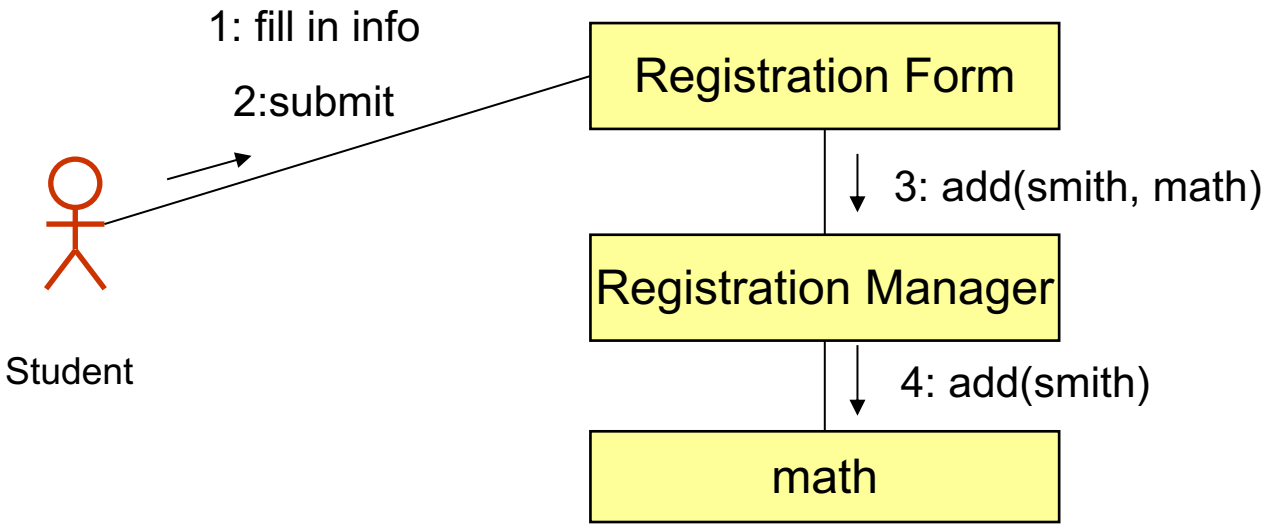
（续）顺序图


顺序图
角色
生命线
激活
状态
保持
Caller
Operator
Callee
call
ack
number
call
ack
transfer
talk
时间轴

协作图（collaboration diagram）

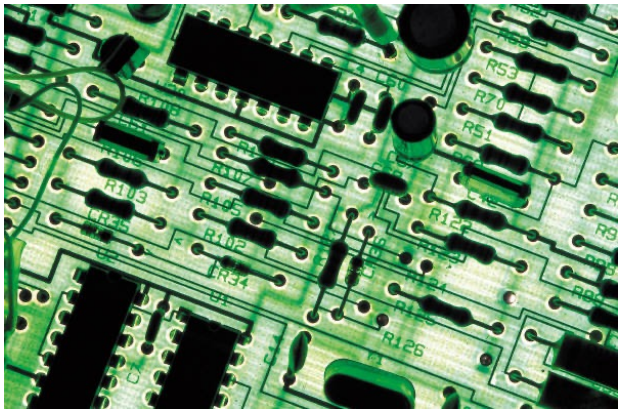
- 协作图显示的信息与顺序图相同，但显示方式不同，作用也不同。
- 协作图中对象的图示与顺序图中一样。
- 协作图着重于协作对象之间的交互和链接（消息），它可以表现系统中的操作、用例执行和或一个简单的交易场景。

协作图示例

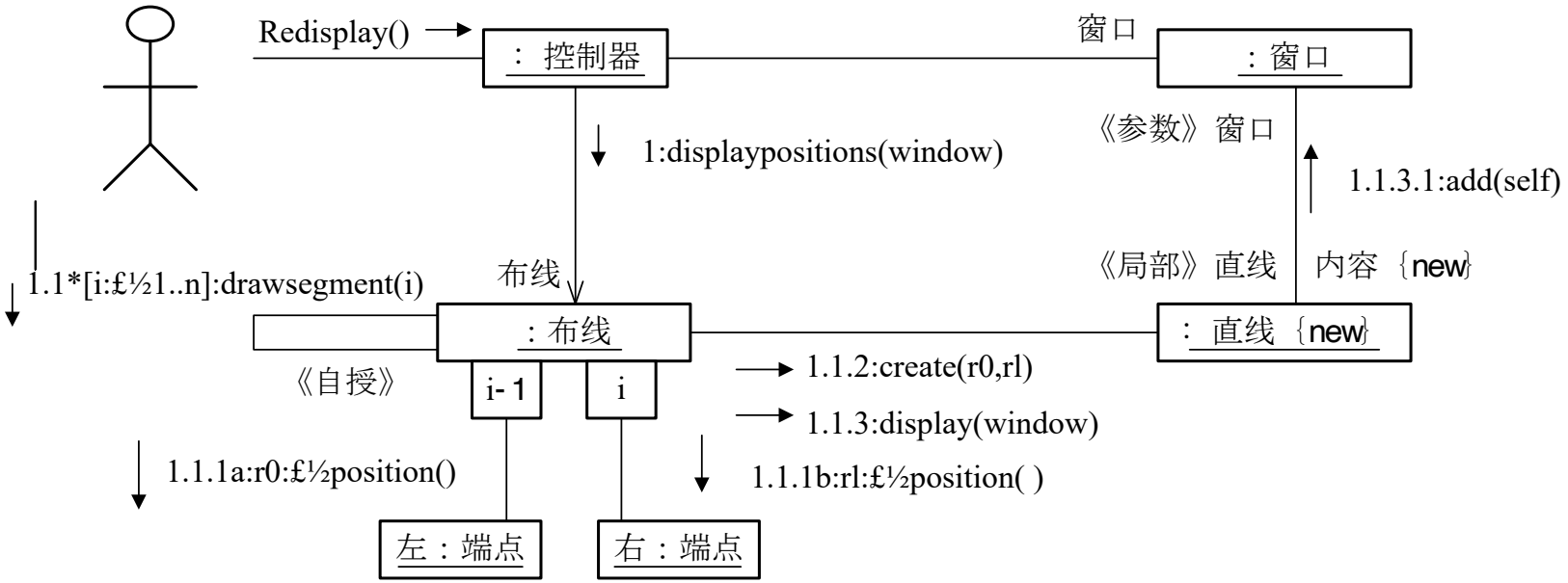


例：协作图的示例

- 在电路设计中，有“控制器”、“布线”、“端点”、“直线”及“窗口”等对象和类，在控制器控制下进行布线。
- 每布一条线， 先定位两个端点，即左端点r0、右端点r1，再以这两个端点为参数创建直线，并在窗口上显示出来，然后重复定位、 创建直线、显示，从而完成电路布线设计。



（续）协作图的示例


Redisplay()
：控制器
窗口
：窗口
1:displaypositions(window)
《参数》窗口
1.1.3.1:add(self)
1.1*[i:£½1..n]:drawsegment(i)
布线
《局部》直线
内容 {new}
：布线
：直线 {new}
《自授》
i-1
i
1.1.2:create(r0,rl)
1.1.3:display(window)
1.1.1a:r0:£½position()
1.1.1b:rl:£½position( )
左：端点
右：端点

7.4 物理结构表示

- 实现视图
- 构件图（component diagram）
- 部署图（deployment diagram）



系统架构（ architecture ）

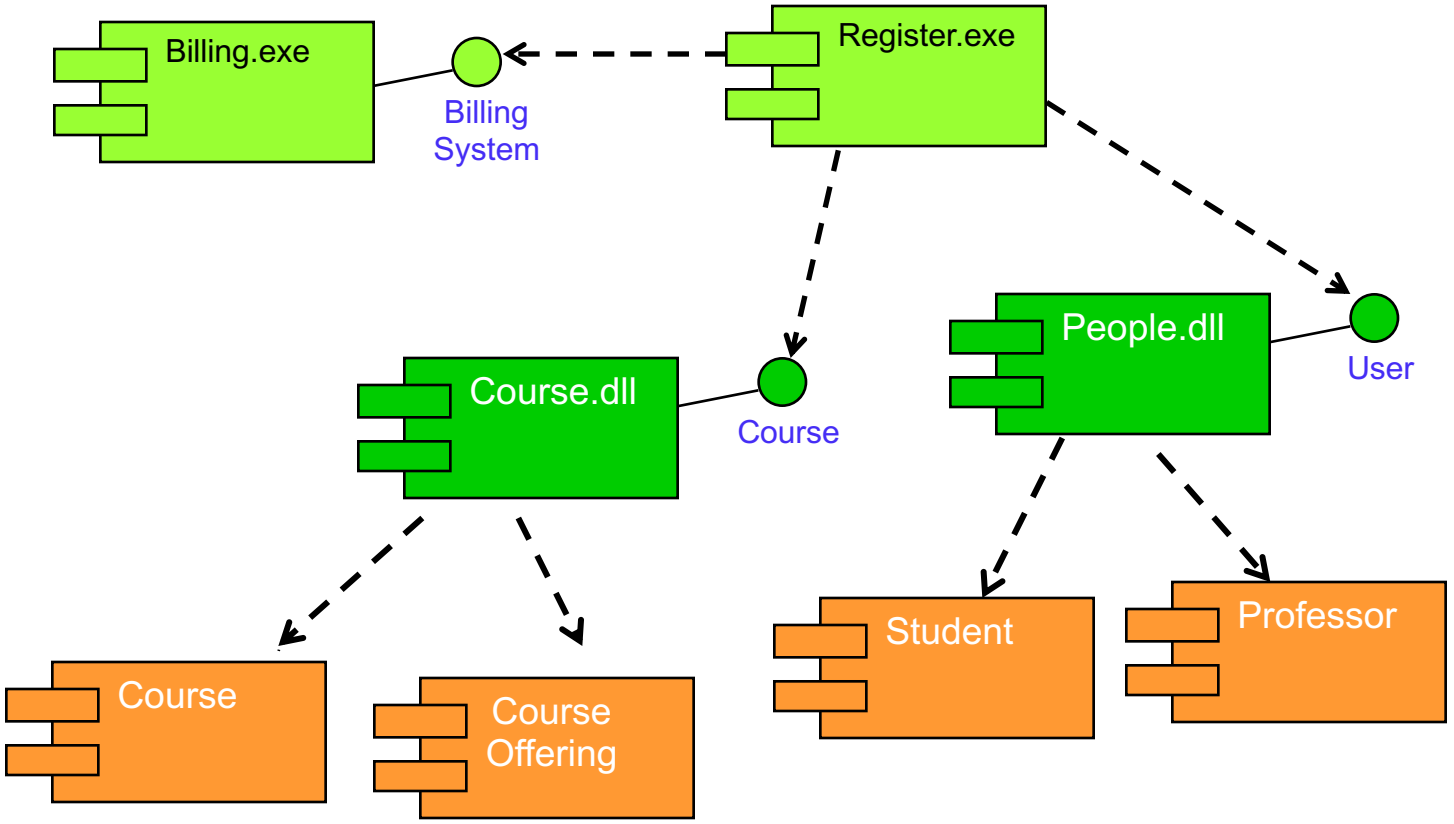
- 系统架构(或称为体系结构)是对构成系统的各个部分的框架性描述。
- 在UML中，架构的定义是：架构是系统的组织结构。可以递归地把架构分解成：
  - 通过接口交互的部分
  - 连接各个部分的关系
  - 组装各个部分的约束。
- 系统架构分为两大类：
  - 逻辑架构（logic architecture）
  - 物理架构（physical architecture）



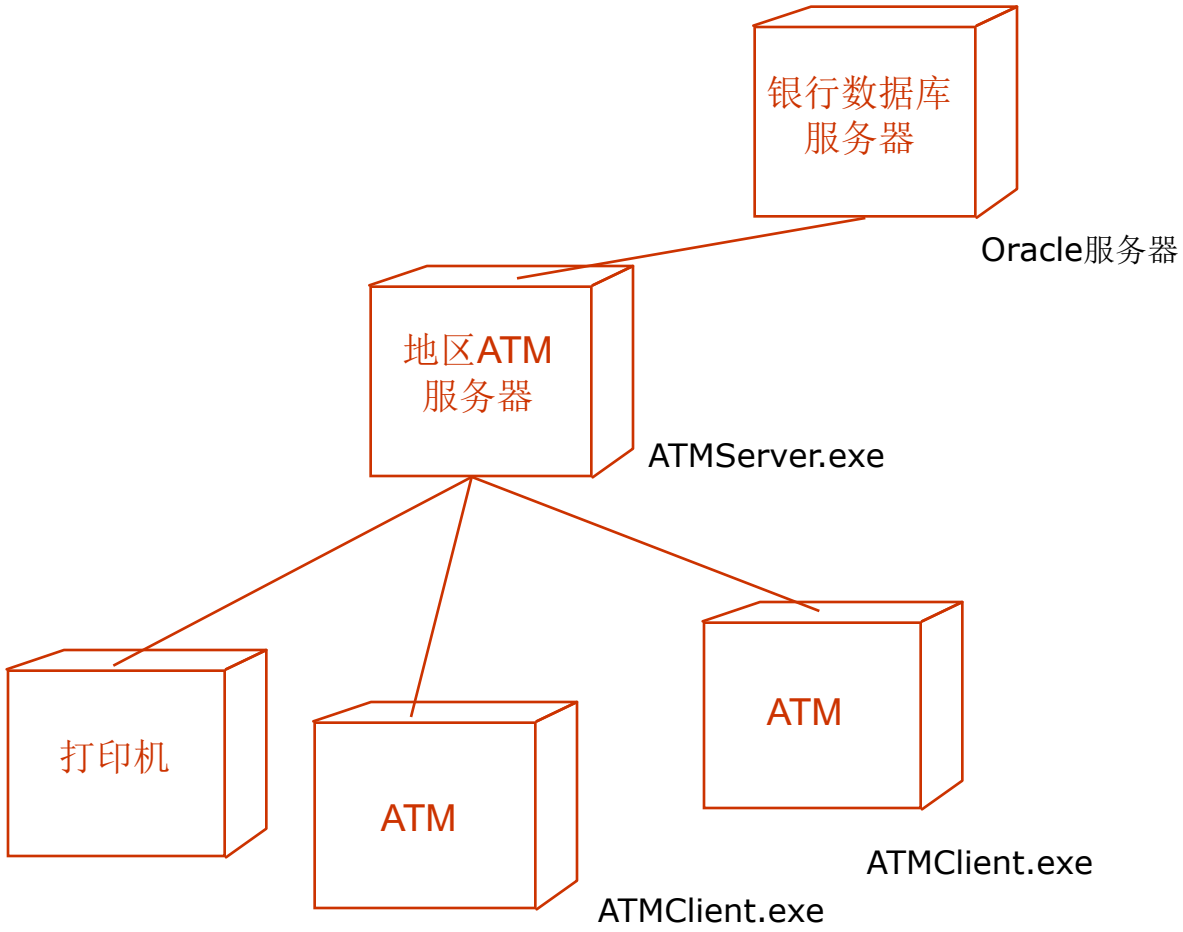
实现视图（implementation view）

- 实现视图描述可重用的系统组件以及组件之间的依赖关系。
- 实现视图一般的组成是：
  - 构件图（component diagram）
    - 描述构件及其之间的相互依赖关系。
    - 它通常是开发环境中的实现文件，包括：源代码构件、二进制构件和可执行构件。
  - 部署图（deployment diagram）
    - 描述处理器、设备和软件构件运行时的体系结构。
    - 描述系统资源在运行时的物理分布，系统资源成为结点。

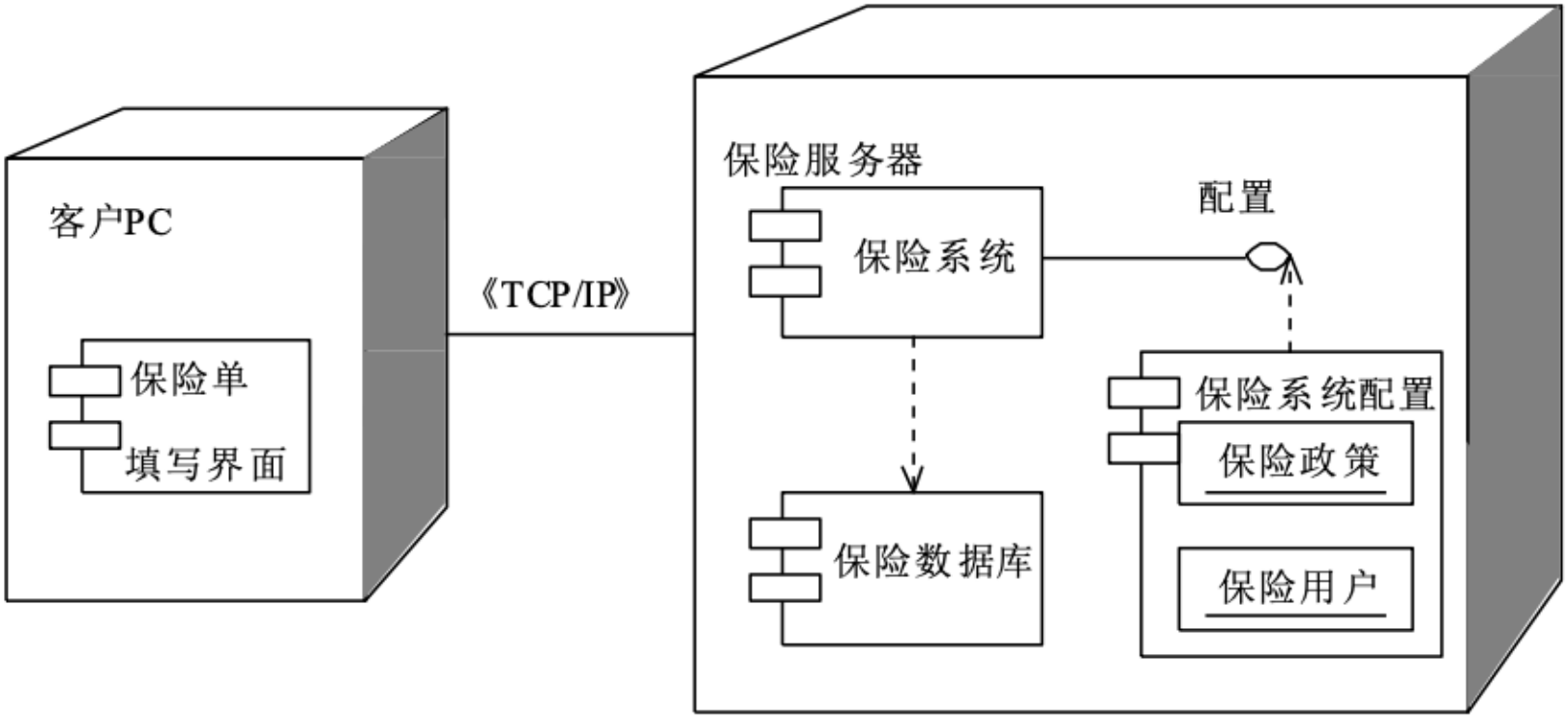
构件图（component diagram）



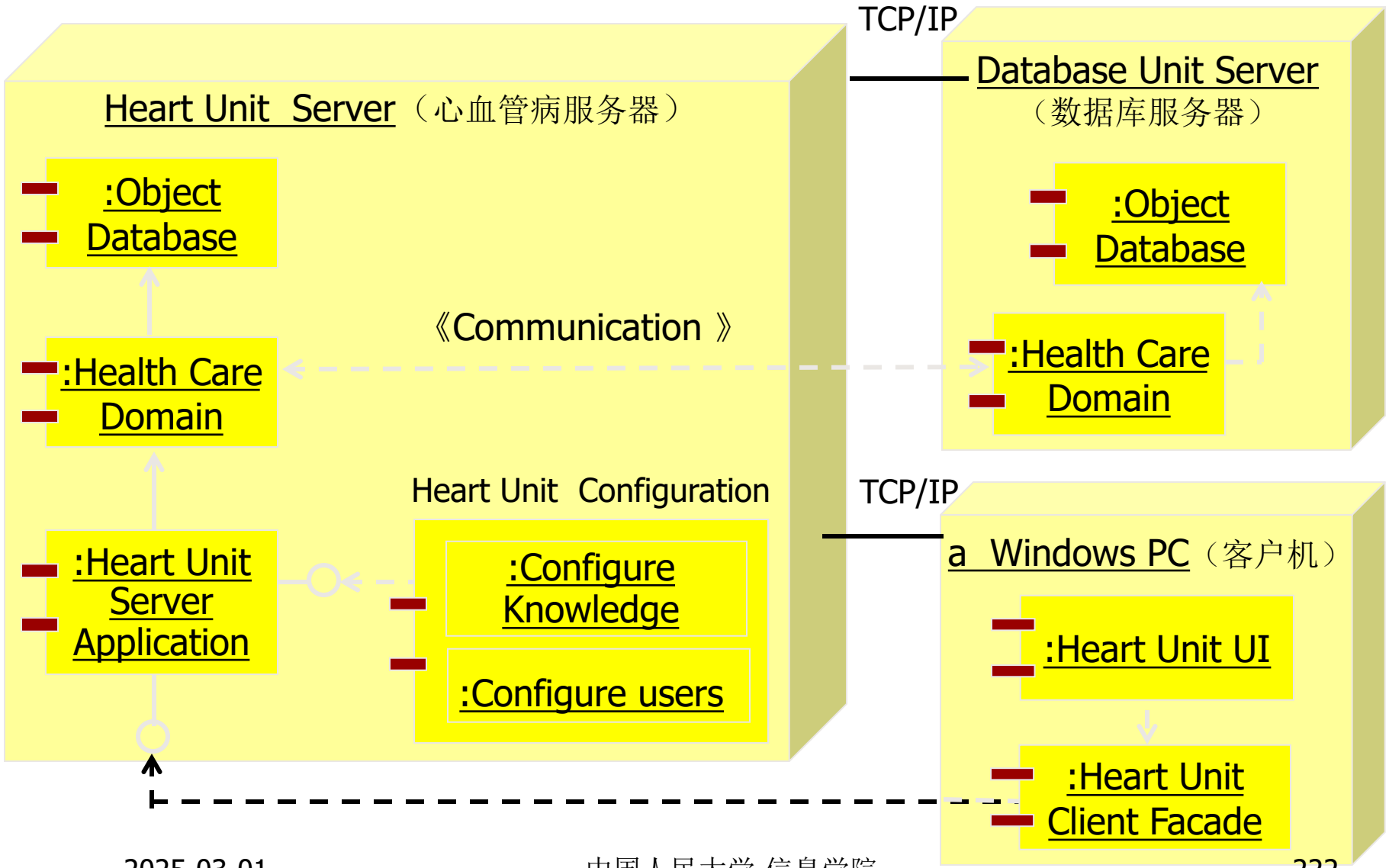
部署图（deployment diagram）



例：保险信息系统的配置图



例：医院诊疗系统的配置图



课堂讨论：

(1) 识别UML的图形符号，叙述每种UML图的作用。

(2) 对下列领域之一进行概略的领域分析，并用UML的图形符号来表示分析的成果：

a) 大学生成绩记录系统

b) 大学生选课系统

c) 银行的客户服务

d) 手机短信服务

课堂练习题

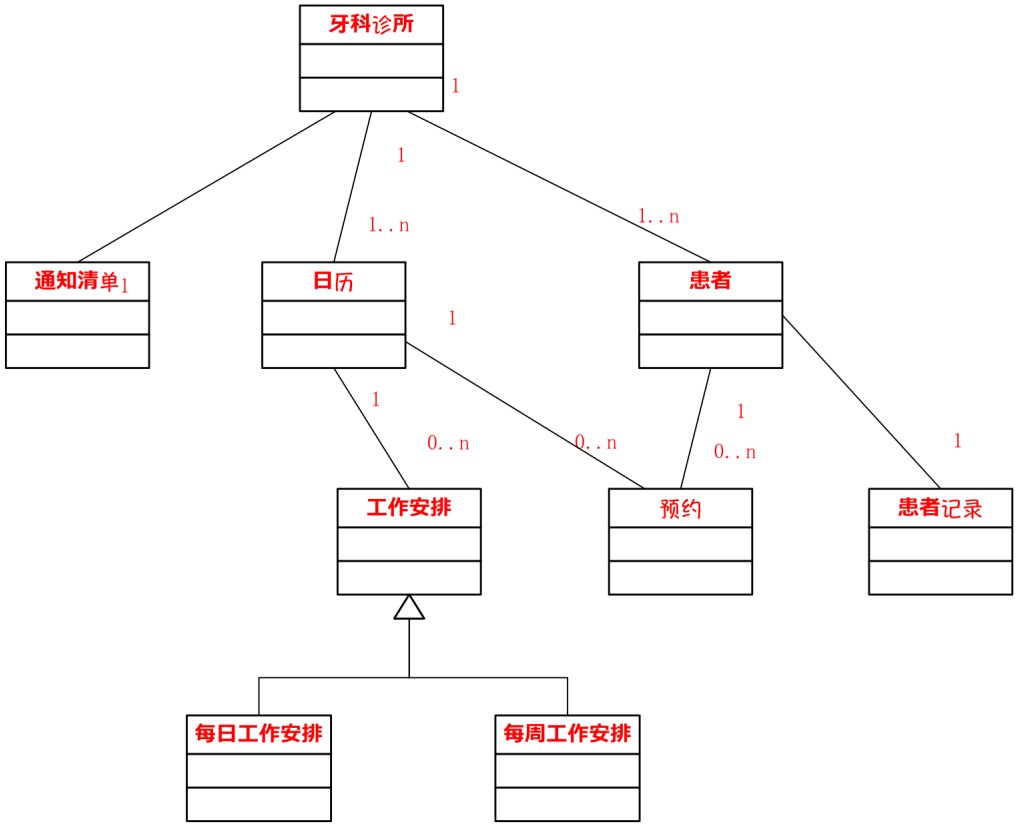
确定牙科诊所问题陈述的对象，用UML描述问题静态模型：

汤姆在小镇上开了一个牙科诊所。他有一个牙科助手、一个牙科保健员和一个接待员。汤姆需要管理预约系统。

在患者打电话预约时，接待员将查阅日历，并安排患者尽早诊治。如果患者对约定时间感到满意，接待员将输入约定时间和患者名。系统将核实患者名并提供患者的记录数据，这些记录数据包括患者ID号等。在每次检查或洗牙后，保健员或助手将标记相应预约已完成，增加说明，如果必要，会安排患者下一次再来。

系统要响应按患者名和按日期进行的查询。患者记录数据与预约信息一起显示。接待员可取消预约，可打印出前两天预约尚未接诊的通知清单。系统中有来自患者记录的电话号码。接待员还可打印出有关所有患者每天和每周的工作安排。

静态模型


牙科诊所
1
1
1..n
1..n
通知清单1
日历
患者
1
1
0..n
0..n
1
0..n
1
工作安排
预约
患者记录
每日工作安排
每周工作安排

习题

(1) 什么是OOA建模语言，它应该包括哪些方面？

(2) 什么是Use-Cases模型，主要作用是什么？

(3) UML中有哪些动态建模的工具，各表达什么内容？

(4) UML中的接口类有什么特点，举例说明。

练习：读类图

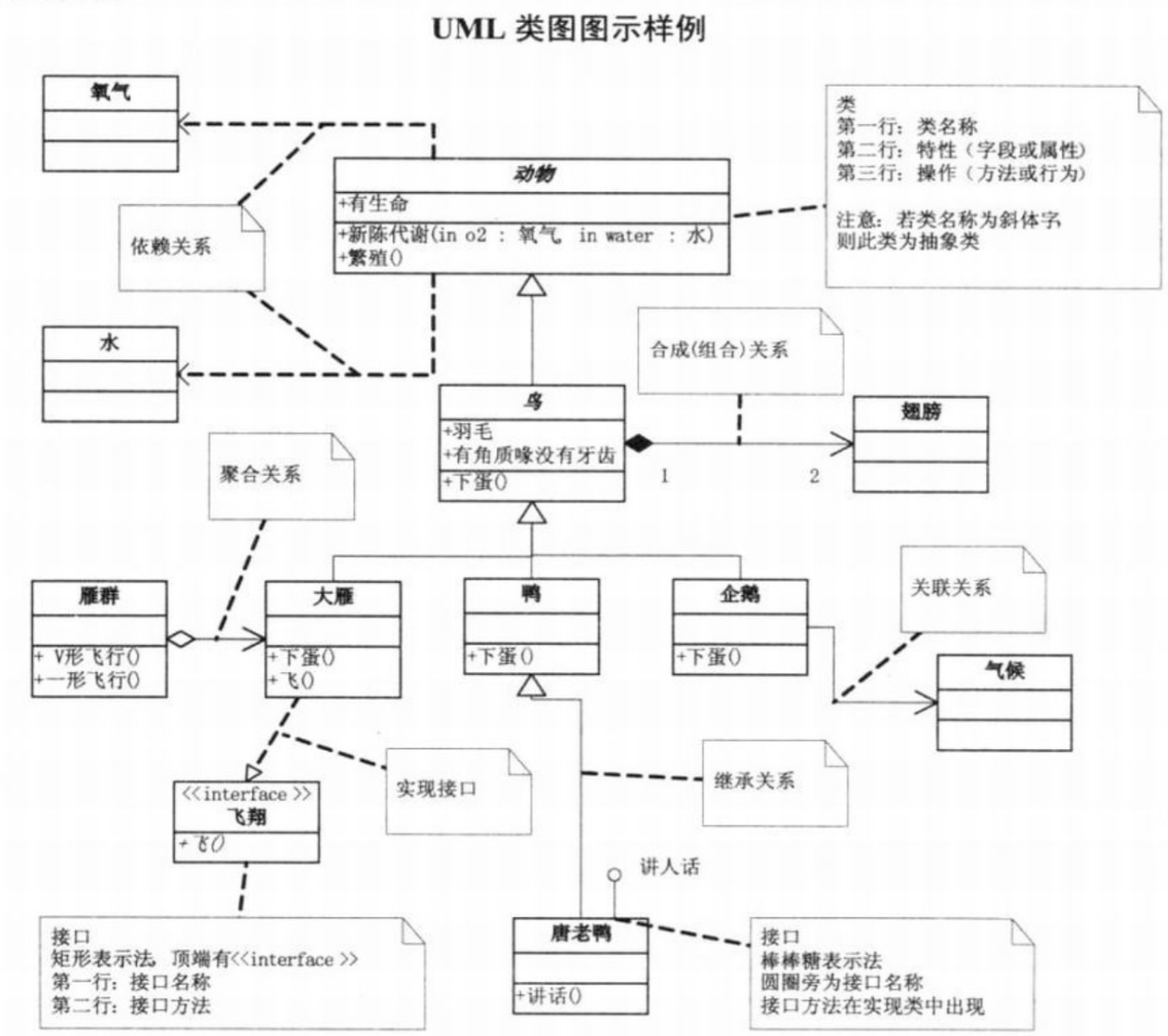
① 解释类图表达的对象及对象之间的关系。

② 重点说明的类：

- 水
- 气候
- 飞翔
- 翅膀

③ 重点说明的关系：

- 动物←→大雁
- 雁群←→大雁
- 唐老鸭←→鸭

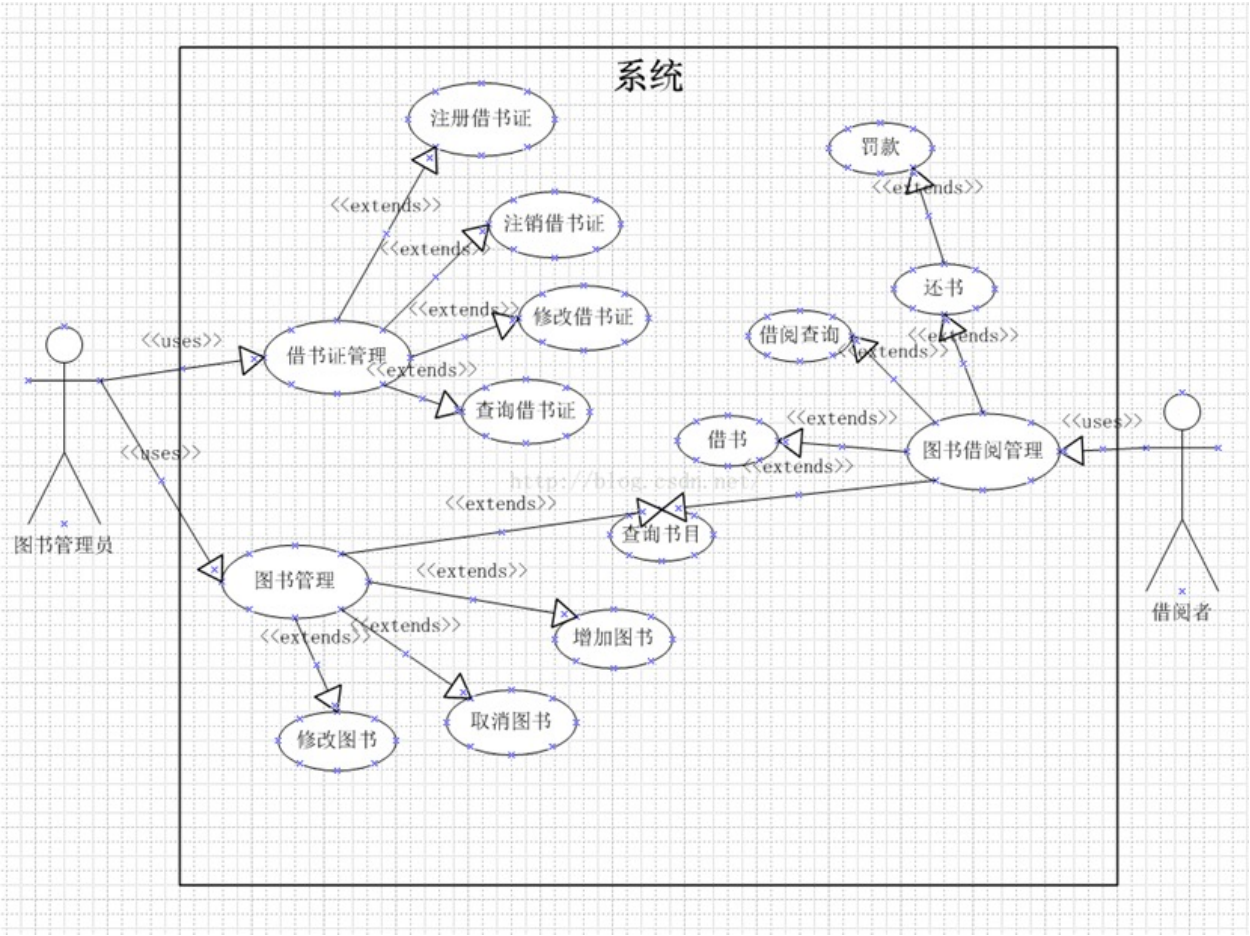


练习题：图书馆管理系统建模

- 图书馆管理系统中，读者包括教师、学生（学生中有本科生、研究生、）等等，图书包括杂志、各类书籍。
- 图书管理员：图书馆工作人员，主要有以下权限：分类管理、借书处理、还书处理、解除预定
- 图书借阅者：是系统中最多也是最重要的参与者，主要有以下权限：查询个人信息、查询图书信息、预定图书、借阅图书、返还图书。
- 要求：

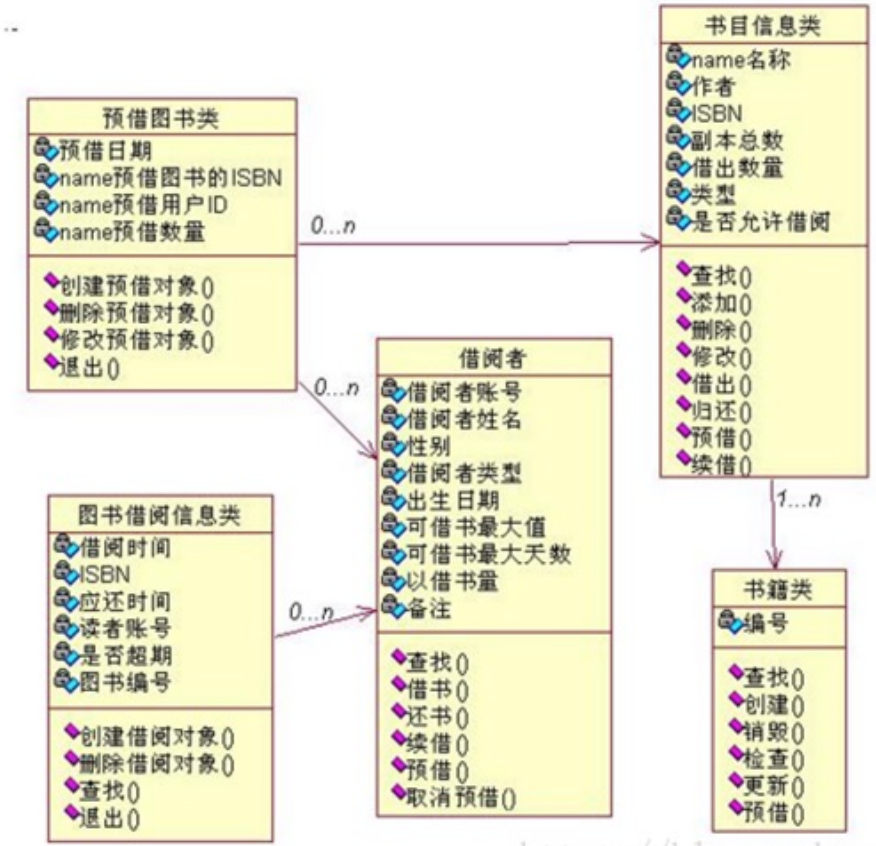
1. 请用UML建立UseCase图
2. 读者和图书的类图
3. 图书的状态图

1. 请用UML建立UseCase图



（续）

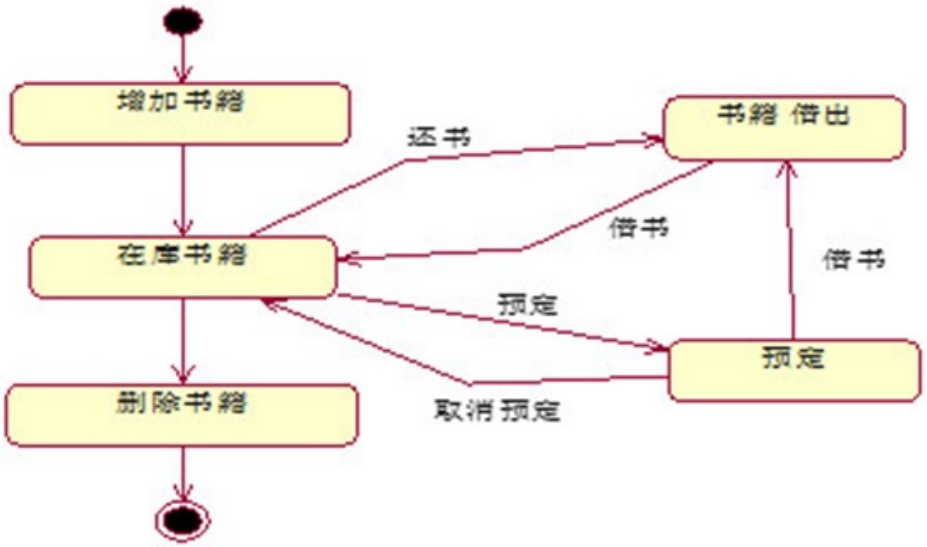
2. 读者和图书的类图



https://blog.csdn.net/abcdefghwelcome

（续）

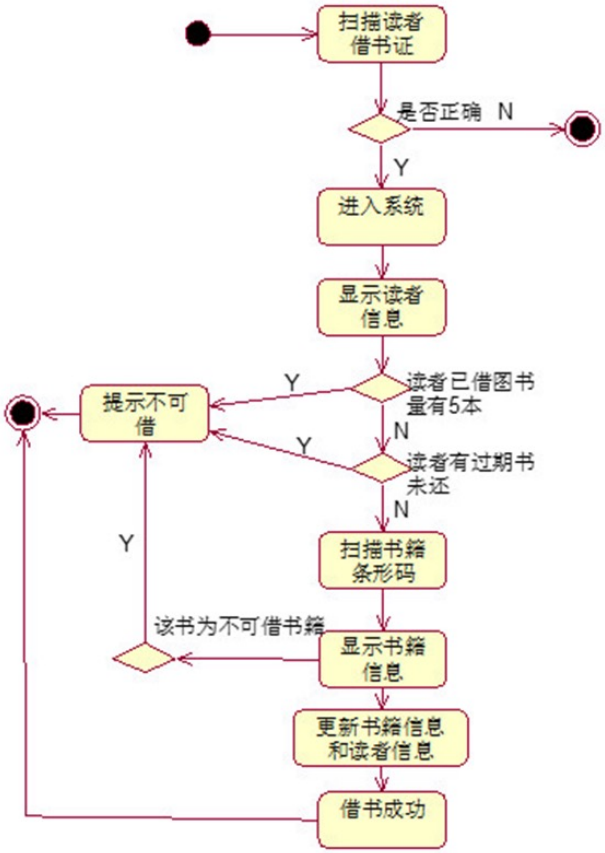
3. 图书的状态图



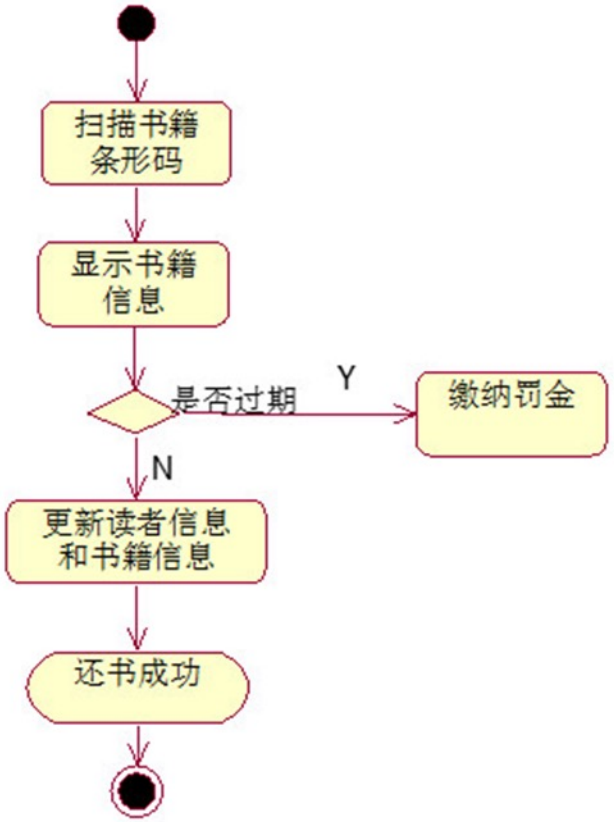
（续）

4. 以及借还书过程的活动图

借书：



还书：

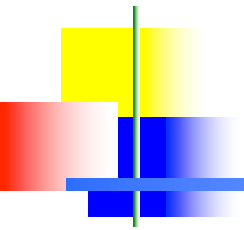


课堂练习：案例分析A——打车软件分析

用OOA方法，进行“打车软件”分析和建模，要求：

① 用OO方法进行系统分析，画出系统的用例图（use-case）：
- 顶层（0层）
- 第1层；

② 用UML图描述系统模型：
- 识别主要的类，定义类的结构和层次
- 建立对象-关系模型，画出类图
- 建立对象-行为模型，画出约车单的状态图、约车成功的活动图、约车单处理过程的顺序图，约车过程的协作图
- 进行系统总体设计，画出打车软件的包图、构件图、部署图。



面向对象方法

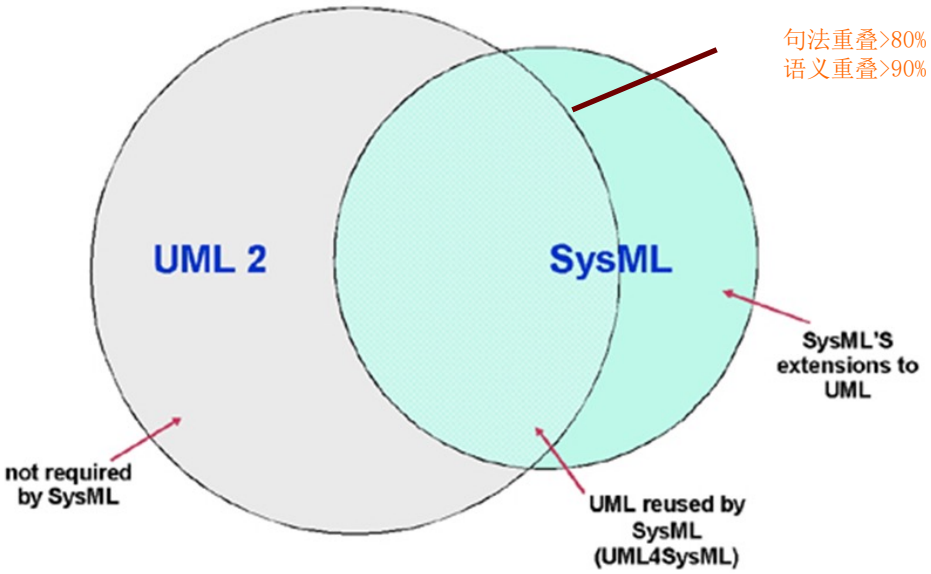
8 系统建模语言SysML

SysML – Systems Modeling Language

- 对象管理组织OMG（Object Management Group）在对UML2.0的子集进行重用和扩展的基础上，提出一种新的系统建模语言——SysML（Systems Modeling Language），作为系统工程的标准建模语言，目的是统一系统工程中使用的建模语言。
- SysML支持各种系统和系统对系统的规范、分析、设计、验证和确认。这些系统可能包括硬件、软件、信息、流程、人员和设施。

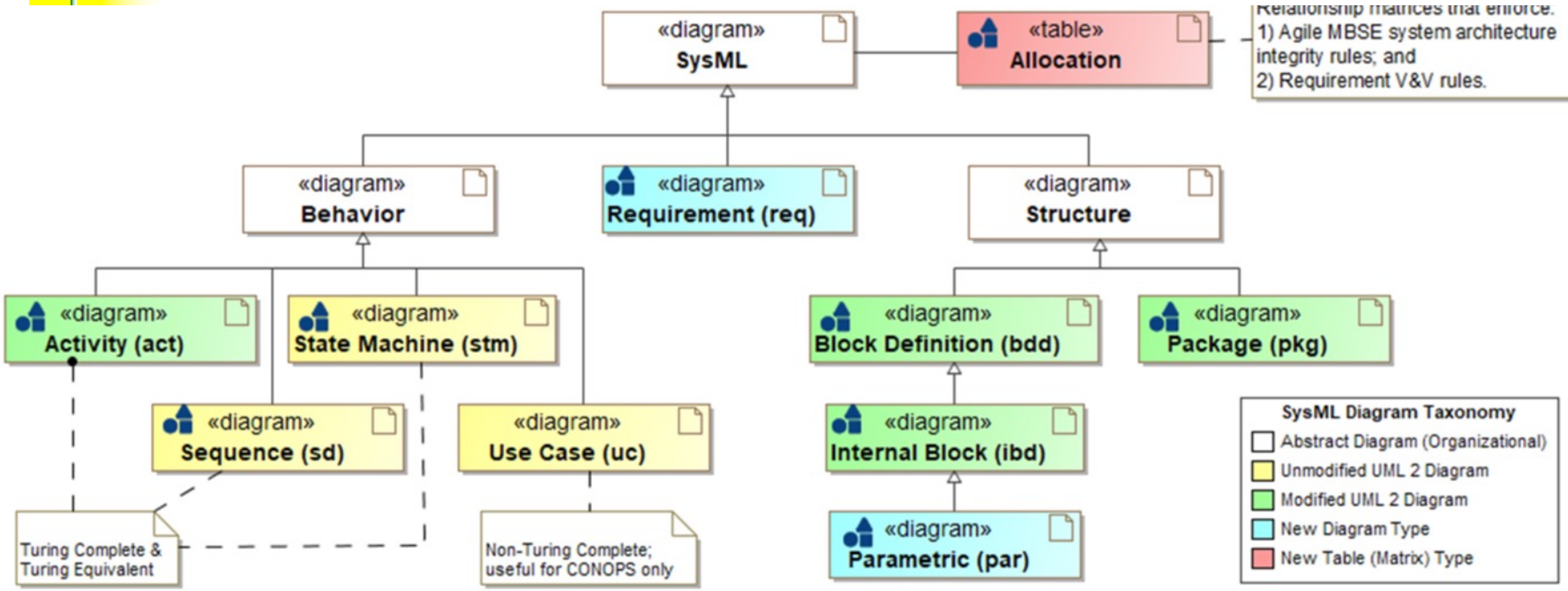
SysML 与 UML 2 的关系

- SysML是用于基于模型的系统工程（MBSE， Model-based Systems Engineering ）的赋能技术。
- SysML 是 UML 2 的子语言，UML for SE (System Engineering），定义为 UML 2 Profile，SysML的SysML最初于2003年由SysML Partners的SysML开源 规范化项目创建。2006年，SysML被对象 管理组（OMG）改编并采用为OMG SysML。



SysML和UML之间的关系

SysML中的图



- SysML 的图形表示是可视化表示，是用来为系统建模的工具。
- SysML 定义了9种基本图形来表示模型的各个方面。从模型的不同描述角度来划分，这些基本图形分成四类：
  - 结构图(Structure Diagram)，包括类图(Class Diagram) 和装配图(Assembly Diagram)
  - 参数图(Parametric Diagram)
  - 需求图(Requirement Diagram)
  - 行为图(Behavior Diagram)，包括活动图(Activity Diagram)、顺序图(Sequence Diagram)、时间图(Timing Diagram)、状态机图(State Machine Diagram)和用例图。